



人工智能原理与算法

第19章-样例学习

中科院自动化研究所 朱翔昱

样例学习

■ 样例学习定义

- **定义**：智能体通过对世界进行观测来提高他的求解问题的能力。
- 英文为**Learning From Examples**，也就是从观察到的例子中学习。
- 当智能体是计算机时，就称为**机器学习**：一台计算机观测到一些**数据**，基于这些数据构建一个模型，并将这个模型作为关于世界的一个**假设**以及用于求解问题。（数据和假设是样例学习的核心概念）

样例学习

■ 为什么要学习？

- 我们希望计算机能解决复杂问题，这些问题 **(1)** 有大量可能出现的未知情形； **(2)** 难以设计固定的程序进行求解。
 1. 设计者不能预测智能可能驻留的所有情境，例如走迷宫的机器人必须学习它所遇到的新迷宫的布局
 2. 有时候人类设计者本身对程序求解没有思路。例如，人类大多善于识别家庭成员的面相，但使用的是潜意识，所以即使是最好的程序员也不能编写出完成此任务的计算机程序，除非使用学习算法。

- **学习形式**
- **监督学习**
- **决策树学习**
- **模型选择与模型优化**
- **学习理论**
- **线性回归和分类**
- **非参数化模型**
- **集成学习**
- **人工神经网络**
- **开发机器学习系统**

19.1 学习形式

■ 学习的三个主要因素

- 组件：哪些组件可以被改进
- 先验：智能体应当有哪些先验知识
- 数据：有哪些数据，以及关于这些数据的反馈

19.1 学习形式

■ 学习的三个主要因素

- **组件**：哪些组件可以被改进
- **先验**：智能体应当有哪些先验知识
- **数据**：有哪些数据，以及关于这些数据的反馈

本章假设智能体从0学习

19.1 学习形式

■ 智能体的组件：

1. **决策：从当前状态条件到动作的直接映射**（在什么情景下，应该做什么事）
2. **感知：用于从感知序列推断世界相关性质的方法**（从感知到的信息，估计世界处于一个什么样的状态）
3. **预测：关于世界演化方式和关于智能体动作导致的结果信息**（如果做了某些动作，会导致什么结果）
4. **效用：表明状态意向的效用信息**（世界的每个状态，价值有多大）
5. **动作效用：表明动作意向的动作-价值信息**（每个动作所能产生的价值）
6. **目标：最大化智能体效用的世界状态的集合，即目标**（最希望达到的状态）
7. **学习元素：问题生成器、评判标准和使系统得以改进的学习元素**

19.1 学习形式

■ 示例：出租车司机智能体

- 当教练每次喊出“刹车”指令时，可以学习一个关于何时刹车的“条件-动作”规则。（组件1-决策）
- 某些情况下教练不叫喊时，智能体也可以学习。通过查看含有巴士的大量图像，它能够识别它们。（组件2-感知）
- 通过尝试动作并观察结果——例如在潮湿的路面上急刹车——它可以学习动作的效果。（组件3-预测）
- 进一步，当乘客在旅途因为颠簸而抱怨，它没能得到小费时，它可以学习到总体效用函数的有效信息。（组件4-效用）

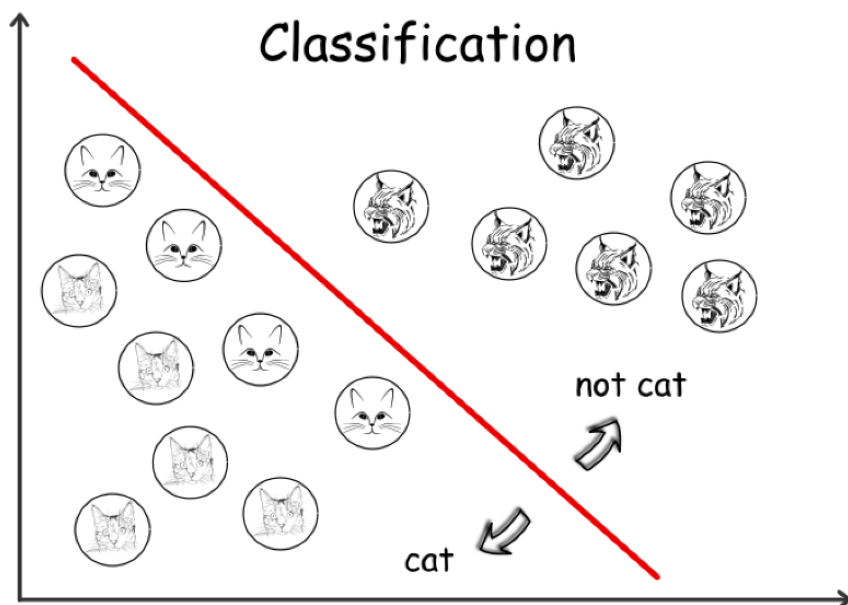
19.1 学习形式

■ 学习的原料：数据

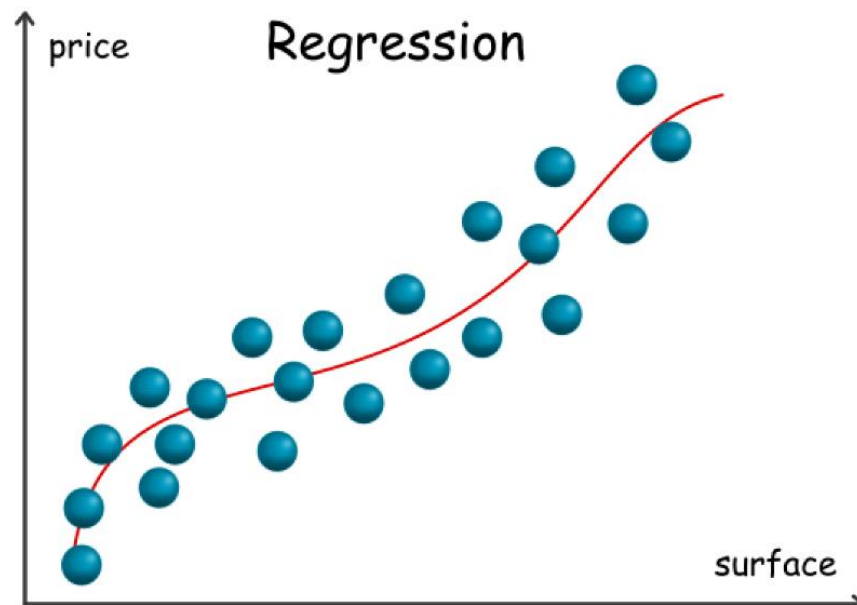
- 样例：[输入-输出] 对
- 输入形式：默认样例的输入是因子化表示的，即输入表示为属性值组成的向量。
- 输出形式：样例的输出**格式**随着具体的问题而有所不同。
 - ▶ **分类问题**：模型的输出是一个有限集合中的某个值（如晴天/雨天，True/False）。
 - ▶ **回归问题**：输出是一个具体的数值（温度，湿度）
- 用于学习假设的数据称为**训练集**。

19.1 学习形式

■ 学习的原料：数据



分类：猫or非猫



回归：房产价格

19.1 学习形式

■ 学习的类型：

伴随样例的输入，存在三种反馈形式（输出），对应于三种不同类型的学习：

- **监督学习**：智能体接受输入信号以及人为给定的输出信号（称为标签），并学习由输入到输出的映射函数。（人为给出标准答案）
- **无监督学习**：智能体仅获取输入信号，并自行发掘数据内部的规律。最典型的无监督学习是**聚类**。（从互联网的百万个图像中找到一组相似的图像——猫）
- **强化学习**：根据智能体的行为提供奖励/惩罚，智能体学习如何最小化惩罚、最大化奖励。（激励函数：做的好，要奖赏，做的差，要惩罚）

- 学习形式
- 监督学习
- 决策树学习
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

19.2 监督学习

■ 监督学习的任务形式

- 给定一个**训练集**，其中包含 N 个“输入-输出对”样例

$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$$

这些数据通过一个未知的**真实函数** $y = f(x)$ 生成， y 称**真实数据**。

- **目标**：寻找一个函数 h 来近似 f 并预测 y 。函数 h 就是关于世界的**假设**，也可以称其为关于数据的**模型**，我们希望这个模型能预测正确的答案。
- h 取自一个包含所有可能函数的 $\mathcal{H}$ ， $\mathcal{H}$ 被称为**假设空间**或者**模型类**。
- Ground Truth：绝对**真实**，绝对正确的信息

19.2 监督学习

■ 寻找好的假设

- **寻找一致性假设**：假设 h ，对于**训练集**中任意样例对 (x_i, y_i) ，都有：

$$h(x_i) = y_i$$

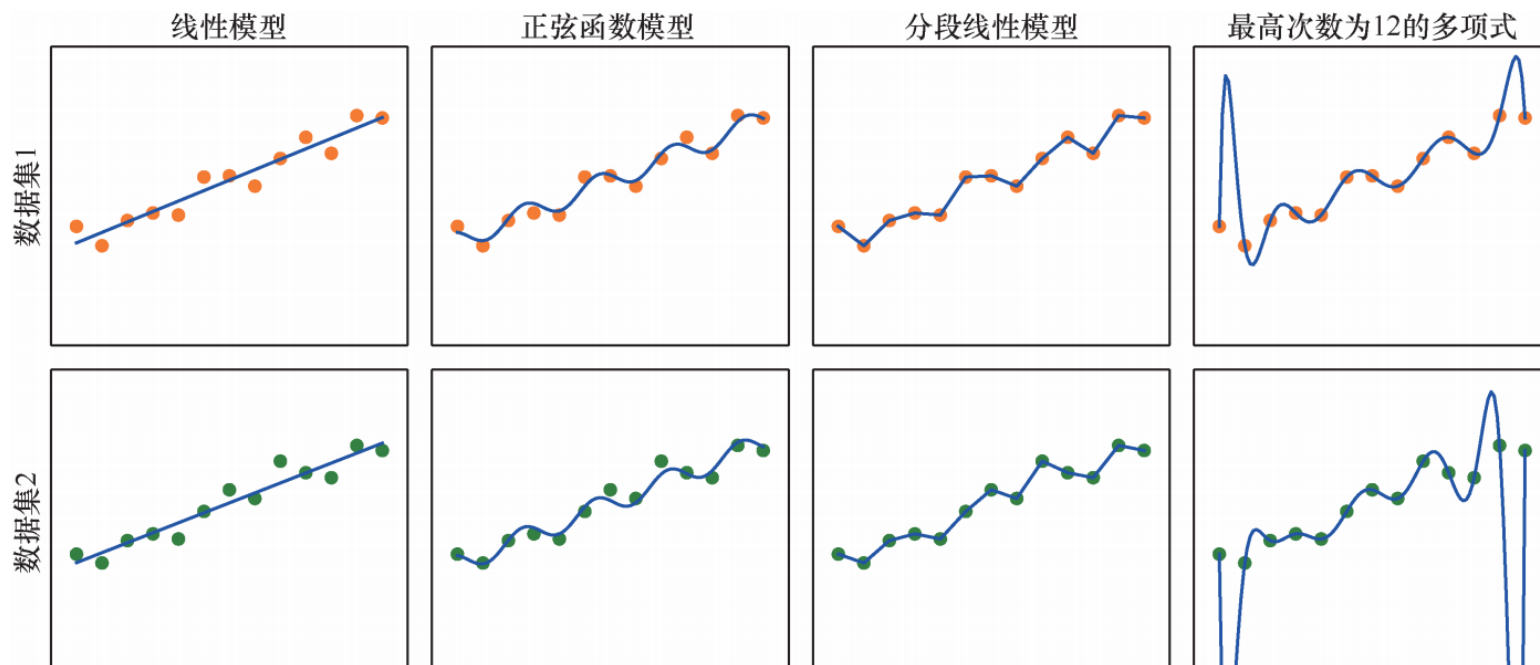
- ▶ 如果模型的输出是连续值，就不能期望 $h(x_i)$ 精确地等于 y_i ，而是要找一个最佳拟合函数，使 $h(x_i)$ 与 y_i 非常接近。
- ▶ 一致性就是能够完美地预测我们已经观测到的数据。

- **寻找泛化能力强的假设**：衡量一个假设的标准不是看它在训练集上的表现，而取决于它如何处理尚未观测到的输入：

- ▶ 使用一个**测试集**（第二组样本对 (x_i, y_i) ）来评估假设
- ▶ 如果 h 准确地预测了测试集的输出， h 具有很好的泛化能力

19.2 监督学习

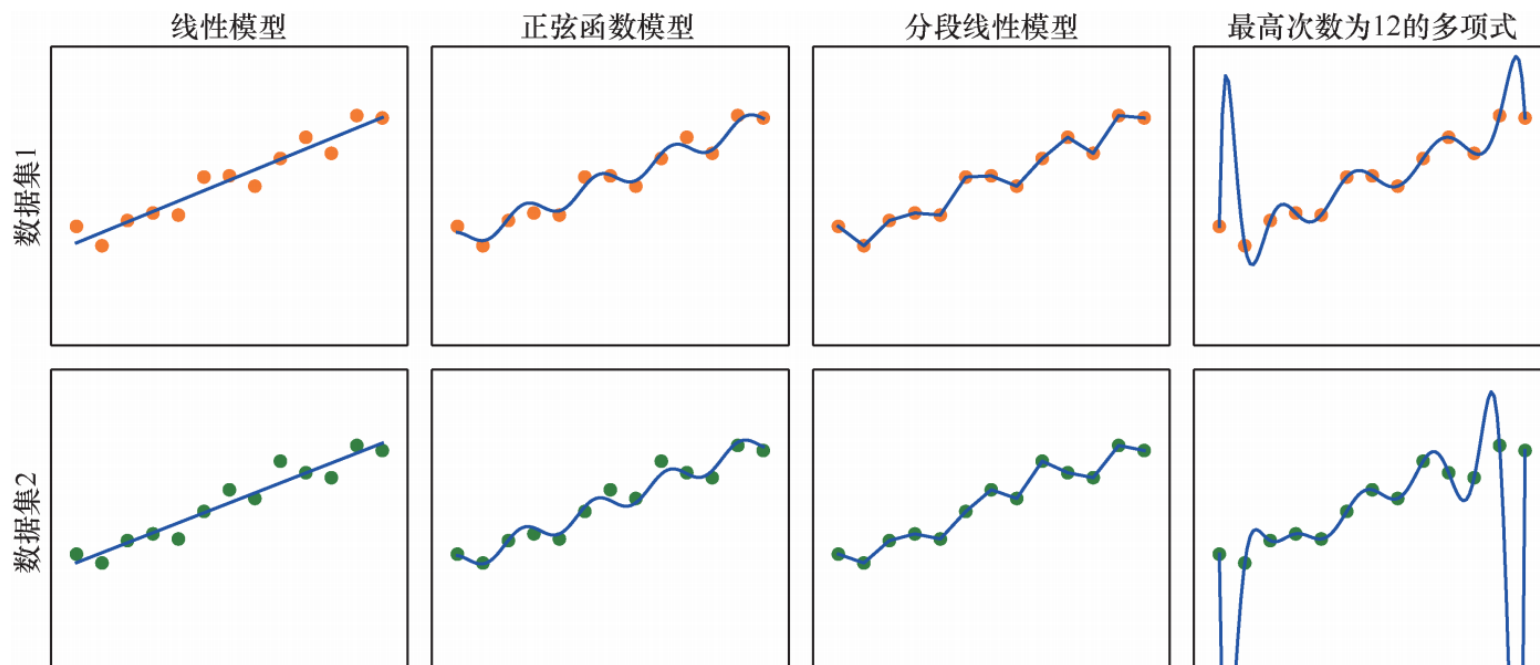
■ 示例：曲线拟合



- 每行的使用相同的训练集，点 (x, y) 的位置相同。
- 第一行和第二行采样自相同的真实函数 f 。
- 每一列代表一个假设空间：直线、正弦曲线、分段直线、多项式曲线
- 每条曲线都是训练集的假设（模型） h

19.2 监督学习

■ 示例：曲线拟合

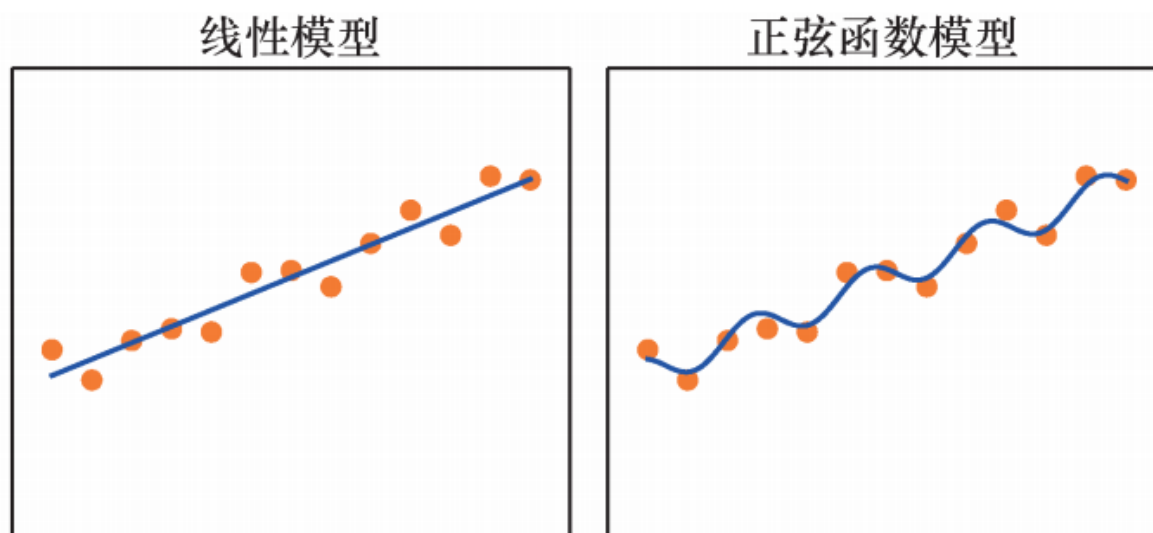


- 线性模型：不存在一致的曲线（无法拟合训练集）。
- 正弦模型：虽然没有完全一致，但较好拟合了训练集。
- 分段线性模型：一致的，完全拟合训练集，泛化能力强。
- 12次多项式：一致的，完全拟合训练集，泛化能力弱。

19.2 监督学习

■ 寻找好的假设：偏差与方差

- 偏差：假设所预测的值偏离期望值的平均趋势。
- 偏差常常是由假设空间所施加的约束造成的，用来评估一致性。
- 当一个假设不能找到数据中的模式时，称为是欠拟合（underfitting）。

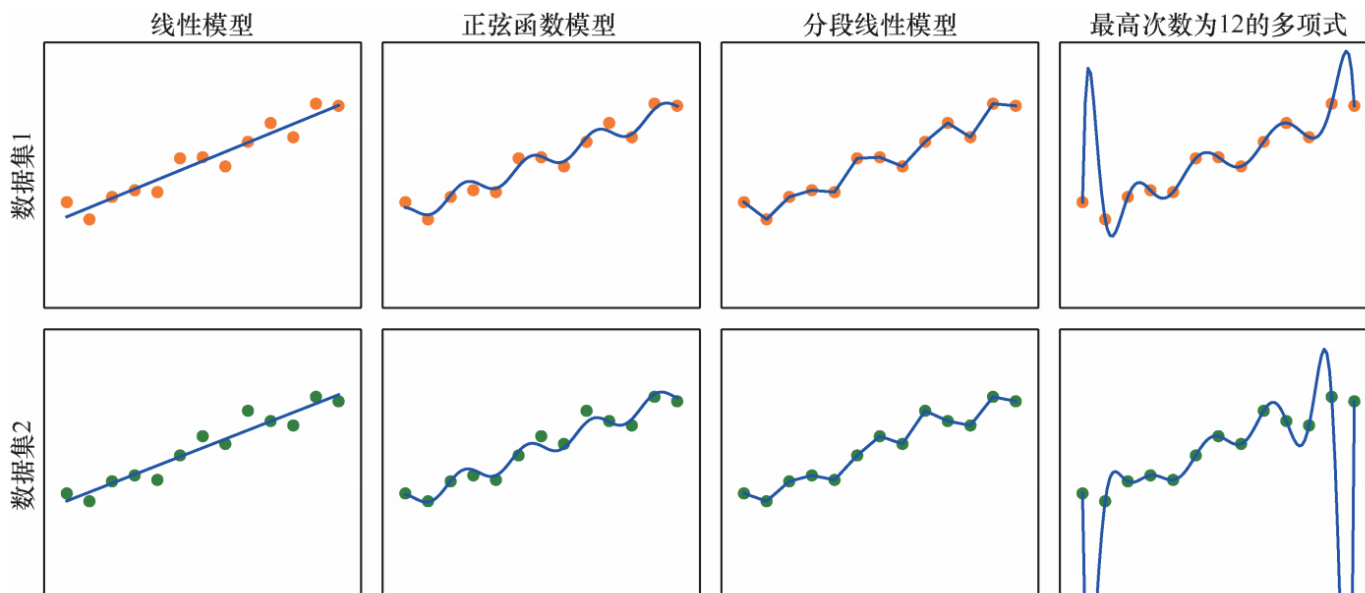


直线无法反映除斜率外的其他模式。
相比正弦函数，线性函数无法很好地拟合给出的点数据集

19.2 监督学习

■ 寻找好的假设：偏差与方差

- 方差：指由训练数据波动而导致假设的变化量。
- 当一个函数过于关注它用来训练的特定训练集，进而导致它在没有见过的数据上表现较差时，称该函数对数据集是过拟合的



对两组稍有不同的数据点进行拟合，前三种得到的假设变化不大（低方差）；而最后一组变化较大（高方差）。

19.2 监督学习

■ 寻找好的假设：偏差-方差权衡

- 通常来说，假设越复杂偏差越低，假设越简单方差越低。
- **偏差-方差权衡**：在**更复杂、低偏差**的能较好拟合训练集的假设与**更简单、低方差**的可能泛化得更好的假设中做出选择。
- **奥卡姆剃刀原则**：“如无必要，勿增实体”
 - ▶ 选择与数据相符的**最简单假设**。
 - ▶ **Simpler is Better**

19.2 监督学习

■ 寻找好的假设：偏差-方差权衡

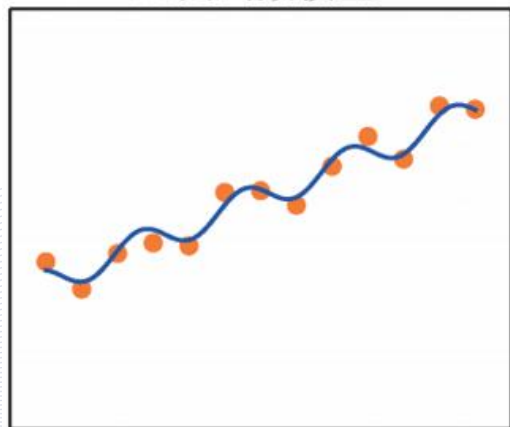
- 监督学习可以通过选择在数据集上发生概率最大的假设 h^* 来实现

$$h^* = \underset{h \in \mathcal{H}}{\operatorname{argmax}} P(h|\text{data})$$

- 根据贝叶斯法则，上式等价于

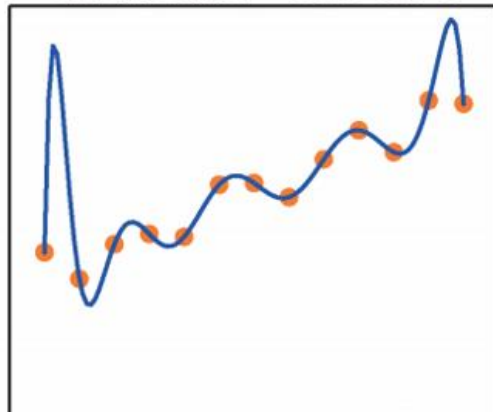
$$h^* = \underset{h \in \mathcal{H}}{\operatorname{argmax}} P(\text{data}|h) \overset{\text{偏差}}{P(h)}$$

正弦函数模型



vs.

最高次数为12的多项式



光滑的一次或二次多项式的先验概率较高；较大波动的12次多项式概率较低。

19.2 监督学习

■ 示例：餐厅等待问题

假设有一家餐厅，一名顾客正在其中排队等待就餐，一系列的外部因素决定了他是否会继续等待下去，

□ 样例输入 x (外部因素) 包括：

- (1) 候补 (Alternate)：附近是否有其他可替代的餐厅 (Yes/No)
- (2) 吧台 (Bar)：餐厅是否有舒适的吧台用于等待 (Yes/No)
- (3) 周五/六 (Fri/Sat)：现在是否是周五/六 (Yes/No)
- (4) 饥饿 (Hungry)：顾客现在是否饥饿 (Yes/No)
- (5) 顾客 (Patrons)：餐厅中现在有多少顾客 (None、Some、Full)
- (6) 价格 (Price)：餐厅的价格范围 (\$、\$\$、\$\$\$)
- (7) 下雨 (Raining)：外面是否在下雨 (Yes/No)
- (8) 预约 (Reservation)：是否预定了位置 (Yes/No)
- (9) 种类 (Type)：餐厅的种类 (French、Italian、Thai、Burger)
- (10) 预计等待时间 (WaitEstimate)：(0~10, 10~30, 30~60, >60)

□ 样例输出 y 是一个布尔值WillWait：输出真时，顾客继续等待位置；反之则不再等待。

19.2 监督学习

■ 示例：餐厅等待问题，属性空间9204，样例数12

样例	输入属性										输出
	<i>Alternate</i>	<i>Bar</i>	<i>Fri/Sat</i>	<i>Hungry</i>	<i>Pattrans</i>	<i>Price</i>	<i>Raining</i>	<i>Reservation</i>	<i>Type</i>	<i>WaitEstimate</i>	<i>WillWait</i>
x_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0 ~ 10	$y_1 = \text{Yes}$
x_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30 ~ 60	$y_2 = \text{No}$
x_3	No	Yes	No	No	Some	\$	No	No	Burger	0 ~ 10	$y_3 = \text{Yes}$
x_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10 ~ 30	$y_4 = \text{Yes}$
x_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	> 60	$y_5 = \text{No}$
x_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0 ~ 10	$y_6 = \text{Yes}$
x_7	No	Yes	No	No	None	\$	Yes	No	Burger	0 ~ 10	$y_7 = \text{No}$
x_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0 ~ 10	$y_8 = \text{Yes}$
x_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	> 60	$y_9 = \text{No}$
x_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10 ~ 30	$y_{10} = \text{No}$
x_{11}	No	No	No	No	None	\$	No	No	Thai	0 ~ 10	$y_{11} = \text{No}$
x_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30 ~ 60	$y_{12} = \text{Yes}$

- 学习形式
- 监督学习
- **决策树学习**
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

19.3 决策树学习 - 决策树的表达能力

■ 决策树的概念

- 定义：将属性值向量映射到单个输出值的函数。
- 通过执行一系列测试完成决策：它从根节点出发，在每个节点处对输入属性进行一次测试，并根据测试结果前往对应的分支，直到达到叶节点。叶节点指定了函数最终返回的值。
- 决策树的优点在于可解释性，与人类思维类似。如许多指南手册就是按决策树的形式书写的。

■ 决策树的表示法：

- 输入值和输出值既可以是离散的，也可以是连续的。
- 布尔分类：输入值是离散，输出值为二值（True or False）。

19.3 决策树学习 - 从样例中学习决策树

■ 示例：餐厅等待问题

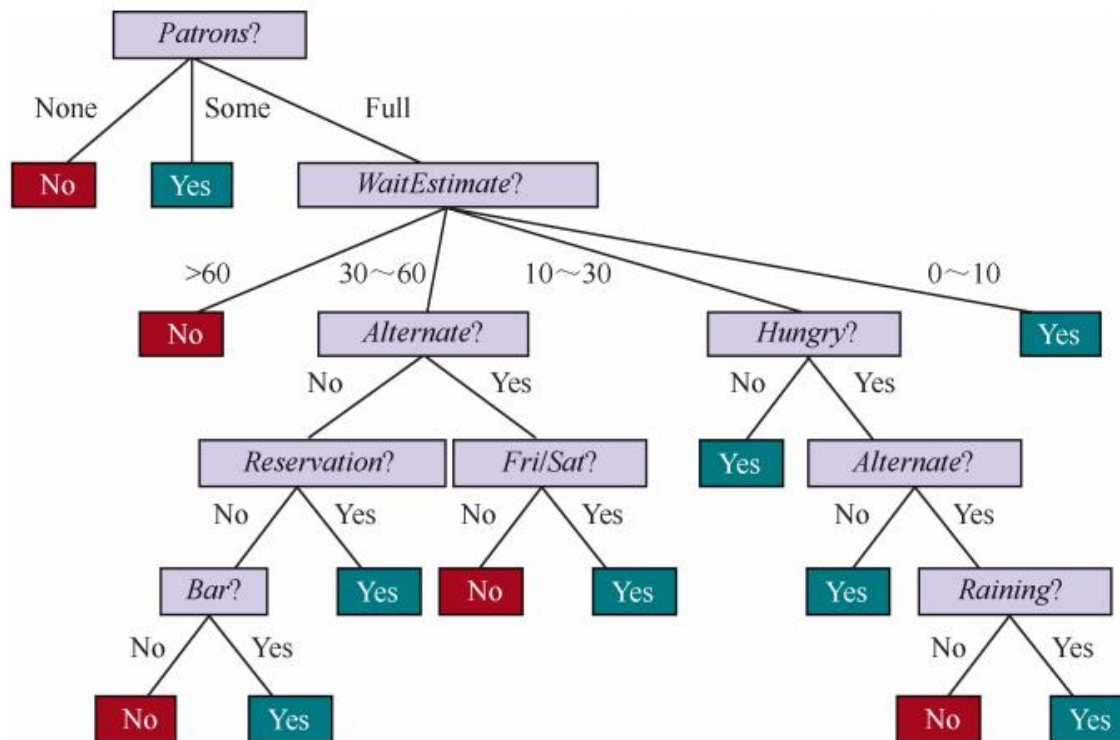


图 19-3 决定是否在餐厅等待的决策树

在每个节点处对输入属性进行一次测试，并根据测试结果前往对应的分支

19.3 决策树学习 - 从样例中学习决策树

■ Learn-Decision-Tree算法:

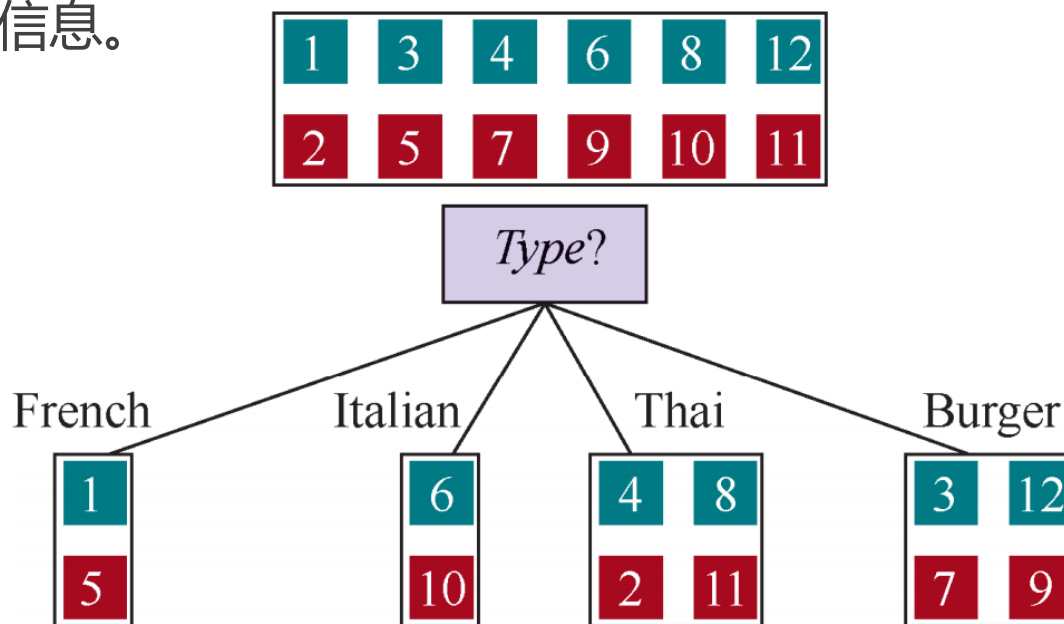
我们希望找到一颗**与样例一致且尽可能小**的决策树。理论最小的树难以发现，但可以通过一些启发式方法，通过“**贪心与分治**”策略高效地搜索到一颗接近最小的决策树。

- 每次测试时，我们选出当前“**最重要**”的属性进行测试。根据测试结果产生的子问题，**递归**地继续选出最重要的属性，从而解决子问题。
- “**最重要**”：对分类结果产生最大影响的属性。（后面介绍）
- 通过每次选取最重要的属性，我们能用最可能少的测试来得到正确的分类结果，整棵树所有路径都比较短，整棵树的层数也较浅。

19.3 决策树学习 - 从样例中学习决策树

■ 属性的重要性

餐厅示例：Type（餐厅种类）是不重要的。对于Type的每种可能值，都包含了相同数量的正负样例，因此它目前无法提供任何关于分类结果的有效信息。

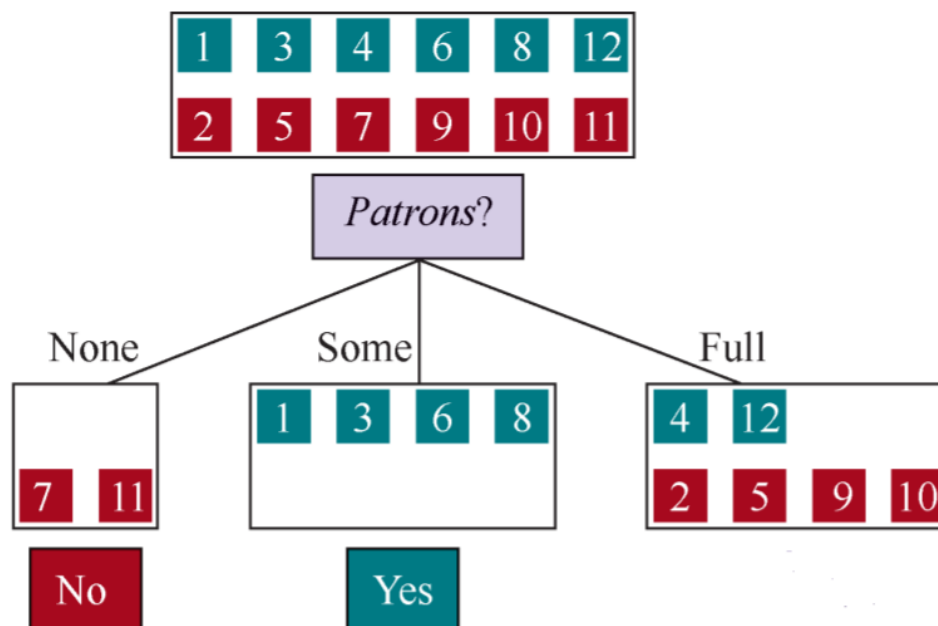


原来分不开，用Type切割后也分不开

19.3 决策树学习 - 从样例中学习决策树

■ 属性的重要性

餐厅示例：Patrons（当前餐馆中人数）是重要的。Patrons取值None和Some时，分别仅包含负/正样例，可以直接给出分类结果。仅在取值Full时，仍有正负样例混合，需要进一步测试。



19.3 决策树学习 - 从样例中学习决策树

■ 属性测试的可能结果

- 分支中样例全为正/负。此时已经达成目标，可以直接输出
- 分支中仍混合正负样例。此时需要继续选取合适的属性进行测试
- 分支中不存在任何样例。说明训练样例中没有对应于这一分支的输入属性组合（训练数据不够或者属性组合太稀有），此时以父节点中最常见的输出值作为输出
- 分支中仍混合正负样例，但已经没有未测试的属性值。这说明存在**输入完全一致，而分类结果不同**的样例，这是可能出现的，例如数据出现错误、问题本身就包含不确定性、存在未观测到的属性，这些都可能導致出现这一情形。

19.3 决策树学习 - 从样例中学习决策树

■ Learn-Decision-Tree伪代码

function LEARN-DECISION-TREE(*examples*, *attributes*, *parent examples*) **returns** 一棵树

if *examples* 为空 **then return** PLURALITY-VALUE(*parent examples*)—— 1. 分支中不存在任何样例。
else if 所有*examples*有相同的分类 **then return** 分类—— 2. 分支中样例全为正/负。
else if *attributes*为空 **then return** PLURALITY-VALUE(*examples*)—— 3. 已经没有未测试的属性值。
else —— 4. 分支中仍混合正负样例。

$A \leftarrow \arg\max_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$

tree $\leftarrow$ 一个以测试*A*为根的新的决策树

for each *A*中的值*v* **do**

$\text{exs} \leftarrow \{e : e \in \text{examples} \text{ and } e.A = v\}$ —— 以*A*的值为参考对剩余样例分割。

$\text{subtree} \leftarrow \text{LEARN-DECISION-TREE}(\text{exs}, \text{attributes}-A, \text{examples})$ *A*的每个值都构造一个子集。

将一个带有标签 ($A = v$) 和子树 *subtree*的分支加入*tree*

return *tree*

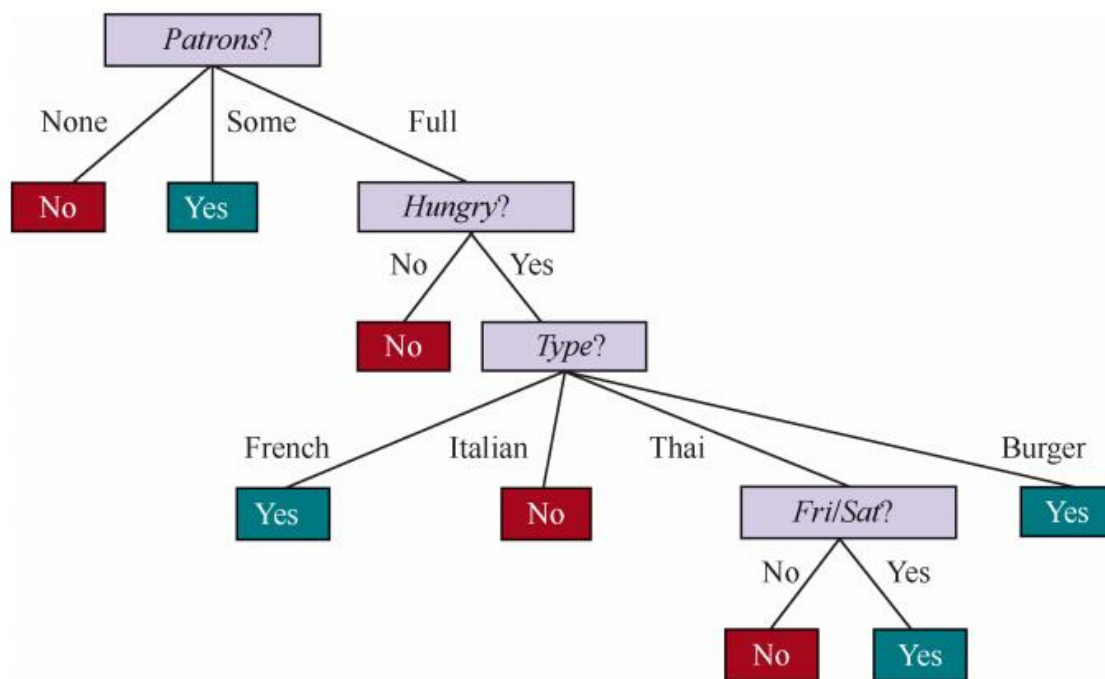
重要性函数 Importance 将在后续给出

函数 Plurality-Value 将选择样例集中最常见的输出

19.3 决策树学习 - 从样例中学习决策树

■ 决策树学习结果

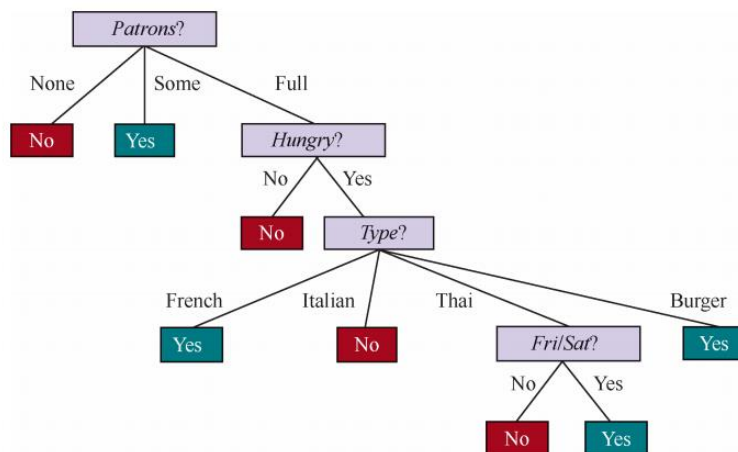
根据上页的伪代码，在餐厅问题中学习到的决策树如下。仅使用全部10个属性中的四个即可完成所有样例的分类。



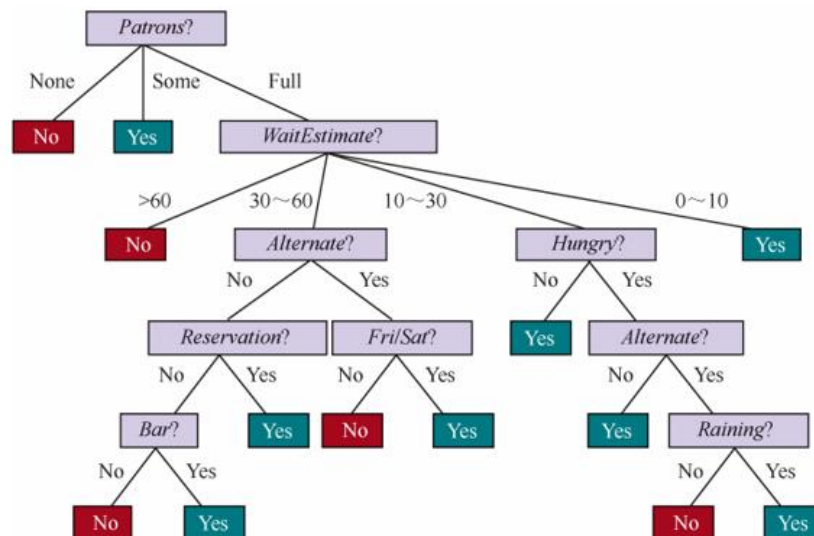
19.3 决策树学习 - 从样例中学习决策树

决策树学习结果

- 在餐厅问题中学习到的决策树如下。仅使用全部10个属性中的四个即可完成所有样例的分类。
- 注意：学习算法仅仅着眼于**分类训练集，而不是解决问题**。即使学习到了更小的树，也不意味着他比人工的更好。



算法

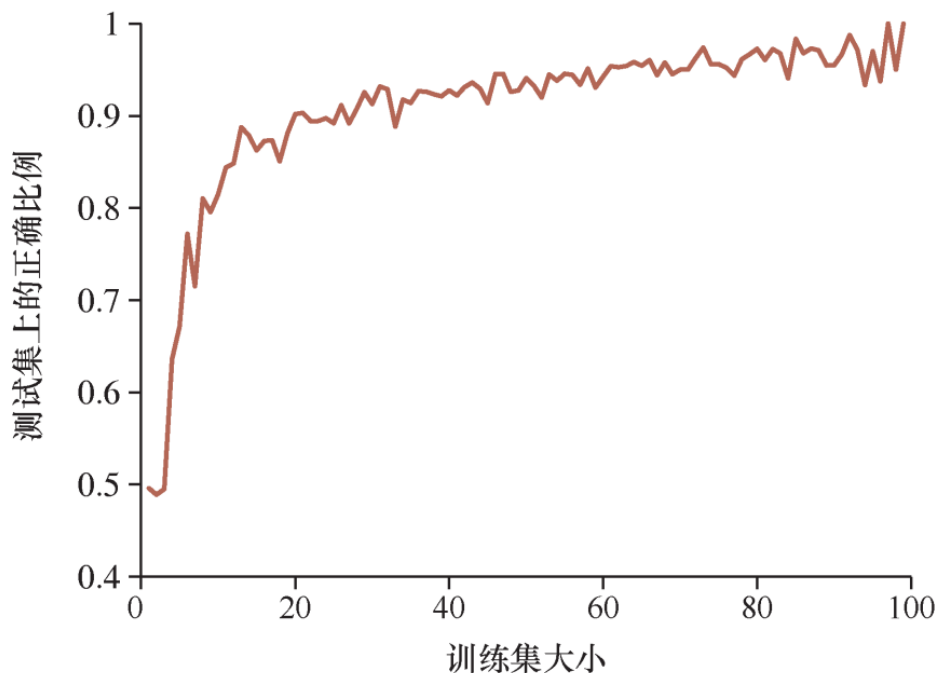


人工

19.3 决策树学习 - 从样例中学习决策树

■ 学习曲线

在以上的例子中，我们仅使用了12个样例来学习一颗决策树，这在现实情况下显然是不够的。就这个餐厅问题来说，所有属性值的组合共有9216种，但我们只知道其中12种组合对应的输出。实际情况下，用于训练的样例越多，预测准确率也将提高，我们可以用学习曲线来评估学习算法的表现：



19.3 决策树学习 - 选择测试属性

■ 熵的定义

- 学习决策树的过程中，“**选出最重要的属性**”是一步关键操作。使用**信息增益**来量化每个属性的重要性。
- 信息增益是从**熵**的角度进行定义的：熵是对不确定性的度量，不确定性越高，熵越大；如果获得信息减小了不确定性，则熵减小。
 - ▶ 一枚公平的硬币在抛掷时出现正面或反面朝上的概率相同，不确定性大，熵较大。
 - ▶ 一枚不公平的硬币，它在 99% 的情况下都是正面朝上，不确定性小，熵较小。

19.3 决策树学习 - 选择测试属性

■ 熵的定义

- 随机变量 V 有 K 个取值，取值为 v_k 的概率为 P_k ，那么它的熵定义为：

$$H(V) = \sum_K P(v_k) \log_2 \frac{1}{P(v_k)} = -\sum_k P(v_k) \log_2 P(v_k)$$

- 一枚公平硬币（50%正面）被抛掷的熵：

$$H(\text{Fair}) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$$

- 一枚不公平硬币（99%正面）被抛掷的熵：

$$H(\text{Loaded}) = -(0.99 \log_2 0.99 + 0.01 \log_2 0.01) \approx 0.08$$

19.3 决策树学习 - 选择测试属性

■ 信息增益

- 对于一个**布尔随机变量**，如果其取值为真的概率为 q ，则它的熵为

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q))$$

- 在学习决策树时，我们将某个属性A的**信息增益**，定义为在对A进行测试后，对于熵的减少的期望值。
- 对属性A测试前，一共有 p 个正样例和 n 个负样例；对A的测试生成了 d 个分支（即A有 d 种取值），其中第 k 个分支包含 p_k 个正样例和 n_k 个负样例，则：

测试前的熵： $B(\frac{p}{p+n})$

测试后的熵：

$$\sum_{k=1}^d \frac{p_k + n_k}{p + n} B(\frac{p_k}{p_k + n_k})$$

第 k 个分支的熵

第 k 个分支的概率

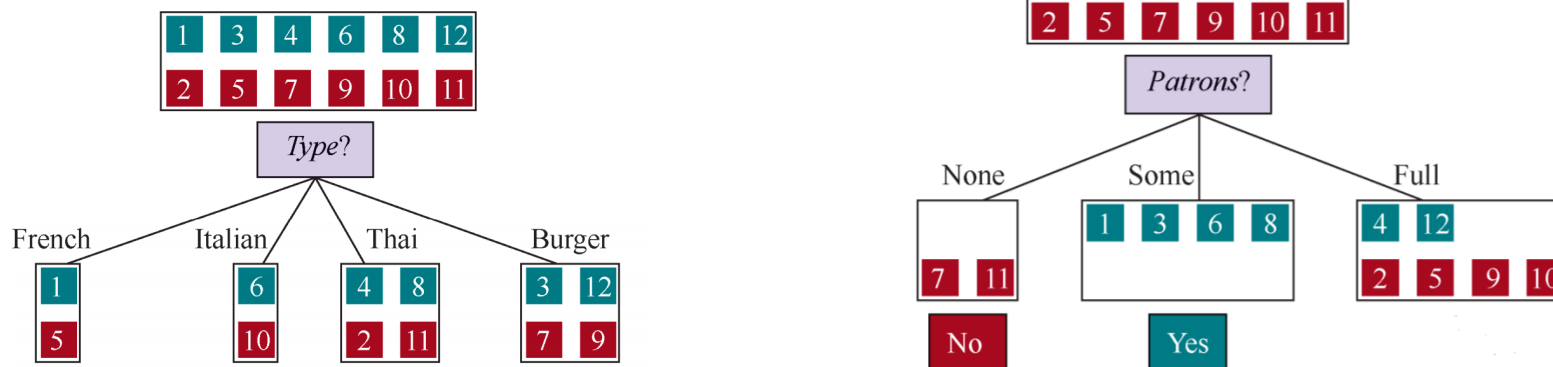
19.3 决策树学习 - 选择测试属性

■ 信息增益

- 属性A测试前后熵减少的期望值，即A的信息增益为

$$Gain(A) = B\left(\frac{p}{p+n}\right) - \sum_{k=1}^d \frac{p_k + n_k}{p+n} B\left(\frac{p_k}{p_k + n_k}\right)$$

- 以餐馆例子中，Type和Patron两个属性的信息增益，说明Patron确实是“重要”的属性：



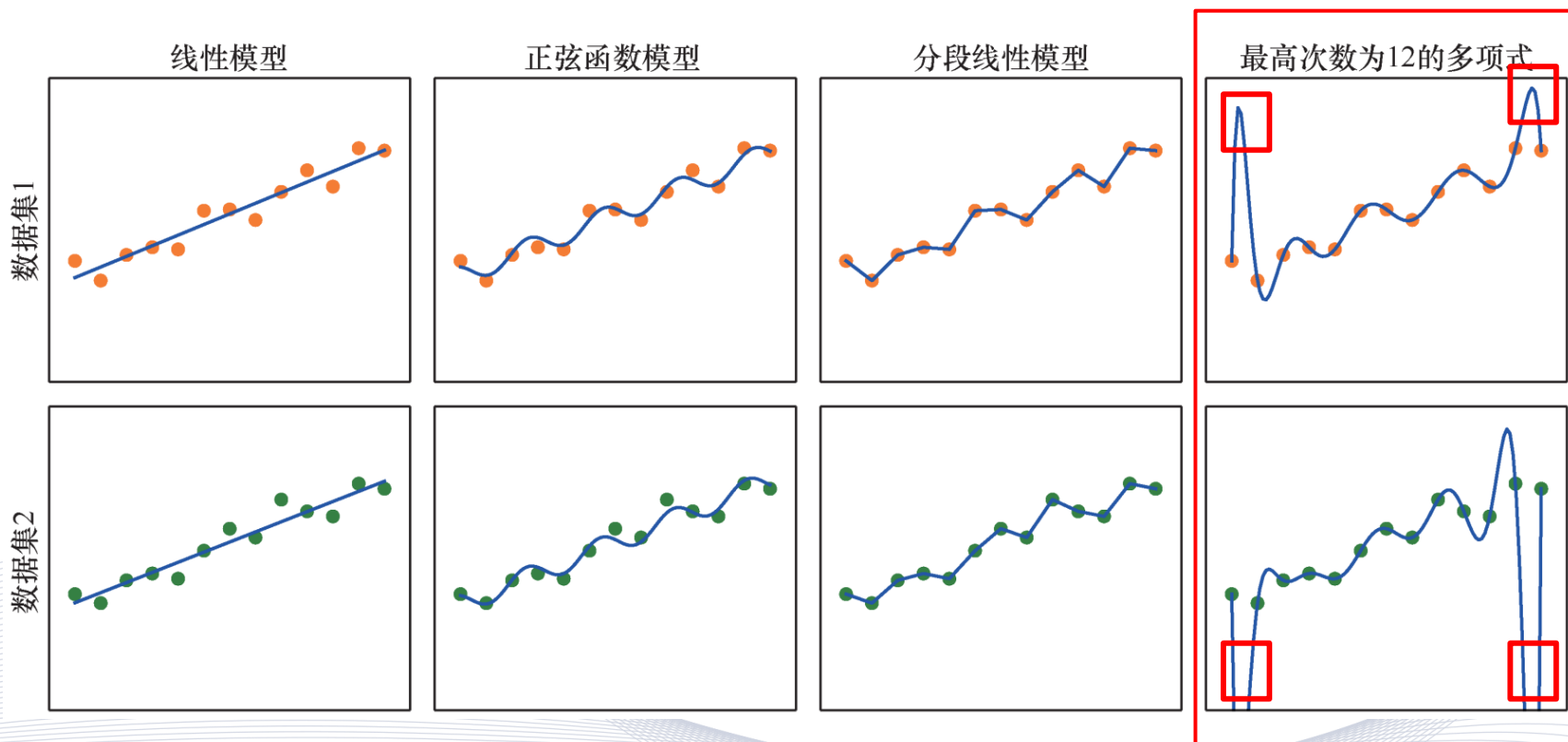
$$Gain(type) = B\left(\frac{6}{12}\right) - \left(\frac{2}{12} B\left(\frac{1}{2}\right) + \frac{2}{12} B\left(\frac{1}{2}\right) + \frac{4}{12} B\left(\frac{2}{4}\right) + \frac{4}{12} B\left(\frac{2}{4}\right)\right) = 0$$

$$Gain(patron) = B\left(\frac{6}{12}\right) - \left(\frac{2}{12} B\left(\frac{0}{2}\right) + \frac{4}{12} B\left(\frac{4}{4}\right) + \frac{6}{12} B\left(\frac{2}{6}\right)\right) = 0.541$$

19.3 决策树学习 - 泛化与过拟合

■ 决策树的泛化性

- **过拟合**：虽然希望学习算法找到一个能够吻合训练数据的假设，但更重要的是，**希望它能很好地推广到还没有被观测到的数据上。**

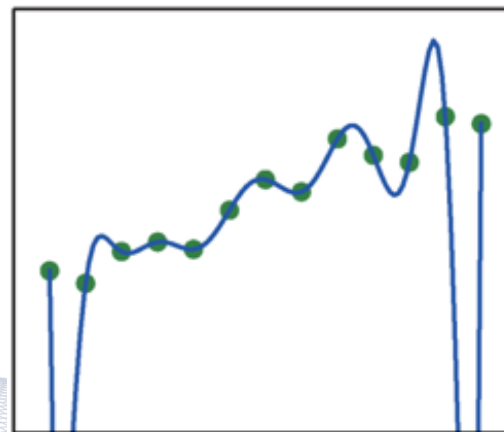
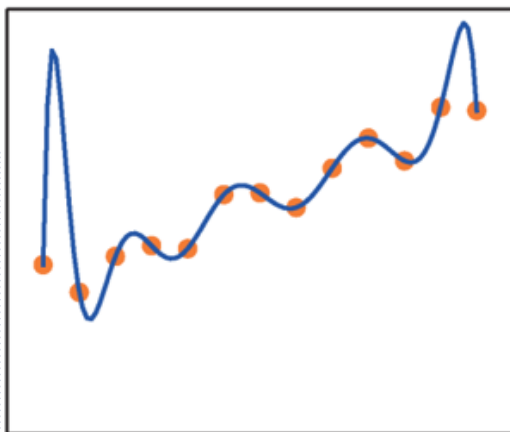


不合理

19.3 决策树学习 - 泛化与过拟合

■ 导致过拟合的要素

- **样例维度**：属性数量（输入维度）越多，过拟合的可能性越大。
- **数据量**：随着训练样例数量的增加，过拟合的可能性会越来越小。
- **模型复杂度**：较大的假设空间（例如，具有更多结点的决策树或具有更高阶的多项式空间）具有更强的拟合和过拟合能力，某些模型类比其他模型类更容易过拟合

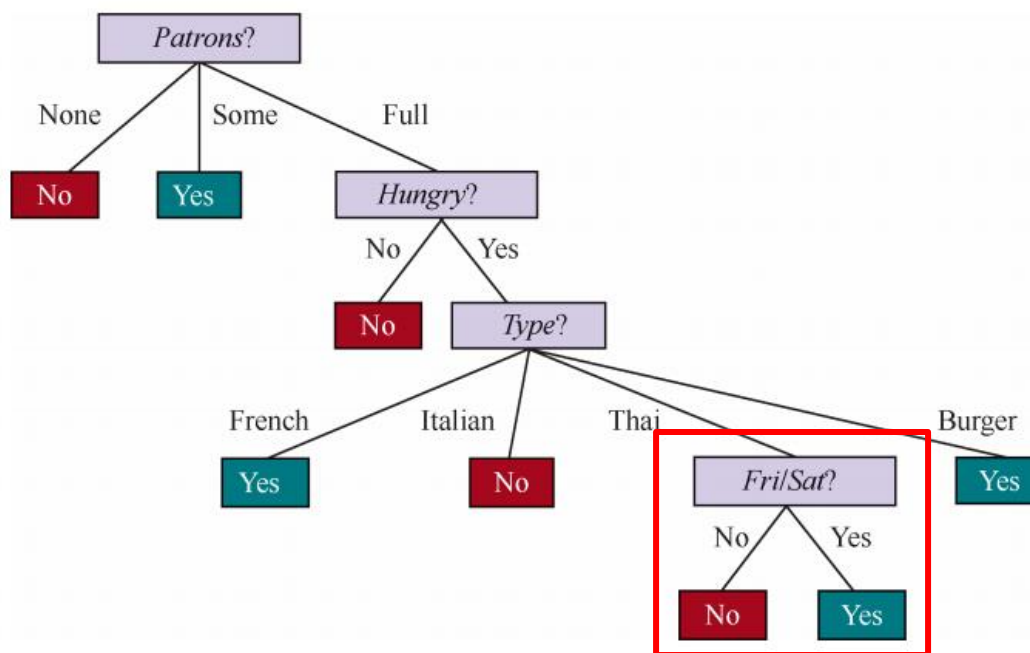


19.3 决策树学习 - 泛化与过拟合

■ 决策树剪枝

使用决策树剪枝 (pruning) 删去与目标无关的属性测试。

- 首先通过之前介绍的Learn-Decision-Tree算法生成初始决策树
- 取**只有叶节点的子节点**，并研究该节点所测试的属性与最终的分类结果是否**相关**。若不相关，则删除该节点，并直接返回该节点最常出现的类别。重复该步骤，直到删除了所有不相关的测试节点。



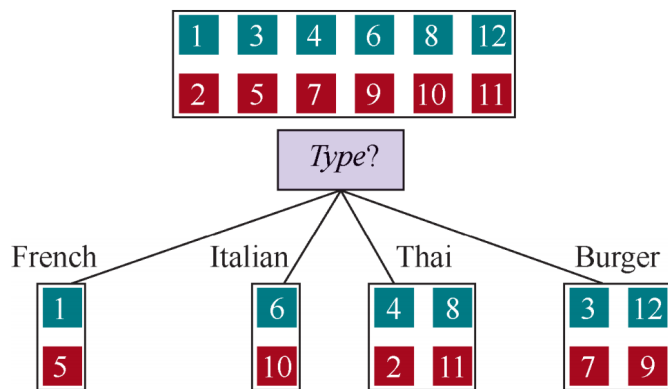
19.3 决策树学习 - 泛化与过拟合

■ 决策树剪枝

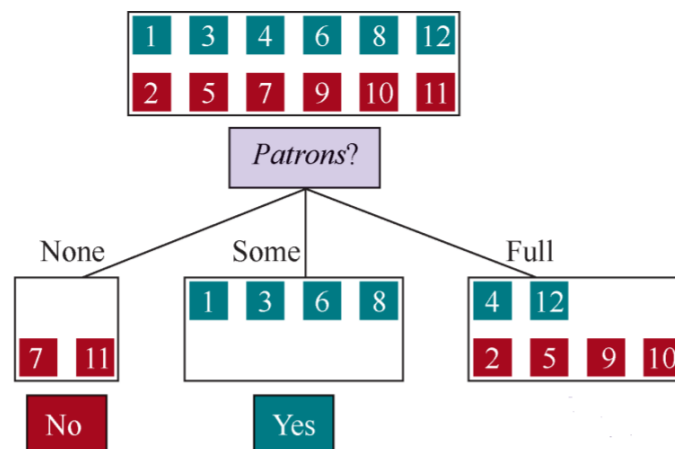
使用决策树剪枝 (pruning) 删去与目标无关的属性测试。

- 首先通过之前介绍的Learn-Decision-Tree算法生成初始决策树
- 取**只有叶节点**的**子节点**，并研究该节点所测试的属性与最终的分类结果是否**相关**。若不相关，则删除该节点，并直接返回该节点最常出现的类别。重复该步骤，直到删除了所有不相关的测试节点。
- 什么是“不相关”？信息增益是一个衡量相关性的有效标准。

$Gain(type) = 0$: 不相关



$Gain(patron) = 0.541$: 相关



19.3 决策树学习 - 泛化与过拟合

■ 决策树剪枝

阈值：需要多大的信息增益来判断某个属性是否相关？

- **零假设与显著性检验：**首先假设不存在基础的模式（即零假设，在该的问题中，假设当前测试的属性不相关），然后计算实际数据偏离零假设的程度，如果该偏离程度在统计上发生的概率过低（通常以5%概率作为阈值），则判断零假设不成立，数据中仍存在显著的模式，即测试属性与分类结果是相关的
- 具体到决策树，假设目标节点处测试属性与分类结果不相关，则这种假设下**各子节点的正负样例的比例应与父节点相同**，从而可以得到在零假设下正负样例数量的期望：

如果子节点与父节点正负比例相同，应该具有的正样本数量

$$\widehat{p}_k = p * \frac{p_k + n_k}{p + n},$$

$$\widehat{n}_k = n * \frac{p_k + n_k}{p + n}$$

如果子节点与父节点正负比例相同，应该具有的负样本数量

p, n ：父节点正/负样本数量

p_k, n_k ：子节点正负样本实际数量

19.3 决策树学习 - 泛化与过拟合

■ 属性相关性度量

- 通过下式，计算实际的正负样例分布 与 期望的分布 之间的偏差：

$$\text{偏差值 } \Delta = \sum_{k=1}^d \left[\frac{(p_k - \widehat{p}_k)^2}{\widehat{p}_k} + \frac{(n_k - \widehat{n}_k)^2}{\widehat{n}_k} \right]$$

- 偏差 Δ 服从 χ^2 分布（卡方分布），自由度为 $d-1$ 。
 - ▶ 以餐厅类型Type为例，共4个属性值，自由度为3
 - ▶ 在5%的置信水平下，总偏差 $\Delta=7.82$ 或更大的值将拒绝零假设
 - ▶ 在1%的置信水平下，总偏差 $\Delta=11.35$ 或更大的值将拒绝零假设
- 低于阈值的偏差值将接受零假设，即当前节点测试的属性不相关，因此树的分支应该被剪枝。这个方法称为 χ^2 剪枝

19.3 决策树学习 - 泛化与过拟合

■ 属性相关性度量

- 当我们通过样例计算出偏差之后，可以根据自由度和置信水平查表获得偏差的阈值：

自由度	0.5	0.25	0.1	0.05	0.03	0.01
1	0.455	1.323	2.706	3.841	5.024	6.635
2	1.386	2.773	4.605	5.991	7.378	9.21
3	2.366	4.108	6.251	7.815	9.348	11.345
4	3.357	5.385	7.779	9.488	11.143	13.277
5	4.351	6.626	9.236	11.07	12.833	15.086
6	5.348	7.841	10.645	12.592	14.449	16.812
7	6.346	9.037	12.017	14.067	16.013	18.475
8	7.344	10.219	13.362	15.507	17.535	20.09
9	8.343	11.389	14.684	16.919	19.023	21.666
10	9.342	12.549	15.987	18.307	20.483	23.209

- 例如，餐厅的type值有四种可能性，即自由度为3，在5%置信水平下，偏差的阈值为7.815，即如果计算出的 $\Delta > 7.815$ ，说明零假设成立的可能性低于5%，此时认为测试属性与分类结果相关；反之若 $\Delta < 7.815$ ，则认为测试属性为无关属性，进行剪枝。

19.3 决策树学习

■ 拓展决策树的适用范围

□ 缺失数据：

- ▶ 给定一颗完全决策树，如何分类失去一个测试属性的样例。
- ▶ 当样例某些属性值未知时，怎样修改信息收益公式？

□ 连续属性：

- ▶ 决策树的输入可能是连续值（如身高、体重），处理连续值的一个的方法是采用分割点（split point）测试，即一个关于属性值的不等式测试（ $\text{体重} > 160$ 及 $\text{体重} \leq 160$ ）。

□ 连续值输出属性：

- ▶ 如果要预测数值类型的输出，则需要一颗回归树。
- ▶ 回归树的每个叶节点上应记录一个针对该叶节点样例的线性回归函数，该函数同样需要通过学习获得。

- 学习形式
- 监督学习
- 决策树学习
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

19.4 模型选择与模型优化

■ 数据的组织：训练集、验证集、测试集

□ **学习目标：**在机器学习中，希望选择一个和**未来的样例**拟合得**最好的**假设/模型。

1. **未来样例：**未来的样例必须服从**平稳性假设**，即未来的样本与现有的样本是类似的，各样例 E_j 之间独立同分布 (i.i.d)：

$$P(E_1) = P(E_2) = \dots P(E_j) = P(E_{j+1}) = \dots \quad (\text{先验分布相同})$$

$$P(E_j) = P(E_j | E_1, E_2, \dots, E_{j-1}) \quad (\text{互相独立})$$

2. **最佳拟合：**

- ▶ 最佳拟合是**最小化错误率**的假设，即对于样例 (x, y) ，最小化 $h(x) \neq y$ 的比例。
- ▶ 如果估计一个假设在未来样例上的错误率，最简单的方法是将样例分为两组：一组用于训练得到假设的**训练集**，以及一组用于评估假设的**测试集**。**训练模型时不能以任何形式接触测试集。**

19.4 模型选择与模型优化

■ 数据的组织：训练集、验证集、测试集

- **超参数**：模型并不唯一，他包含一组人工设定的数值来微调模型的行为。在模型选择的过程中，我们需要**调参**。
 - ▶ 例如在决策树的 χ^2 剪枝中，通过调整置信水平阈值对决策树进行不同程度的剪枝。
- **使用测试集调参的风险**：如果通过调整超参数获得一个测试集错误率最小的模型，这个过程中会泄露测试集的信息。避免这个问题的方法是将**测试集完全不可见**，直到完成了所有的训练、超参数调整之后再使用它。
- 完整的数据集设定：**训练集、验证集、测试集**
 - ▶ 训练集：训练备选模型
 - ▶ 验证集：评估备选模型并选择备选模型（调参）
 - ▶ 测试集：无偏估计最佳模型

19.4 模型选择与模型优化

■ k 折交叉验证：应对数据不足

- 当数据不足时，使用k折交叉验证（k-fold cross validation）
- 每个样例都担负双重责任——既作为训练数据又作为测试数据。
- 将数据集分成k份，之后进行k轮训练，每轮训练中使用一份数据做验证集，其他用于训练。k轮平均的测试分数将比无k折测试分数的评估更加客观。
- K通常为10（10折交叉验证）或者5（5折交叉验证），极限情况 $k=n$ ，这时称之为留一交叉验证（Leave-one-out cross-validation, LOOCV）
- K折交叉验证即使采用了交叉验证的方法，仍然需要一个单独的测试集。

19.4 模型选择与模型优化

■ 模型的选择和训练：

- **模型选择**：选择一个好的假设空间（模型类）。
 - ▶ **模型类选择**：模型选择首先需要定性、主观的分析，基于对问题的一些先验了解，判断使用哪一类模型。比如从多项式函数或者决策树中决则。
 - ▶ **超参数选择**：模型选择还需要一些定量的设计来选择超参数，如多项式的次数、决策树的节点数等。
- **模型优化（训练）**：基于数据，在选择好的假设空间中找到最佳的假设。

19.4 模型选择与模型优化 - 模型选择

■ 简单的模型选择算法

■ Learner是一个模型类（如决策树），size是超参数。

■ 遍历备选的超参数（图中size），对每个超参数训练模型并进行k折交叉验证

■ 当超参数明显偏离最优取值（错误率显著增长）后停止，并选择最优的超参数以及对应的模型返回

function MODEL-SELECTION(*Learner*, *examples*, *k*) **returns** 一个(假设, 错误率)对

err $\leftarrow$ 一个数组，以*size*为索引，存储验证集错误率

training_set, *test_set* $\leftarrow$ *examples*划分成两个集合

for *size* = 1 **to** ∞ **do**

err[*size*] $\leftarrow$ CROSS-VALIDATION(*Learner*, *size*, *training_set*, *k*)

if *err*开始显著增长 **then**

best_size $\leftarrow$ 使*err*[*size*]最小的*size*值

h $\leftarrow$ *Learner*(*best_size*, *training_set*)

return *h*, ERROR-RATE(*h*, *test_set*)

function CROSS-VALIDATION(*Learner*, *size*, *examples*, *k*) **returns** 错误率

N $\leftarrow$ *examples*的个数

errs $\leftarrow$ 0

for *i* = 1 **to** *k* **do**

validation_set $\leftarrow$ *examples*[(*i* - 1) $\times$ *N*/*k*:*i* $\times$ *N*/*k*]

training_set $\leftarrow$ *examples* - *validation_set*

h $\leftarrow$ *Learner*(*size*, *training_set*)

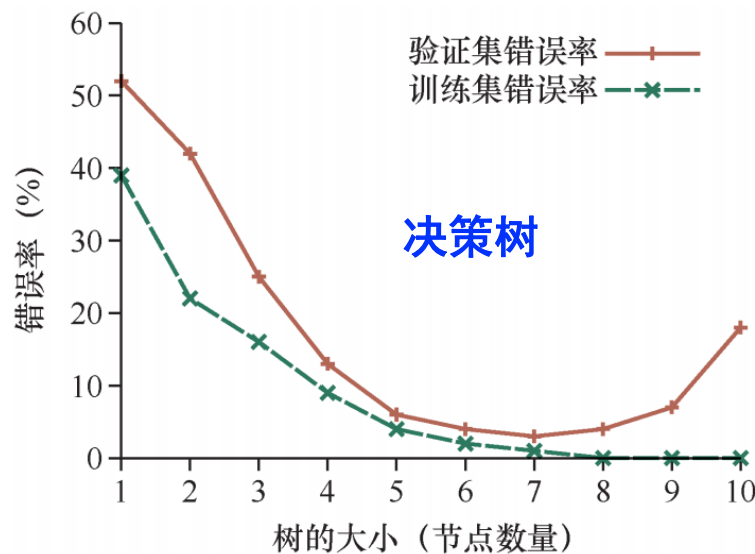
errs $\leftarrow$ *errs* + ERROR-RATE(*h*, *validation_set*)

return *errs* / *k* // 验证集的平均错误率，*k*折交叉验证

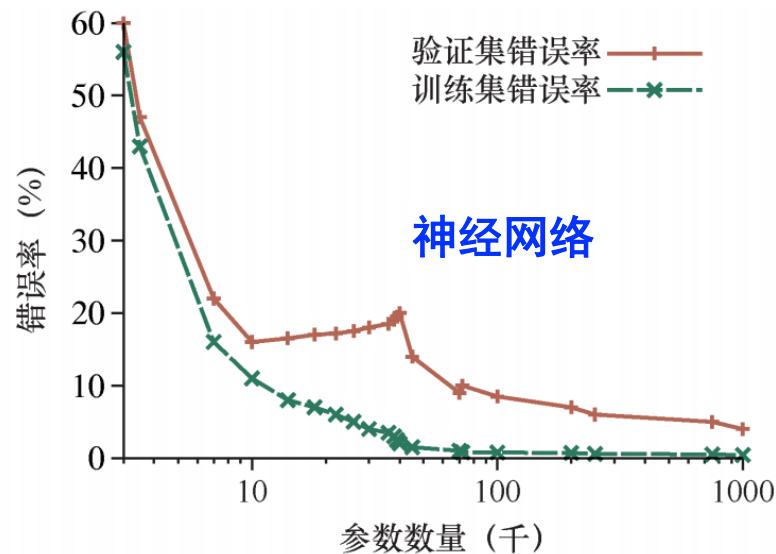
19.4 模型选择与模型优化 - 模型选择

■ 超参数选择的典型模式

- 图(a)是使用决策树拟合餐厅等待问题的结果，验证集的错误率曲线呈现U型，随模型规模增大，验证集错误率先减小后增大；
- 图(b)是使用神经网络拟合MNIST数据集的结果，虽然验证集错误率一开始也呈现出U型曲线，但随着参数继续增大，曲线恢复为下降趋势。



(a)

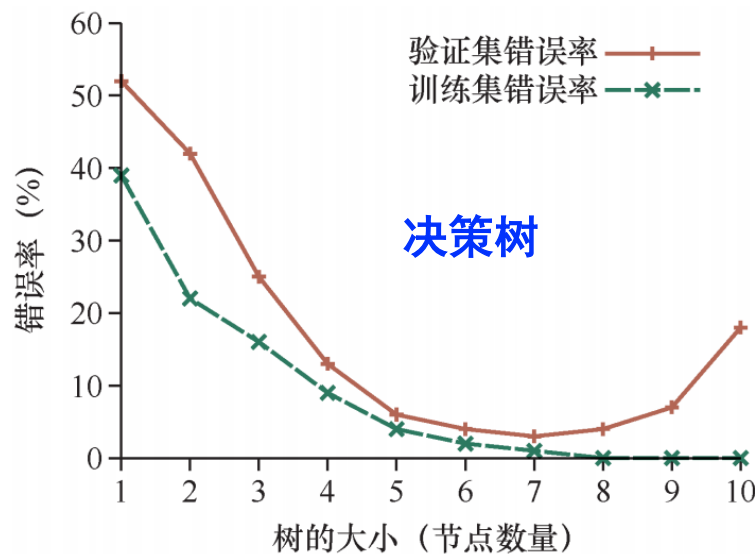


(b)

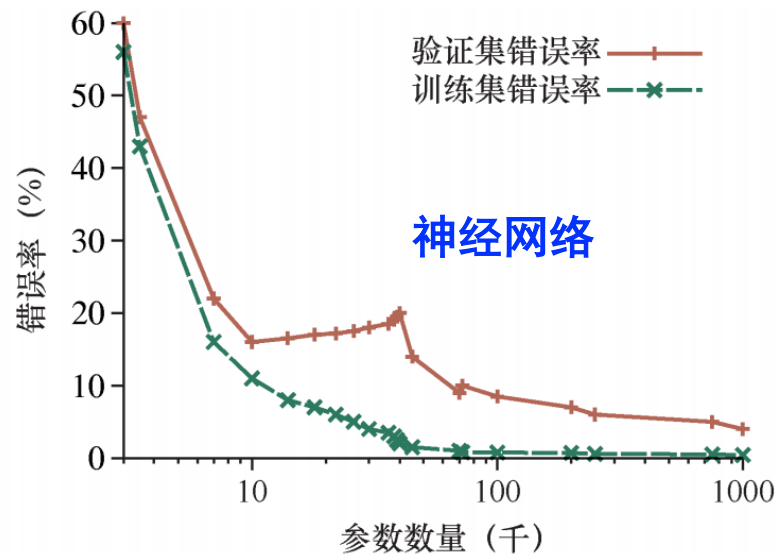
19.4 模型选择与模型优化 - 模型选择

■ 超参数选择的典型模式

- 出现这种现象的原因在于，不同模型类利用其过强表达能力的方式不同。
- 部分模型类很难从过拟合中恢复，如决策树；
- 但对另外一些模型类，更多的参数意味着更大的候选空间，其中很多函数都是有效的，优化方法也有更大可能落到其中一个上。



(a)



(b)

19.4 模型选择与模型优化 - 从错误率到损失函数

■ 使用错误率选择模型的缺陷：

- 使用最小错误率为标准的选择的“最佳拟合”是不够的，**不同错误的严重程度不同**。
- 以邮件分类为例：一个将邮件自动归类为正常邮件/垃圾邮件的模型，会犯两种错误：
 - ▶ 将垃圾邮件归为正常邮件。这顶多损失我们几秒钟的时间。
 - ▶ 将正常邮件归为垃圾邮件。这可能会导致我们错失重要的信息。
 - ▶ 宁可分类器以1%的概率犯第一类错误，也不想分类器以0.5%概率犯第二类错误。

- **解决方法**：预先定义好各类错误出现所导致的效用损失，用最小化效用损失，即**损失函数**，替代之前的最小化错误率。

19.4 模型选择与模型优化 - 从错误率到损失函数

■ 损失函数

- 我们用 $L(y, \hat{y})$ 来表示当正确答案为 y ，而模型输出为 $\hat{y}$ 时的效用损失，例如，如果“正常邮件归为垃圾邮件”比相反的情况严重10倍的话，那么可以规定

$$L(\text{正常}, \text{垃圾}) = 10, L(\text{垃圾}, \text{正常}) = 1$$

- 对于有限、离散的输出，可以为所有可能情况枚举效用损失。当函数的输出值是连续的，有无穷多种可能情况，就需要定义损失函数来衡量对输出结果的满意程度。
- 常用损失函数：

$$\text{绝对值损失: } L_1(y, \hat{y}) = |y - \hat{y}|$$

$$\text{平方误差损失: } L_2(y, \hat{y}) = (y - \hat{y})^2$$

$$\text{0/1损失: } L_{0/1}(y, \hat{y}) = 0 \text{ 若 } y = \hat{y}, \text{ 否则 } 1$$

19.4 模型选择与模型优化 - 从错误率到损失函数

■ 泛化损失与经验损失

- 智能体通过最小化训练集中**所有样例的损失函数**完成学习。
- **期望泛化损失（理想）**：若已知样例的先验分布为 $P(x, y)$ ，令 ε 为所有样例输入-输出的可能，则假设 h 关于损失 L 的期望泛化损失为：

$$GenLoss_L(h) = \sum_{(x,y) \in \varepsilon} L(y, h(x))P(x, y)$$

最佳假设 h^* 是使期望泛化损失最小的假设：

$$h^* = \underset{h}{argmin} GenLoss_L(h)$$

- **经验损失（现实）**：先验分布 $P(x, y)$ 往往是未知的，模型只能通过一组大小为 N 的样例（训练集）进行学习，此时用经验损失来近似泛化损失：

$$EmpLoss_L(h) = \frac{1}{N} \sum_{(x,y) \in E} L(y, h(x))$$

对应的最佳假设也变为

$$\widehat{h^*} = \underset{h}{argmin} EmpLoss_L(h)$$

19.4 模型选择与模型优化 - 从错误率到损失函数

- 最小化经验损失的 $\hat{h}^*$ 得到的函数一般与真实(ground truth)函数 f 不同，有四种可能的原因：
 - **不可实现性**：真实函数 f 本身不包含在假设空间 H 中，例如使用线性函数拟合一个二次的真实函数，则永远无法找到真实函数。如果 H 包含 f 则称该学习问题是可实现的。
 - **方差**：使用不同的样例集合进行训练会返回不同的最佳假设。当问题可实现时，方差会随着样例的增加而逐渐减小至0。（数据不足）
 - **噪声**：真实函数 f 可能是非确定性，或者有噪声的。对于相同的输入 x ，真实函数可能返回不同的值。
 - **计算复杂性**：当假设空间 H 足够庞大时，搜索所有可能性是不可能的，只能搜索部分假设空间并返回一个较优解，但并不能保证它是最优的。（局部极值）

19.4 模型选择与模型优化 - 从错误率到损失函数

■ 小规模学习（早期）

- 训练样例数量几十到几千，模型本身比较简单。
- 主要受不可实现性（假设空间不包含真实函数）与方差（训练数据不足以限制方差）限制。

■ 大规模学习（当今）

- 训练样例和模型参数以M（百万）甚至B（十亿）计。
- 如果有足够大的模型和足够的数据，可以找到一个非常接近真实函数的假设，此时计算复杂性（计算资源）成为了主要的约束。

19.4 模型选择与模型优化 - 正则化

■ 正则化

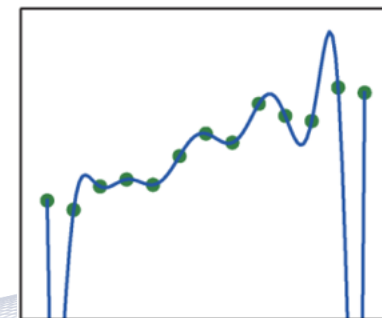
- 在优化模型时，引入模型的复杂性度量，并直接最小化经验损失与复杂性度量的加权和：

$$Cost(h) = EmpLoss_L(h) + \lambda Complexity(h)$$

$$\widehat{h}^* = \underset{h}{argmin} Cost(h)$$

- $Complexity(h)$ 称为**正则化函数**。正则化函数的选择依赖于假设空间。例如，对于多项式假设空间，可以选择系数平方和作为正则化函数，它要求模型的系数保持在较小的水平，可以有效避开如下图的函数值剧烈波动。

- 正则化通过限制模型的拟合能力，来防止过拟合，如果没有任何对模型的专业知识，至少有一个通用知识，就是simpler is better



19.4 模型选择与模型优化 - 正则化

■ 模型的复杂性度量

- 另一种简化模型的方法是减少模型的维数：通过一个特征选择 (feature selection) 的过程来判断属性的相关性，然后舍弃不相关的属性。之前讲的 χ^2 剪枝就是一种特征选择的方式。

19.4 模型选择与模型优化 - 超参数调整

■ 超参数搜索方法（调参）

超参数选择是模型选择的重要环节。

- **手动调参**：最简单的超参数调整方法，根据自身直觉选择合适的超参数并验证效果，直到找到满意的结果。（**最常用**）
- **网格搜索**：尝试所有超参数的所有组合，适合超参数的数量和取值都较少的情况。
- **随机搜索**：从超参数的可能组合中随机采样来获取较优的参数值。
- **贝叶斯优化**：将超参数选择本身视为一个机器学习问题，以超参数 x 作为输入，以对应的模型在验证集上的总损失 y 作为输出，尝试找到 x 与 y 的关系 $y=f(x)$ 。（经常是U型）
- **基于群体的训练**：首先训练大量不同超参的模型，通过选取其中结果较好的部分超参，使用遗传算法或随机搜索生成新的超参数值，（可以手工实现）。

- 学习形式
- 监督学习
- 决策树学习
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

19.5 学习理论

■ 计算学习理论：保证从数据中学习到的模型是有效的

- 如何确定学习到的假设 h 能泛化到未知的样例？
- 我们不知道真实函数 f 是什么样子，如何确定 h 是否接近于 f ？
- 需要多少样例来训练好的假设？
- 如何选择假设空间？
- 在复杂的假设空间中，是否能找到最佳假设 h ？
- h 应该有多复杂？如何避免过拟合？

19.5 学习理论

■ 计算学习理论

- 基本原理：用少量的样本进行训练之后，那些非常不匹配的假设将有很高的概率被“排除”。
- 当一个假设与足够多的训练样例一致，结合样例的平稳性假设，那么它不太可能与真实函数严重不匹配，它是**概率近似正确的**（probably approximately correct, PAC）。
- 一个返回“概率近似正确的假设”的学习算法称为**PAC学习算法**。

19.5 学习理论

■ PAC学习算法

- PAC的基础是平稳性假设，即未来样例和现有样例服从相同的分布 $P(X, Y)$ 。
- 如果一个假设满足 $\text{error}(h) \leq \varepsilon$ ，那么我们称该假设 h 是近似正确 (approximately correct) 的，其中 ε 是一个较小的常数。将证明，能找到一个 N ，在用 N 个样例训练后，所有一致性假设都将有高概率是近似正确的。
- 可以认为近似正确的假设在假设空间中是“接近”真实函数的：它位于真实函数 f 周围的某个 ε 球 (ε -ball) 内。我们将该球外部的假设空间记为 $\mathcal{H}_{bad}$
- 对于一个“严重错误”的假设 $h_b \in \mathcal{H}_{bad}$ ，可以计算出它与前 N 个样本都一致的概率的界限。（一个错误假设反而完美拟合了训练集的概率）

19.5 学习理论

■ 布尔函数的PAC定理

- 最简单的 PAC 定理是有关布尔函数的，取0/1损失作为损失函数，此时error就是假设h对样例分类的错误率：

$$error(h) = \sum_{(x,y) \in \mathcal{E}} L_{0/1}(y, h(x))P(x, y)$$

- 考虑一个假设 $h_b \in H_{bad}$ ，则有分类错误率 $error(h_b) > \varepsilon$ ，则 h_b 正确分类的概率低于 $1 - \varepsilon$ ，由于样例之间独立同分布， h_b 与 N 个训练样例一致的概率

$$error(h_b) > \varepsilon \Rightarrow P(h_b \text{ 与 } N \text{ 个样例一致}) < (1 - \varepsilon)^N$$

- H_{bad} 中包含一个与 N 个训练样例一致的假设的概率将有上界：
 $P(H_{bad} \text{ 中包含一致性假设}) \leq |H_{bad}|(1 - \varepsilon)^N \leq |H|(1 - \varepsilon)^N$

如果一个假设的正确率小于 ε ，那么他反而能正确分类所有训练集的概率为 $(1 - \varepsilon)^N$

19.5 学习理论

■ 布尔函数的PAC学习

- 希望所有一致性假设仅有较小概率 (δ) 位于 H_{bad} 中, 则

$$|H|(1 - \varepsilon)^N \leq \delta \Rightarrow N \geq \frac{1}{\varepsilon} \ln\left(\frac{|H|}{\delta}\right)$$

- 当学习算法观测到了至少 $N = \frac{1}{\varepsilon} \ln\left(\frac{|H|}{\delta}\right)$ 个样例以后, 将以至少 $1 - \delta$ 的概率返回一个错误率至多为 ε 的假设, 这一样例数量称为学习算法的**样本复杂性**。
- δ 可以理解为使用 N 个数据学习到**烂假设**的概率。 N 越小, δ 必须越大, 就越有可能即使完美拟合了训练集, 也得到一个错误的假设。所以数据越多, 学习到 bad 假设的概率越低。
- $|H|$ 越大, 假设可能性越多, 所需的数据量 N 越大。

- 学习形式
- 监督学习
- 决策树学习
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

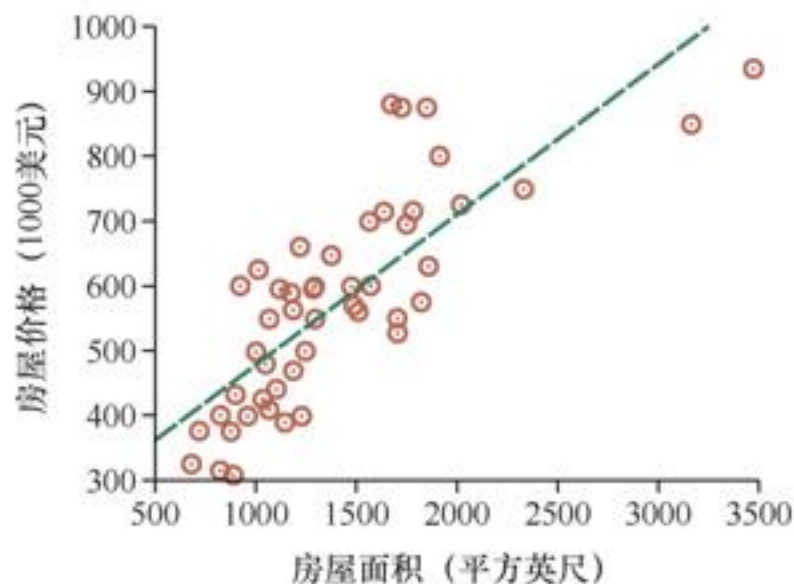
19.6 线性回归和分类 - 单变量线性回归

■ 线性回归: 假设空间为连续输入值的线性函数。

- 单变量线性函数: 输入 x , 输出 y , 其形式为

$$y = w_1 x + w_0$$

- 其中 w_1 和 w_0 是要学习的实数系数。
- 给出 xy 平面上的 n 个样例点, 希望能找到最匹配这些数据的线性函数, 这一任务称为**线性回归**。



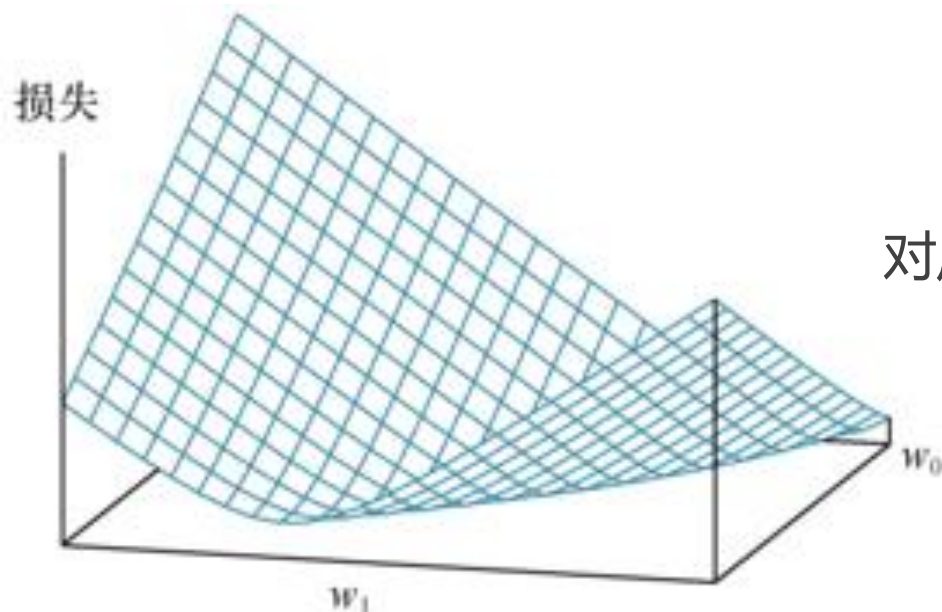
19.6 线性回归和分类 - 单变量线性回归

■ 线性回归的优化目标

- 通常采用平方误差损失，对所有样例求和作为最终的损失函数：

$$Loss(h_w) = \sum_{j=1}^N L_2(y_j, h_w(x_j)) = \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2$$

- 损失函数曲面：损失函数是凸函数，局部极值点为全局极值点。



对应的最优权重 $w^* = (w_0^*, w_1^*)$ 为

$$w^* = \underset{w}{argmin} Loss(h_w)$$

19.6 线性回归和分类 - 单变量线性回归

■ 线性回归的最优解

□ 在 w^* 处, 损失函数关于 w_0 和 w_1 的偏导数为0:

$$\frac{\partial}{\partial w_0} \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2 = 0, \quad \frac{\partial}{\partial w_1} \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2 = 0$$

□ 这个问题有唯一解

$$w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}, \quad w_0 = \frac{\sum y_j - w_1(\sum x_j)}{N}$$

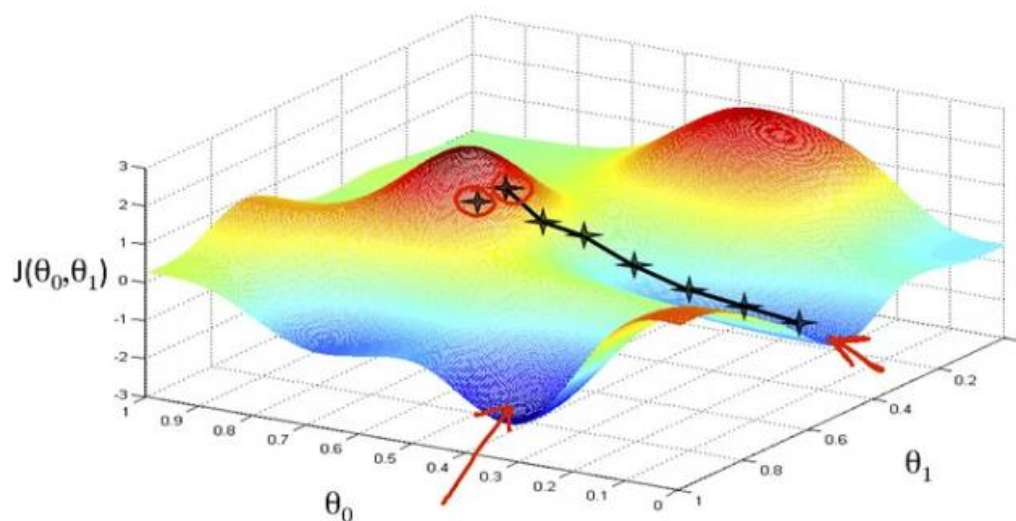
19.6 线性回归和分类 - 梯度下降

- 解析法的缺陷：
 - 不是所有损失函数都是凸的。
 - 求解偏导数的零点对于大多数复杂函数会困难很多，经常没有解析解。

19.6 线性回归和分类 - 梯度下降

■ 梯度下降法

- 首先选取权重空间中的任意一点作为起点
- 计算梯度，并向最陡峭的下坡方向（负梯度方向）移动一小步
- 重复这一过程，直到收敛到权重空间中的某一点，该点的损失在权重空间中为局部最小值。



每次沿负梯度
方向移动一步

19.6 线性回归和分类 - 梯度下降

■ 梯度下降法

□ 伪代码：

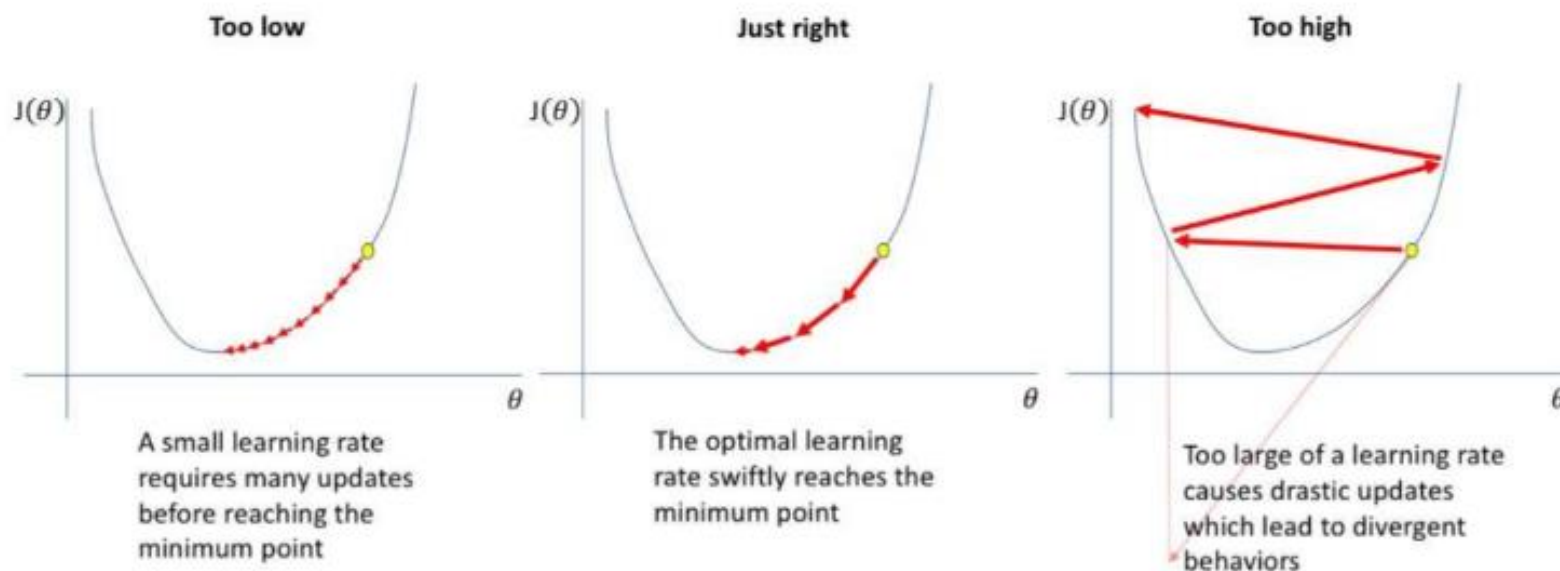
```
 $w \leftarrow \text{参数空间中的任意一点}$   
while not 收敛 do  
  for each  $w_i$  in  $w$  do  
     $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(w)$ 
```

- 伪代码中的参数 α ，在梯度下降中称之为**学习率**，它可以是一个固定的常数，也可以随着训练逐渐衰减。
- **学习率通常手工设定，他的调参是使用梯度下降的重要一环。**

19.6 线性回归和分类 - 梯度下降

■ 梯度下降法

□ 学习率 α ：过低、合适、过高



19.6 线性回归和分类 - 梯度下降

■ 梯度下降法

- 以单变量线性回归为例，将损失函数分别对 w_0 和 w_1 求偏导：

$$Loss(h_w) = \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2$$

$$\frac{\partial}{\partial w_0} Loss(h_w) = \frac{\partial}{\partial w_0} \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2 = \sum_{j=1}^N -2(y_j - (w_1 x_j + w_0))$$

$$\frac{\partial}{\partial w_1} Loss(h_w) = \frac{\partial}{\partial w_1} \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2 = \sum_{j=1}^N -2x_j(y_j - (w_1 x_j + w_0))$$

19.6 线性回归和分类 - 梯度下降

■ 梯度下降法

- 从而得到 w_0 和 w_1 的更新规则：

$$w_0 \leftarrow w_0 + \alpha \sum_{j=1}^N (y_j - (w_1 x_j + w_0))$$

$$w_1 \leftarrow w_1 + \alpha \sum_{j=1}^N x_j (y_j - (w_1 x_j + w_0))$$

- 这就是单变量线性回归的**批梯度下降** (batch gradient descent) , 即使用所有数据进行梯度下降。
- 线性回归中由于损失函数为凸函数, 因此在 α 不过大时一定能收敛到全局极小值, 但可能会非常慢。

19.6 线性回归和分类 - 梯度下降

■ 随机梯度下降

- 随机梯度下降 (stochastic gradient descent, SGD) : 在每一步中都**随机选择少量样例来进行更新**。
 - ▶ 例如: 每次仅选取1%的样例进行更新, 这样每一步的计算代价也减少了100倍。尽管只用1%的样例估算的梯度的需要10倍以上的步骤才能达到相同的收敛效果, 但总体来看SGD仍然比批梯度下降快10倍。
- SGD中随机选择的样本量 (batchsize) 是一个重要的超参数, 它受计算资源 (内存/显存) 的限制, 且过大和过小都无法得到理想的结果。
- SGD 已经被广泛地应用于除线性回归以外的模型, **尤其是神经网络**。即使损失平面不是凸的, 该方法也已被证明可以有效地找到接近全局最小值的性质良好的局部极小值。

19.6 线性回归和分类 - 多变量线性回归

■ 多变量线性回归

- 假设每个样例的输入 x_j 是一个 n 元向量，则假设空间中的函数形式变为：

$$h_w(x_j) = w_0 + w_1x_{j,1} + \dots + w_nx_{j,n}$$

- 通常我们引入一个虚拟的输入 $x_{j,0}$ 且始终等于1，从而将 h_w 表示为输入向量与权重的点积：

$$h_w(x_j) = \sum_{i=0}^n w_i x_{j,i}$$

- 矩阵形式：

$$\text{令： } \mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ \dots \\ w_n \end{pmatrix} \quad \mathbf{x}_j = \begin{pmatrix} 1 \\ x_{j,1} \\ \dots \\ x_{j,n} \end{pmatrix} \quad \text{则： } h_w(x_j) = \mathbf{x}_j^T \mathbf{w}$$

19.6 线性回归和分类 - 多变量线性回归

■ 多变量线性回归的最优解

□ 令 $X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix}$ 为大小是 $N \times (n+1)$ 数据矩阵, $y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$ 为大小是 $N \times 1$ 的输出矩阵, $w = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix}$ 为大小是 $(n+1) \times 1$ 的参数矩阵。

□ 预测的输出向量为

$$\hat{y} = Xw$$

□ 平方误差损失为

$$L(w) = \|\hat{y} - y\|^2 = \|Xw - y\|^2$$

19.6 线性回归和分类 - 多变量线性回归

■ 多变量线性回归的最优解

- 平方误差损失为

$$L(\mathbf{w}) = \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

- 对应梯度为

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = 2\mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

- 容易求出梯度的零点为：

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

其中 $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ 为数据矩阵 $\mathbf{X}$ 的**伪逆**

19.6 线性回归和分类 - 多变量线性回归

■ 加入正则化

- 使用正则化的方法避免过拟合：经验损失+模型复杂性

$$Cost(h) = EmpLoss(h) + \lambda * Complexity(h)$$

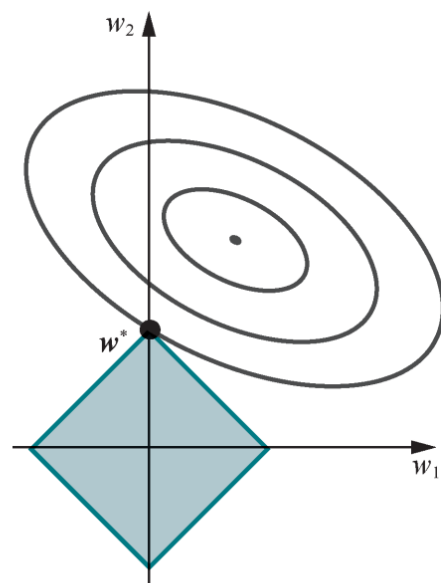
- 线性函数的复杂性取决于权重的大小：

$$Complexity(h_w) = \sum_i |w_i|^q$$

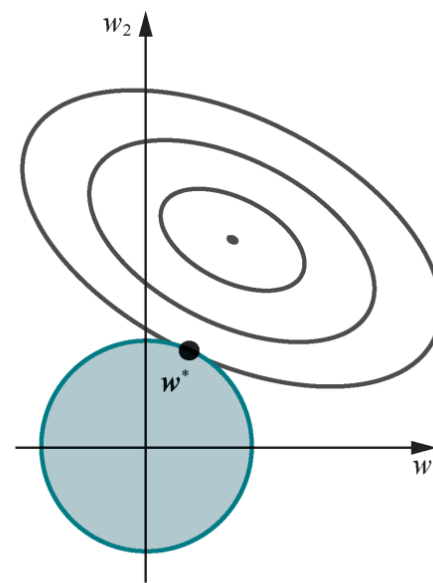
- 当q取1时，为 L_1 正则化，它极小化绝对值之和。
- 当q取2时为 L_2 正则化，它极小化平方和。
- L_1 正则化有倾向于生成一个**稀疏模型**。能够在优化过程中将许多**权重置零**，从而完成类似**特征选择**的操作，舍弃一些不相关的特征。（扩展：稀疏编码Sparse Coding）

19.6 线性回归和分类 - 多变量线性回归

■ L1正则化的稀疏性



(a) L1正则



(b) L2正则

经验损失和正则项的平衡点会落在同心椭圆和正则区域的交界

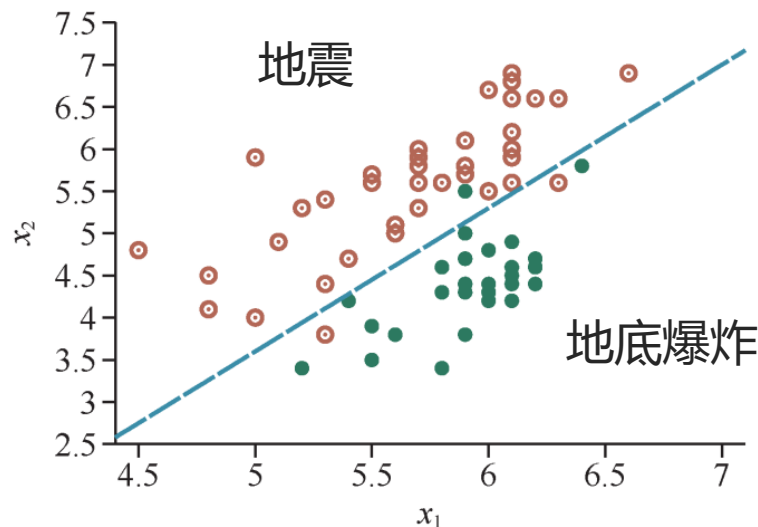
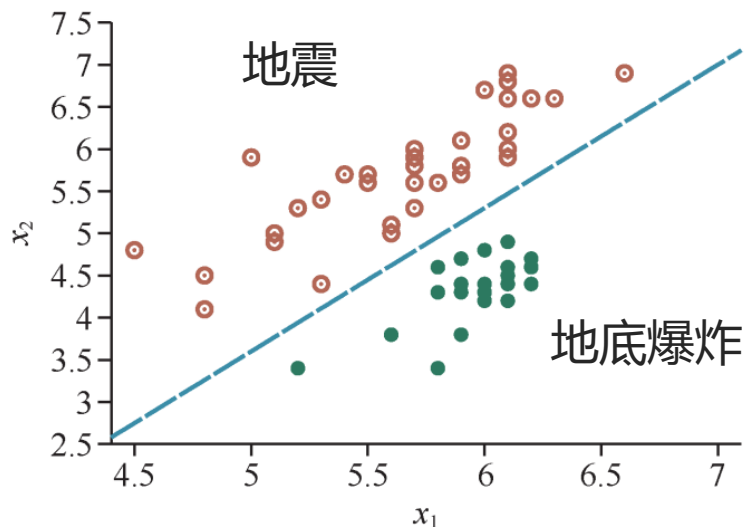
19.6 线性回归和分类 - 带硬阈值的线性分类器

■ 线性函数应用于分类：

□ 举例：分类两种类型的地震数据

➤ 横纵坐标分别表示地震的体波震级和面波震级。

➤ 橙色空心圆为地震，绿色实心圆为地底爆炸

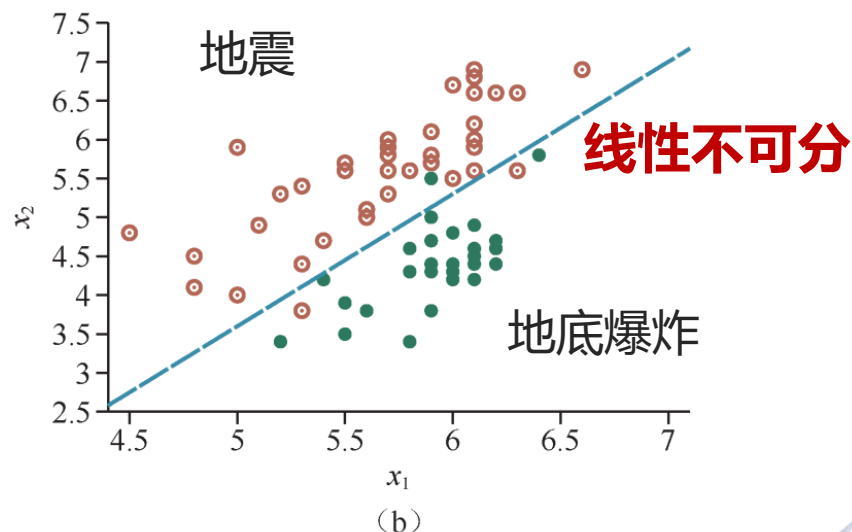
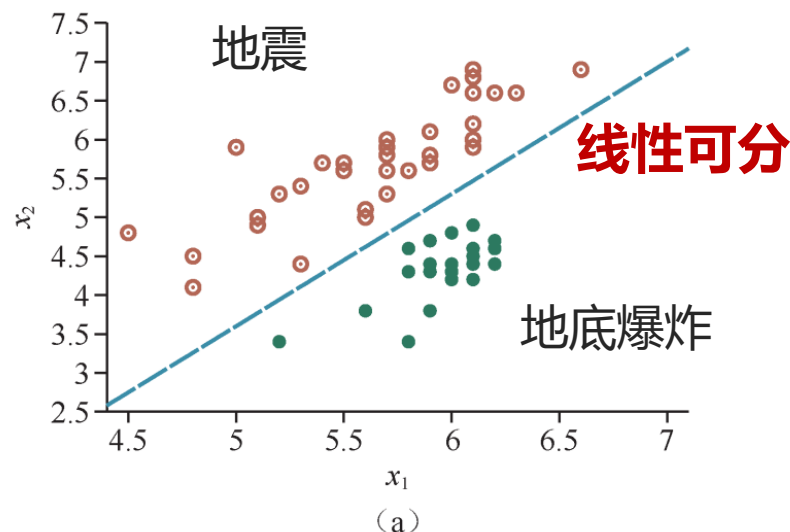


我们希望通过一个线性函数作为**决策边界**，
将两类数据分离。

19.6 线性回归和分类 - 带硬阈值的线性分类器

■ 线性分类器

- 决策边界定义为分离两个类的一条线（在高维中是一个高维面）
- 线性决策边界称为线性分离器，允许线性分离器存在的数据称为线性可分。



线性分类器形式：左边是一个线性函数，右边是0

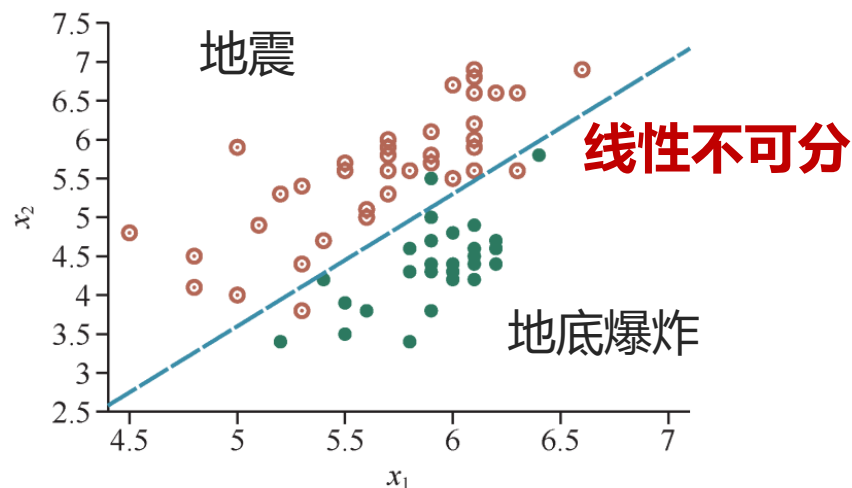
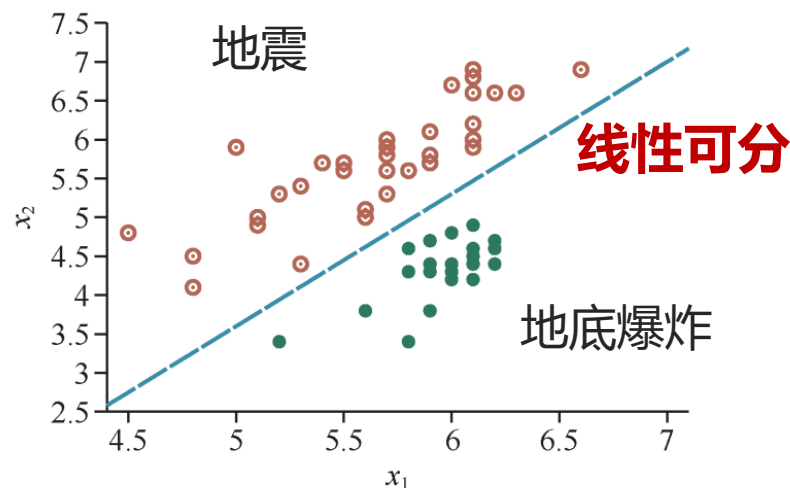
决策边界: $-4.9 + 1.7x_1 - x_2 = 0$

地底爆炸: $-4.9 + 1.7x_1 - x_2 > 0$; 地震: $-4.9 + 1.7x_1 - x_2 < 0$

19.6 线性回归和分类 - 带硬阈值的线性分类器

■ 线性分类器

- 决策边界定义为分离两个类的一条线（在高维中是一个高维面）
- 线性决策边界称为线性分离器，允许线性分离器存在的数据称为线性可分。



- 高维空间中，决策边界的写成矩阵形式： $w \cdot x = 0$

例子中 $w = (-4.9, 1.7, -1)$, $x = (1, x_1, x_2)$

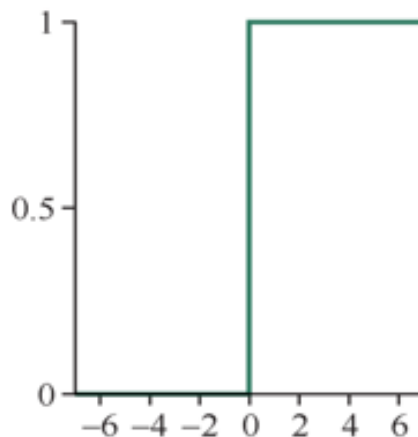
19.6 线性回归和分类 - 带硬阈值的线性分类器

■ 硬阈值线性分类器

- 定义一个**硬阈值函数**来表示线性分类器，即

$$h_w(x) = \text{Threshold}(w \cdot x) = \begin{cases} 1, & \text{if } w \cdot x \geq 0 \\ 0, & \text{else} \end{cases}$$

$h_w(x)$ 可视化:



硬阈值

硬阈值线性分类器求解时，对权重 w 的求解难以直接用梯度下降法。
除了决策边界上恰好存在数据点的情况，其他地方梯度处处为0

19.6 线性回归和分类 - 带硬阈值的线性分类器

■ 硬阈值线性分类器的学习

□ 感知机学习规则：

$$w_i \leftarrow w_i + \alpha(y - h_w(x))x_i$$

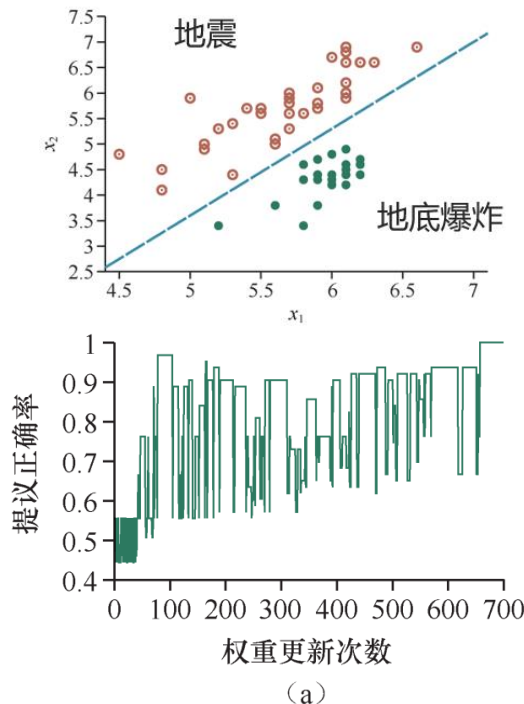
□ 真值 y 和模型输出值 $h_w(x)$ 均为0/1取值，因此存在三种情况：

- ▶ $y = h_w(x)$ ，分类正确，权重不变
- ▶ $y = 0$ 而 $h_w(x) = 1$ ， w_i 向 $-x_i$ 方向移动，这会使 $w \cdot x$ 减小，从而使 $h_w(x)$ 趋向于输出0
- ▶ $y = 1$ 而 $h_w(x) = 0$ ， w_i 向 x_i 方向移动，这会使 $w \cdot x$ 增大，从而使 $h_w(x)$ 趋向于输出1

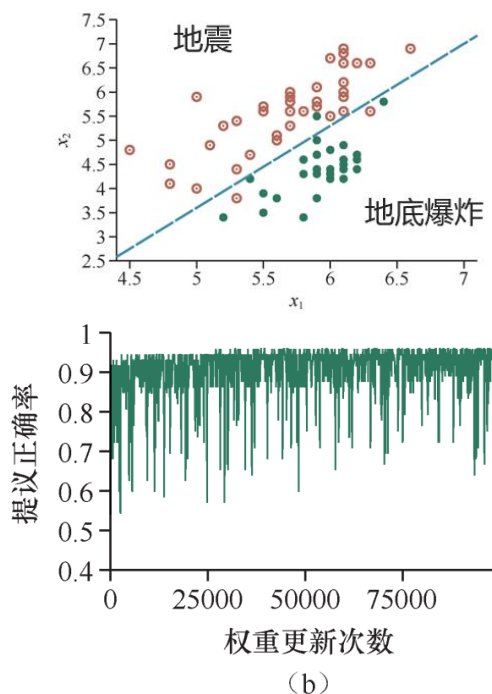
19.6 线性回归和分类 - 带硬阈值的线性分类器

■ 硬阈值线性分类器的学习

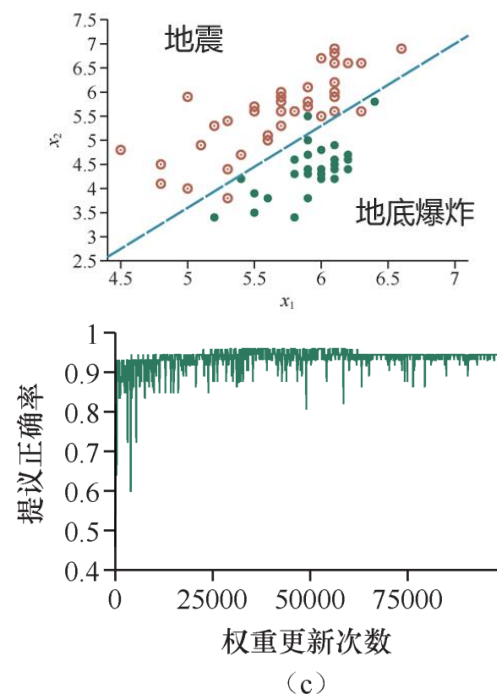
- 感知机的学习曲线：在感知机学习规则中，每一次更新随机选择一个样例进行学习，可以画出学习过程的训练曲线：



线性可分数据
 α 不变



线性不可分数据
 α 不变



线性不可分数据
 α 逐渐衰减

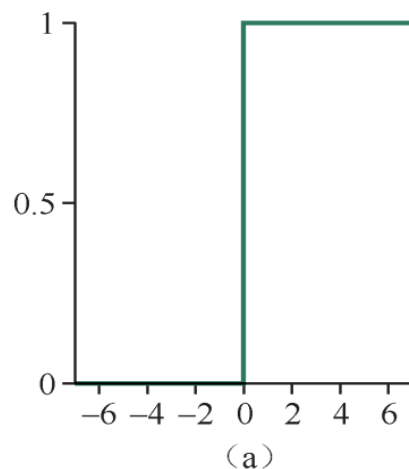
19.6 线性回归和分类 - 基于逻辑回归的线性分类器

■ 软阈值线性分类器：Logistic回归

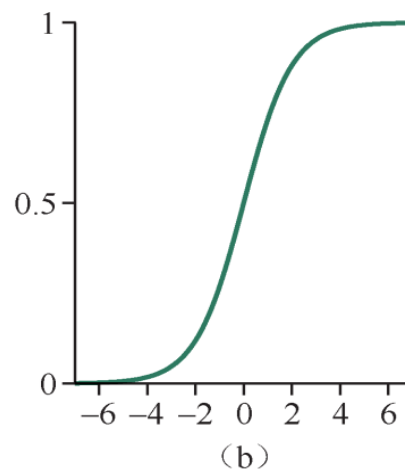
- 将远离边界的样本分类为明确的 1 或 0，而将处于边界附近的样本分类为“不清楚”。
- 引入软阈值函数来将之前的硬阈值拓展到连续、可微的情形。
- Logistic函数（它也被称为Sigmoid函数）：

$$\text{Logistic}(z) = \frac{1}{1 + e^{-z}}$$

硬阈值



软阈值



19.6 线性回归和分类 - 基于逻辑回归的线性分类器

■ 逻辑斯谛回归

- 使用Logistic函数，把线性分类函数修改为

$$h_w(x) = \text{Logistic}(w \cdot x) = \frac{1}{1 + e^{-w \cdot x}}$$

- 分类函数将 $w \cdot x$ 映射为一个0~1之间的数字，表示样本属于类别1的概率。
- 决策边界附近的样本，该概率接近0.5，随着远离决策边界，该概率接近0/1。
- 在此基础上拟合权重并最小化数据集上损失的过程，称为**逻辑斯谛回归**（logistic regression），很适合使用梯度下降求解。

19.6 线性回归和分类 - 基于逻辑回归的线性分类器

■ Logistic回归参数更新

$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{x}}}$$

- Logistic函数的导数为 $g'(z) = g(z)(1 - g(z))$ ，这里我们使用 L_2 损失函数，可以计算梯度为

$$\begin{aligned} \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w}) &= \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(\mathbf{x}))^2 \\ &= 2(y - h_{\mathbf{w}}(\mathbf{x})) \times \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(\mathbf{x})) \\ &= -2(y - h_{\mathbf{w}}(\mathbf{x})) \times g'(\mathbf{w} \cdot \mathbf{x}) \times \frac{\partial}{\partial w_i} \mathbf{w} \cdot \mathbf{x} \\ &= -2(y - h_{\mathbf{w}}(\mathbf{x})) \times g'(\mathbf{w} \cdot \mathbf{x}) \times x_i \\ &= -2(y - h_{\mathbf{w}}(\mathbf{x})) \times h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x})) \times x_i \end{aligned}$$

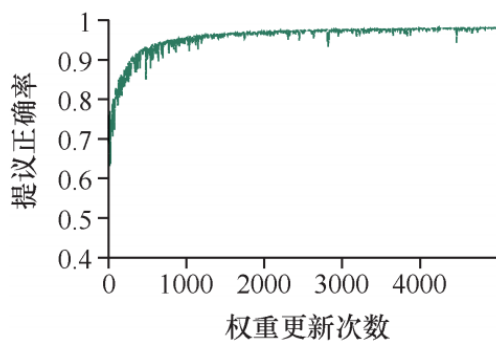
19.6 线性回归和分类 - 基于逻辑回归的线性分类器

■ Logistic回归参数更新

- Logistic回归的参数更新规则为

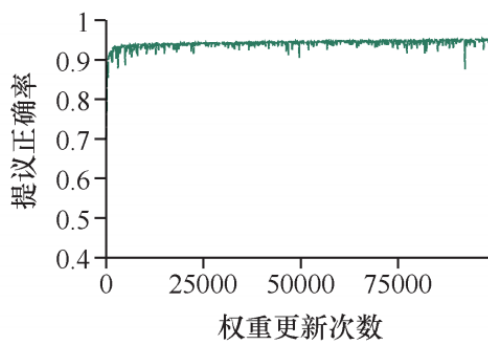
$$w_i \leftarrow w_i + \alpha(y - h_w(x))h_w(x)(1 - h_w(x))x_i$$

- 逻辑回归在线性可分的数据下收敛速度慢于感知机学习规则
- 对于噪声数据或不可分数据，其收敛速度明显更快、更稳定。



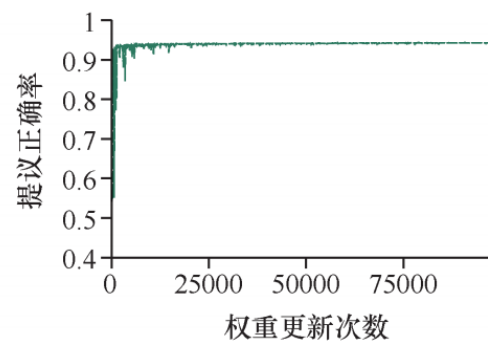
(a)

线性可分数据



(b)

带噪声数据



(c)

线性不可分数据

- 学习形式
- 监督学习
- 决策树学习
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

19.7 非参数化模型

■ 非参数化模型

- **参数模型**：用固定数量参数组成的集合 w 概括数据的学习模型。
 - ▶ 参数数量独立于训练样例的数量
 - ▶ 通过优化参数获得假设 $h_w(x)$ 后，可以丢弃训练数据，因为 w 中已经包含了有关训练数据的信息。
- **非参数化模型**：无法用参数个数固定的参数集来表示的模型。
 - ▶ 例如：分段线性函数将所有数据点保留为模型的一部分。
 - ▶ 也被称为**基于实例的学习**（instance-based learning）或**基于记忆的学习**（memory-based learning）
 - ▶ 当样例数量足够庞大时，用数据本身表示假设就是一个可行的方法，而不需要强行用参数模型来限制假设空间：

19.7 非参数化模型 - 最近邻模型

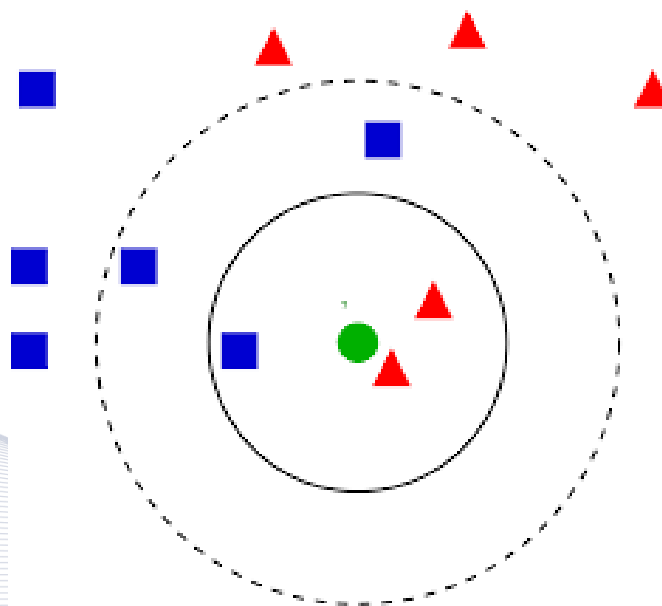
■ 最近邻模型

- **查找表**（最简单的基于实例的模型）：把所有的训练样例放在一张查找表中，然后查找测试样例是否在表中，如果在，返回对应的输出值。
- **k近邻查找**：对于待查询的输入 x_q ，寻找训练集中和它最接近的k个样例。使用 $NN(k, x_q)$ 来表示与 x_q 最近邻的k个样例。
 - ▶ 为了避免发生纠纷，总是令k为奇数
 - ▶ 如果k=1，则称为最近邻查找

19.7 非参数化模型 - 最近邻模型

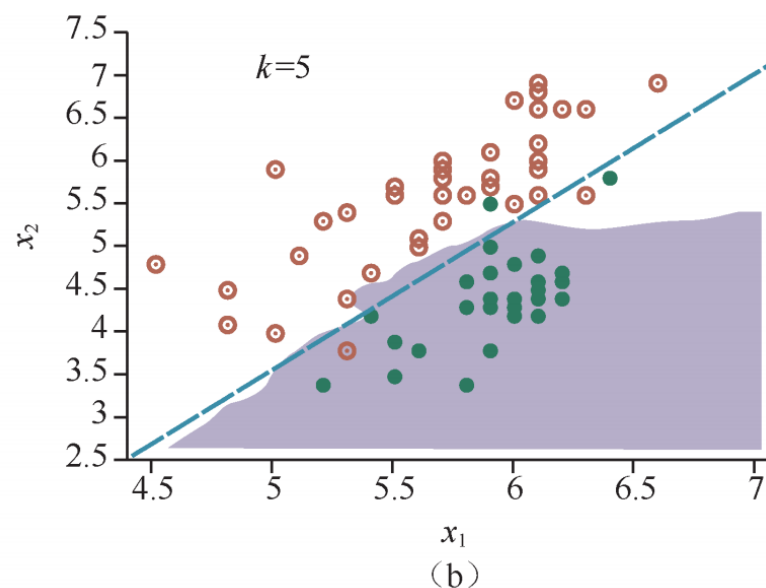
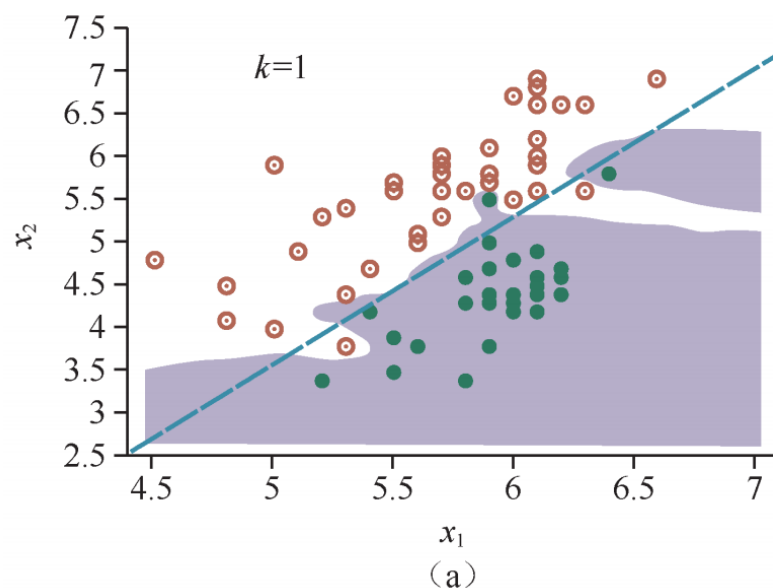
■ 最近邻模型

- k 近邻查找可以帮助实现分类和回归任务：
 - ▶ 对于分类任务，取 k 近邻中最多的一个类别作为输出值。
 - ▶ 对于回归任务，可以简单地取 k 近邻的平均值或中位数，也可以在 k 近邻上求解一个线性回归模型。



19.7 非参数化模型 - 最近邻模型

■ 通过k近邻查找进行分类



- 取 $k=1$ ，出现了明显的过拟合现象；
- 取 $k=5$ ，则获得了较好的决策边界。但如果继续增大 k ，则可能出现欠拟合现象。
- 通常可以使用交叉验证来确定较好的 k 值。

19.7 非参数化模型 - 最近邻模型

■ k近邻模型的距离度量

- 测量从查询点 x_q 到样例点 x_j 的距离，使用**闵可夫斯基距离**（或者叫 L^p 范数）：

$$L^p(x_j, x_q) = \left(\sum_i |x_{j,i} - x_{q,i}|^p \right)^{\frac{1}{p}}$$

- 当 $p=2$ 时，该距离为**欧几里得距离**。
- 当 $p=1$ 时，该距离为**曼哈顿距离**。
 - ▶ 对于布尔属性值，将两个数据点不同属性的数量称为**汉明距离**。

19.7 非参数化模型 - 最近邻模型

■ k近邻模型的距离度量

- 测量从查询点 x_q 到样例点 x_j 的距离，使用**闵可夫斯基距离**（或者叫 L^p 范数）：

$$L^p(x_j, x_q) = (\sum_i |x_{j,i} - x_{q,i}|^p)^{\frac{1}{p}}$$

- **归一化**： x 不同维度可能具有不同的物理含义，因此有不同的单位。单位对距离的计算有很大影响（米 vs. 厘米），称为**尺度**

(scale)，尺度大的维度具有主导作用

- ▶ 通常对每个维度进行归一化，对每个维度进行缩放并保持均值为0，方差为1，即 $(x_{j,i} - \mu_i)/\sigma_i$

- ▶ 更复杂的**马氏距离**考虑维度之间的协方差： $\sqrt{(\vec{x} - \vec{\mu})^T S^{-1} (\vec{x} - \vec{\mu})}$

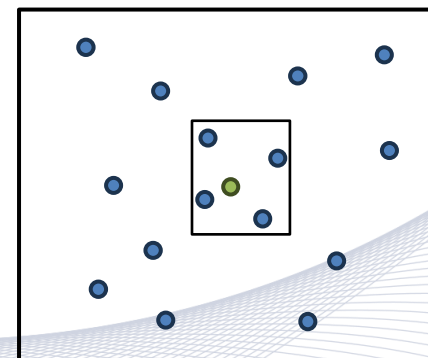
19.7 非参数化模型 - 最近邻模型

■ 维数灾难问题：

- 当数据的维度增大时，某个样本点的k近邻实际距离它并不近。
- 举例：在n维的超立方体中均匀地撒入一些数据，寻找一个样本的k近邻并找到一个最小的、能覆盖这k近邻的超立方体。
- 在低维空间中，样本点密集，这超立方体的边长很小。随着维数增加，超立方体的边长会增大。

最近邻更适合维度较小的情况

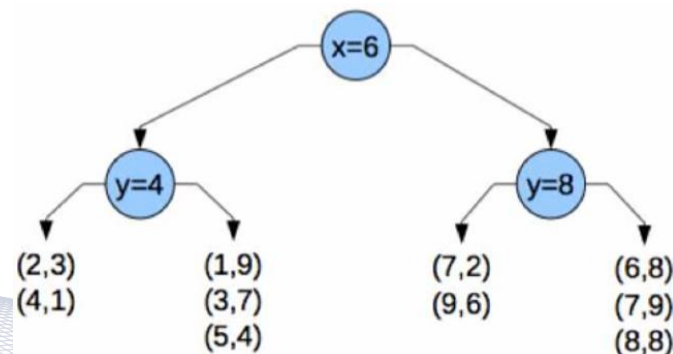
- ▶ 例如：取 $k=10$ ，总样本数 $N=1,000,000$ ，整个立方体体积为1
- ▶ 2维情况：平均边长0.003；
- ▶ 3维情况：平均边长0.02；
- ▶ 17维情况：平均边长0.5；
- ▶ 200维情况：平均边长0.94，几乎占满空间。



19.7 非参数化模型 - k-d 树寻找最近邻

■ 最近邻搜索：k-d树

- k-d 树 (k-dimension tree) : 建立在任意维数的数据集上的平衡二叉树，或称k维树。
- 构造：
 - ▶ 在构建树的第 i 层时，对数据的第 i 维进行测试。以中位数 m 为临界值将样例划分到左右两支： $x_i < m$ 和 $x_i > m$
 - ▶ 分别针对样例的左集合和右集合，递归产生左子树和右子树。
 - ▶ 剩余样例少于2个时停止。



19.7 非参数化模型 - k-d 树寻找最近邻

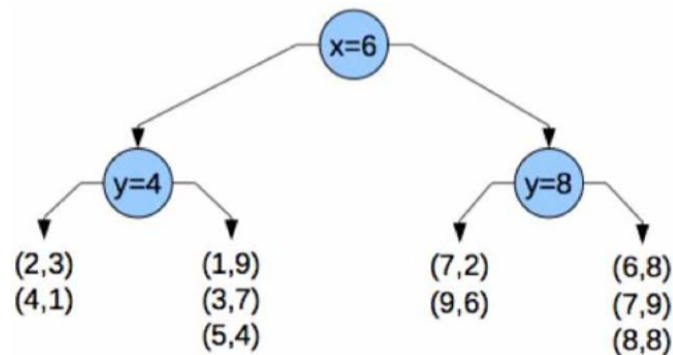
■ 最近邻搜索：k-d树

□ 查询：类似二叉树查找

- ▶ 难点：K近邻可能一部分在左子树，一部分在右子树。
- ▶ 此时根据查询点到划分边界的距离来帮助搜索最近邻。
- ▶ 某些时候需要同时搜索左子树和右子树。

□ 适用范围：k-d 树适用于样例数多于维数的情形（最好至少存在 2^d 个样例）

- ▶ 10个维度以下有数千个样例
- ▶ 10到20个维度有数百万个样例



19.7 非参数化模型 - 局部敏感哈希

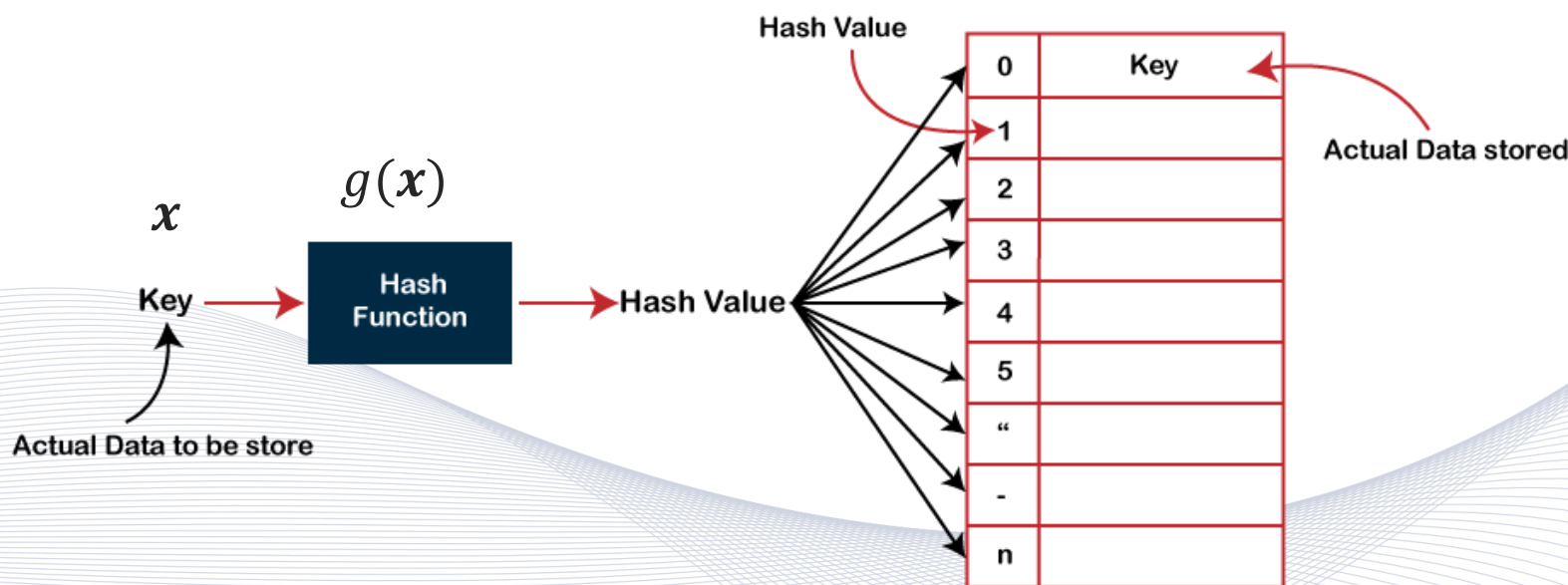
■ 最近邻搜索：局部敏感哈希

- 如果无法高效地找到 k 近邻，我们至少希望找到一个足够接近的结果。
- **近似近邻** (approximate near-neighbor) 问题：给定数据集和查询点 x_q ，以“高概率”找到 x_q 附近的一个或多个样例点。

19.7 非参数化模型 - 局部敏感哈希

■ 最近邻搜索：局部敏感哈希

- **哈希表**：给定样例 x_q ，计算哈希值 $g(x_q)$ ，每个哈希值对应一个哈希桶，具有相同哈希值的样本被存储在同一个桶中。
- **哈希函数** $g(x)$ ：使得邻近的数据点大概率拥有相同哈希值；远离的数据点则大概率哈希值不同。



19.7 非参数化模型 - 局部敏感哈希

■ 最近邻搜索：局部敏感哈希

- 思路：在高维空间中邻近的点，其在低维空间中的投影也大概率靠近。
- 方法：用一个**随机投影**将数据投影到一维空间（一条直线），之后将直线离散化为多个“哈希桶”，同一个桶中的数据拥有相同哈希值。
- 技巧：创建多个不同的随机投影，并随之创建多个哈希表 $g_1(x), g_2(x), \dots, g_l(x)$ ，对于查询点 x_q ，**与其邻近的点将有大概率在至少一个哈希表中与 x_q 具有相同哈希值**。查找哈希表获取所有哈希桶的并集。在并集中计算 x_q 的 k 近邻，作为查找到的近邻点。

19.7 非参数化模型 - 局部敏感哈希

■ 哈希函数

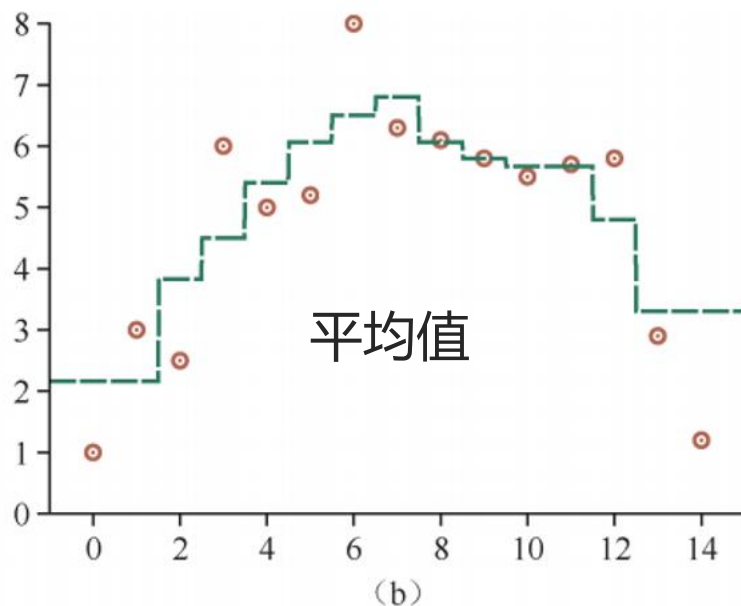
Name	Length	Type
BLAKE-256	256 bits	HAIFA structure ^[16]
BLAKE-512	512 bits	HAIFA structure ^[16]
BLAKE2s	up to 256 bits	HAIFA structure ^[16]
BLAKE2b	up to 512 bits	HAIFA structure ^[16]
BLAKE2X	arbitrary	HAIFA structure, ^[16] extendable-output functions (XOFs) design ^[17]
BLAKE3	arbitrary	Merkle tree
ECOH	224 to 512 bits	hash
FSB	160 to 512 bits	hash
GOST	256 bits	hash
Grøstl	up to 512 bits	hash
HAS-160	160 bits	hash
HAVAL	128 to 256 bits	hash
JH	224 to 512 bits	hash
LSH ^[18]	256 to 512 bits	wide-pipe Merkle–Damgård construction
MD2	128 bits	hash
MD4	128 bits	hash
MD5	128 bits	Merkle–Damgård construction
MD6	up to 512 bits	Merkle tree NLFSS (it is also a keyed hash function)
RadioGatún	arbitrary	ideal mangling function
RIPEMD	128 bits	hash
RIPEMD-128	128 bits	hash
RIPEMD-160	160 bits	hash
RIPEMD-320	320 bits	hash
SHA-1	160 bits	Merkle–Damgård construction
SHA-224	224 bits	Merkle–Damgård construction
SHA-256	256 bits	Merkle–Damgård construction
SHA-384	384 bits	Merkle–Damgård construction
SHA-512	512 bits	Merkle–Damgård construction
SHA-3 (subset of Keccak)	arbitrary	sponge function
Skein	arbitrary	Unique Block Iteration
Snefru	128 or 256 bits	hash
Spectral Hash	512 bits	wide-pipe Merkle–Damgård construction
Streebog	256 or 512 bits	Merkle–Damgård construction
SWIFFT	512 bits	hash
Tiger	192 bits	Merkle–Damgård construction
Whirlpool	512 bits	hash

Name	Tag Length	Type
BLAKE2		keyed hash function (prefix-MAC)
BLAKE3	256 bits	keyed hash function (supplied IV)
HMAC		
KMAC	arbitrary	based on Keccak
MD6	512 bits	Merkle tree NLFSS
One-key MAC (OMAC; CMAC)		
PMAC (cryptography)		
Poly1305-AES	128 bits	nonce-based
SipHash	32, 64 or 128 bits	non-collision-resistant PRF
HighwayHash ^[15]	64, 128 or 256 bits	non-collision-resistant PRF
UMAC		
VMAC		

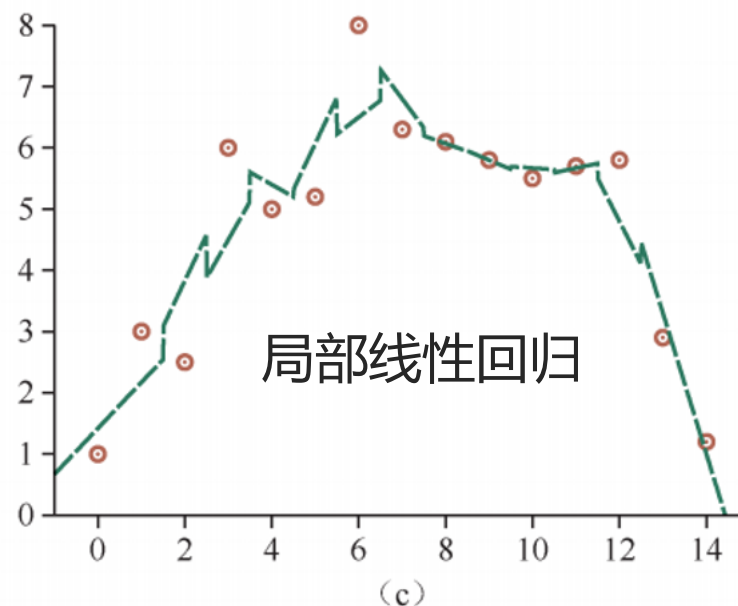
19.7 非参数化模型 - 非参数回归

■ k近邻回归

- 使用查询点 x_q 的k个最近邻对 x_q 的值进行预测。
 - ▶ 使用k近邻的平均值作为 x_q 的回归值（左图）
 - ▶ 利用k近邻训练一个线性回归模型获得 x_q 的回归值（右图）



不连续

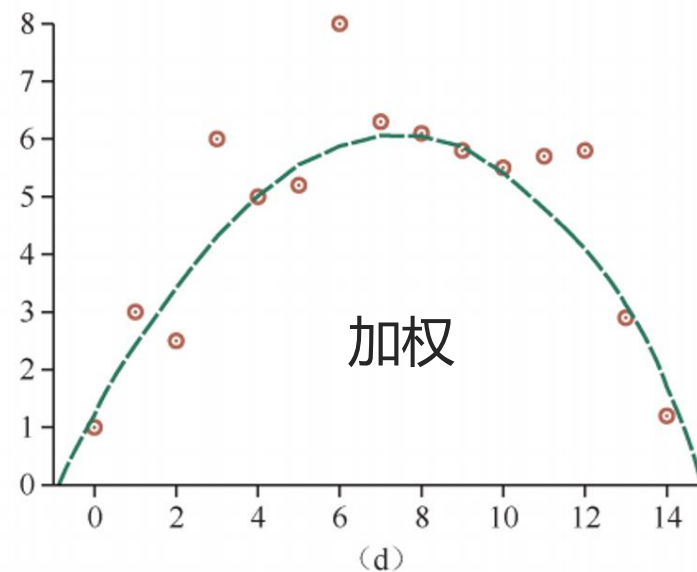
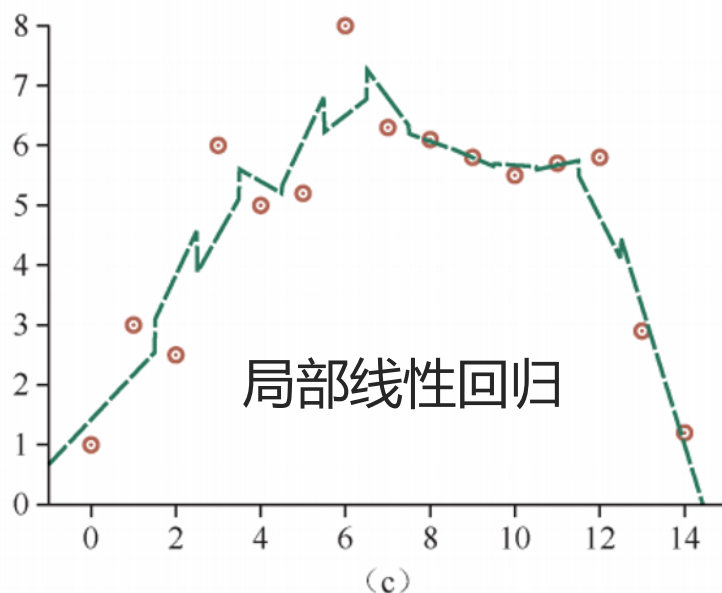


连续但不平滑

19.7 非参数化模型 - 非参数回归

■ 局部加权回归

- 靠近查询点 x_q 的样例具有高权重，远离 x_q 的样例权重逐渐减小，过远的样例将没有权重。
- 各样本的权重随距离渐变，而非突变

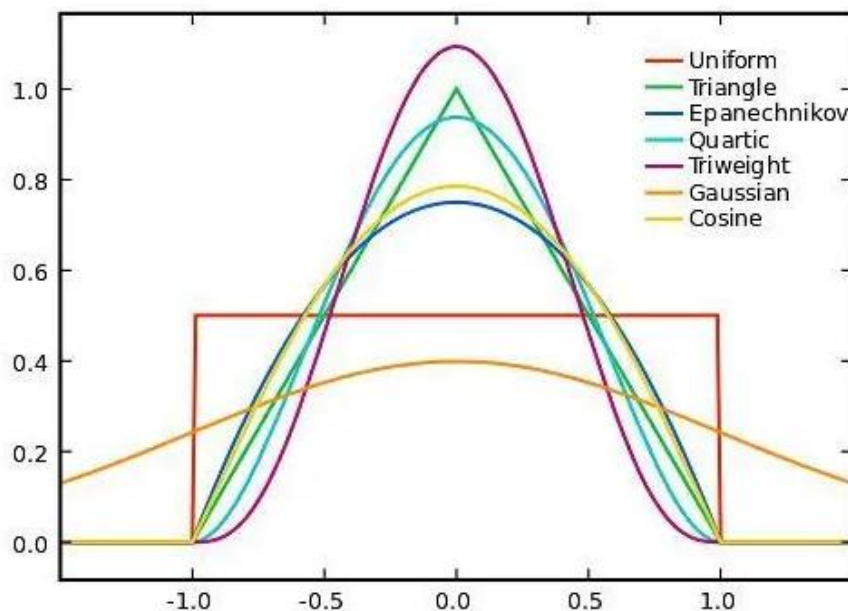


19.7 非参数化模型 - 非参数回归

■ 局部加权回归

□ 核函数(Kernel) : $\mathcal{K}(\text{Distance}(\mathbf{x}_j, \mathbf{x}_q))$

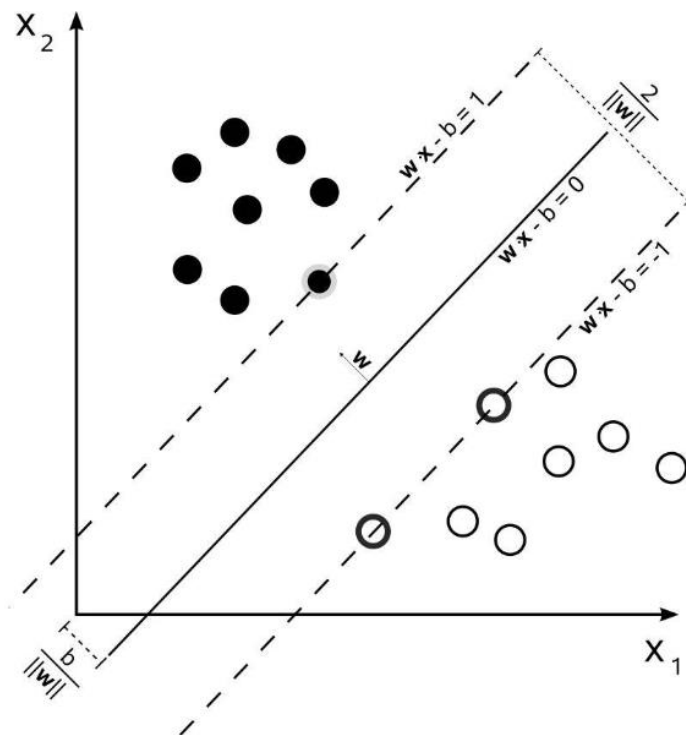
- ▶ **输入**：查询点与样例点之间的距离； **输出**：权重
- ▶ 相对于0，是对称的，并在0处达到最大值
- ▶ 核函数值在 x 的整个输入空间上的积分必须是有限的，一般令其积分为1。



19.7 非参数化模型 - 支持向量机

■ 支持向量机

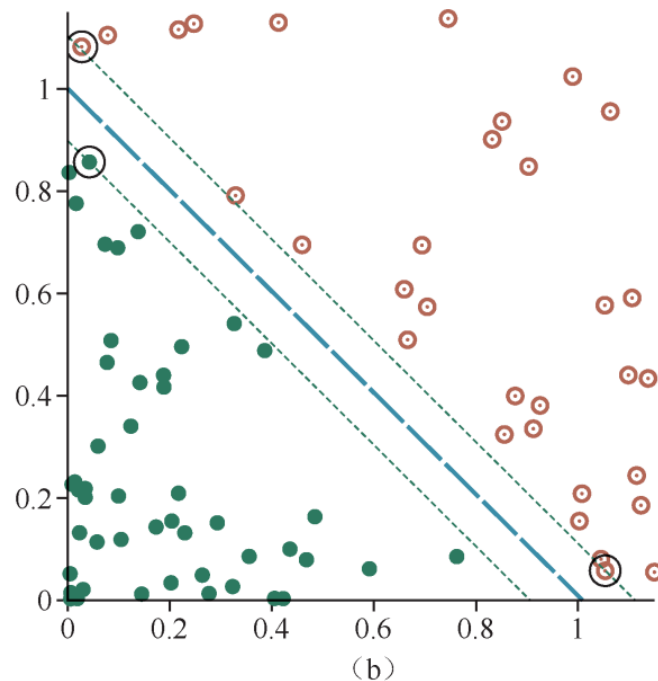
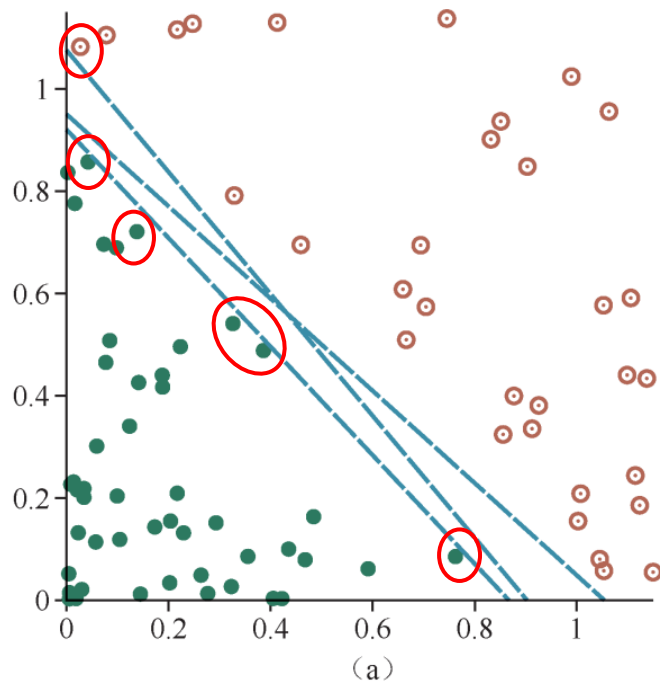
- 支持向量机 (support vector machine, SVM) 曾是最流行的解决监督学习问题的方法。在深度学习之前是具有最佳泛化能力的方法之一。



19.7 非参数化模型 - 支持向量机

■ 极大边距分离器

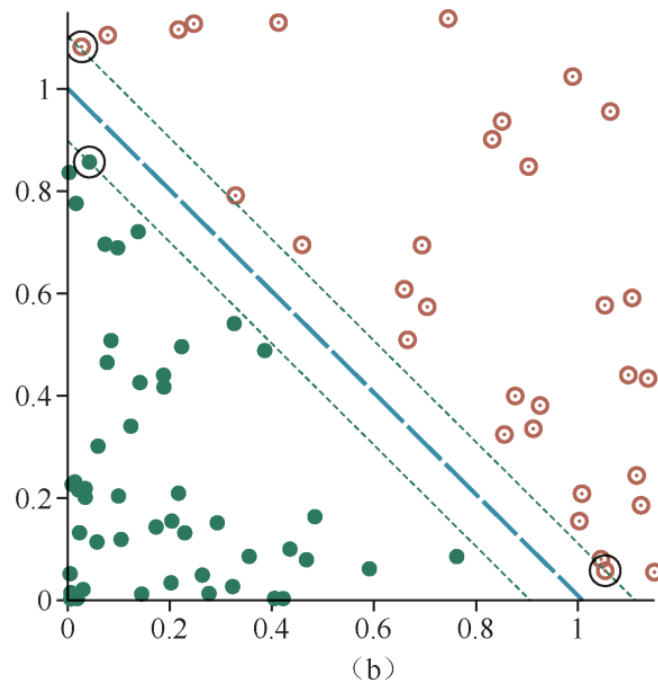
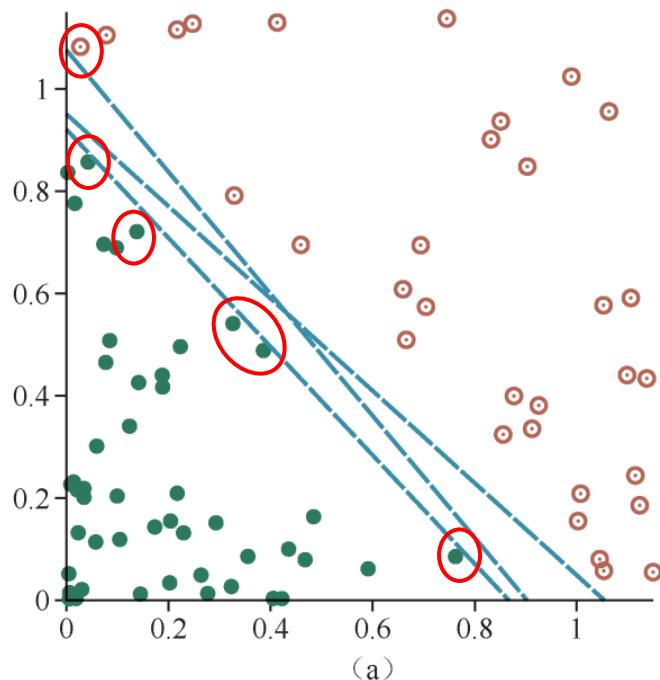
- 示例：左图是三个线性分类器，右图是极大边距分类器。
- 如果仅从经验误差的角度来看，它们都可以完美分类，是一样的。
- 风险：左图分类器有一些**非常接近超平面**的样本点，只要测试集与训练集稍有不同，就可能落在不正确的一侧导致失败。



19.7 非参数化模型 - 支持向量机

■ 极大边距分离器

- SVM：不是最小化训练数据上的期望**经验损失**，而是试图最小化**期望泛化损失**。
- 虽然不知道测试集的分布，他是应该与训练集同分布（一致性假设），**那么分类面与已经观察到的样本距离越远，越安全**。



19.7 非参数化模型 - 支持向量机

■ 支持向量机的定义：间隔的概念

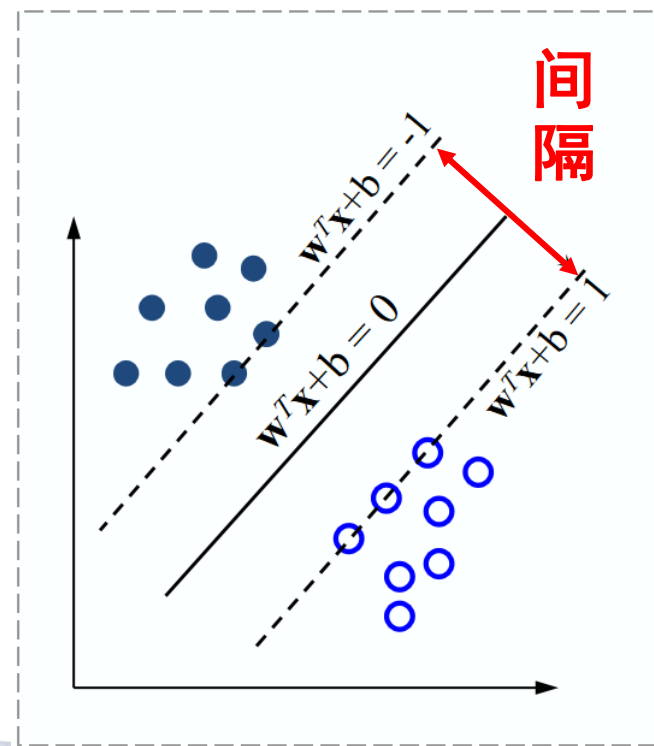
□ 设：分割超平面为 $w^T x + b = 0$ ，那么 $|w^T x + b|$ 表示 x 距离超平面的距离， y 与 $w^T x + b$ 的符号 (+1, -1) 是否一致表示分类是否正确。

□ 间隔 (Margin) 为：

$$r = y(w^T x + b)$$

□ w 和 b 的尺度不影响超平面位置，因此他们的绝对值不重要，相对值才重要。
对 w 归一化，那么间隔最终定义为：

$$r = \frac{1}{\|w\|} y(w^T x + b)$$



19.7 非参数化模型 - 支持向量机

■ 间隔最大化:

最大化最小间隔

$$\max_{\mathbf{w}, b} r, \quad s.t. \quad \frac{1}{\|\mathbf{w}\|} y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq r, \quad i = 1, 2, \dots, n$$



$$\max_{\mathbf{w}, b} \frac{\hat{r}}{\|\mathbf{w}\|}, \quad s.t. \quad y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq \hat{r}, \quad i = 1, 2, \dots, n$$

$$r_i = \frac{\hat{r}_i}{\|\mathbf{w}\|}, \quad r = \frac{\hat{r}}{\|\mathbf{w}\|}$$



$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2, \quad s.t. \quad y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, n$$

$$(\text{令: } \hat{r} = 1)$$

梯度下降法搜索w和b，或者使用对偶方法

19.7 非参数化模型 - 支持向量机

■ 支持向量机的最优解

□ SVM优化问题的对偶问题:

$$\begin{aligned} \operatorname{argmax}_{\alpha} \quad & \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k) \\ \text{s.t.} \quad & \alpha_j \geq 0 \quad \sum_j \alpha_j y_j = 0 \end{aligned}$$

□ 这是一个二次规划问题，使用软件包很容易求解，最终得到分类平面为：

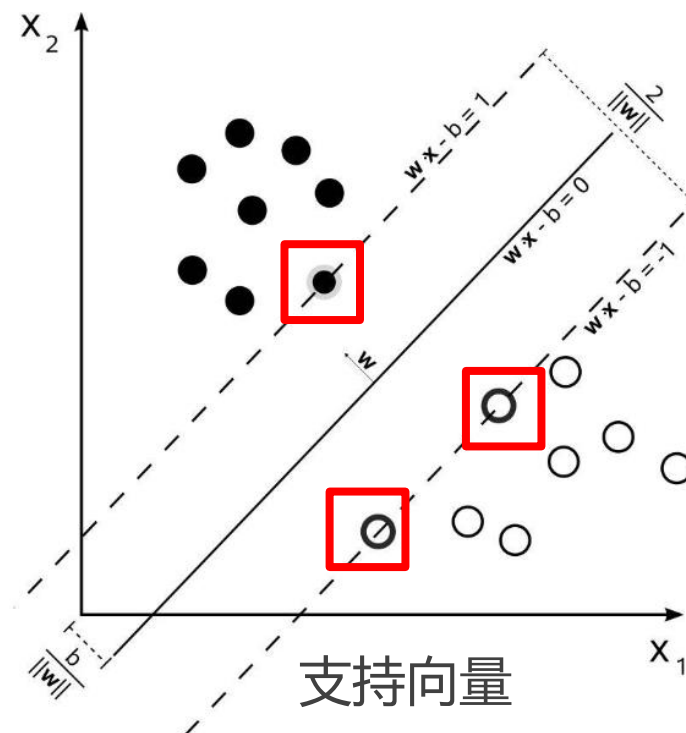
$$h(x) = \operatorname{sign}\left(\sum_j \alpha_j y_j (x \cdot x_j) - b\right)$$

19.7 非参数化模型 - 支持向量机

■ 支持向量机的最优解

- **重要性质**：除支持向量外，每个数据点对应的权重 α_j 为 0。那么仅有支持向量样例决定了分类面的位置而与其他样例无关。
- **支持向量**： support vector，距离分类器最近的样例。出现新的样本时，只要不落入虚线框内，就不会对分类器产生影响。

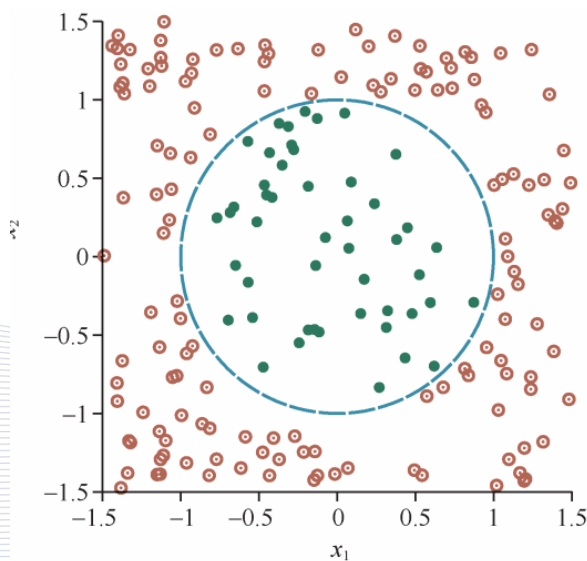
$$h(x) = \text{sign}\left(\sum_j \alpha_j y_j (x \cdot x_j) - b\right)$$



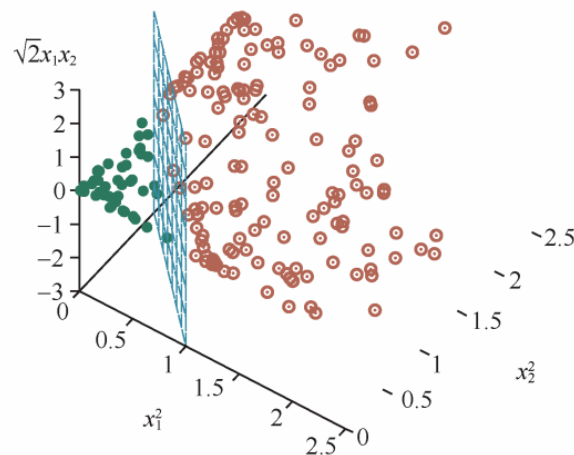
19.7 非参数化模型 - 支持向量机

■ 核技巧：应对线性不可分数据

- 左图：数据不是线性可分的。
- 重新表示输入数据，将 x 映射到新的特征值向量 $F(x)$ ，将 (x_1, x_2) 映射到 $F(x) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$ ，对应右图的三维空间。
- 右图：三维空间中数据点变为了线性可分的。将数据映射到足够高的维度时，它们几乎总是线性可分的。



(a)



(b)

19.7 非参数化模型 - 支持向量机

■ 核技巧：应对线性不可分数据

- 在实际问题中，在SVM的对偶问题中，使用 $F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)$ 替换 $\mathbf{x}_j \cdot \mathbf{x}_k$ ，即可在高维空间中找到线性分离器。

$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)$$



$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)$$



$$F(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)^2$$

19.7 非参数化模型 - 支持向量机

■ 核技巧：应对线性不可分数据

- 在对偶问题中，不需要指定特征空间 $F(x)$ ，只需要指定特征空间下的内积 $F(x_j) \cdot F(x_k)$ （核函数）。

$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)$$



$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)$$



$$F(\mathbf{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)^2$$

不需要 $F(\mathbf{x}_j)$

$(\mathbf{x}_j \cdot \mathbf{x}_k)^2$ 被称为核函数，通常写作 $K(\mathbf{x}_j, \mathbf{x}_k)$

19.7 非参数化模型 - 支持向量机

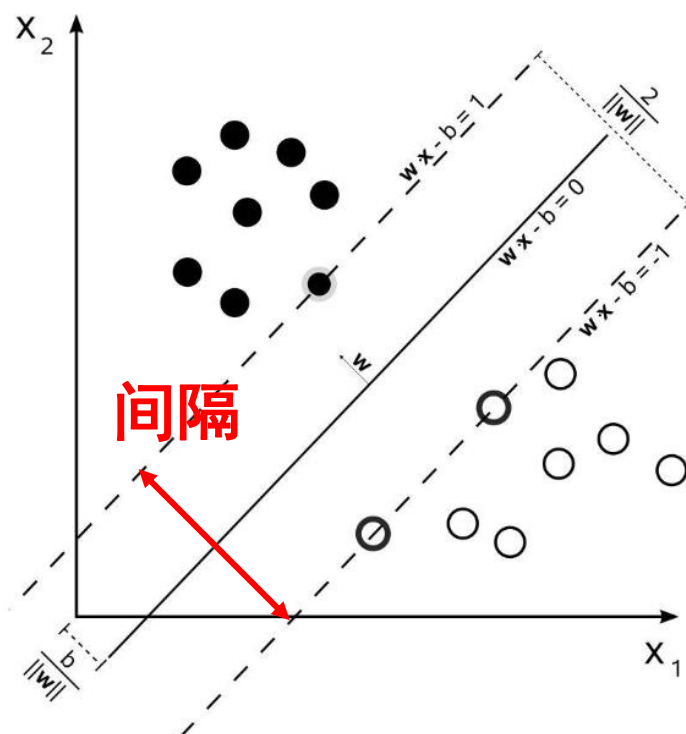
■ 核技巧：应对线性不可分数据

- 优化SVM时，使用核函数来替换内积时，只需要计算核函数，而无需计算每个点的特征，任何合理核函数都对应于某个特征空间。
 - ▶ 需要将数据映射到某个高维空间时，不需明确地给出映射函数 $F(x)$ ，而只需给出该空间内积 $F(x_i) \cdot F(x_j)$
 - ▶ 内积的物理意义是两个样本的相似度，因此在核技巧下，只需给出一个基于 x 的相似度度量即可。
 - ▶ 设计相似度度量比定义高维空间要直观、简单。
- 多项式核 $K(x_j, x_k) = (1 + x_j \cdot x_k)^d$ 对应一个 d 维特征空间。
- 高斯核 $K(x_j, x_k) = e^{-\gamma |x_j - x_k|^2}$ 也是常用的核函数。

19.7 非参数化模型 - 支持向量机

■ 支持向量机——优势一

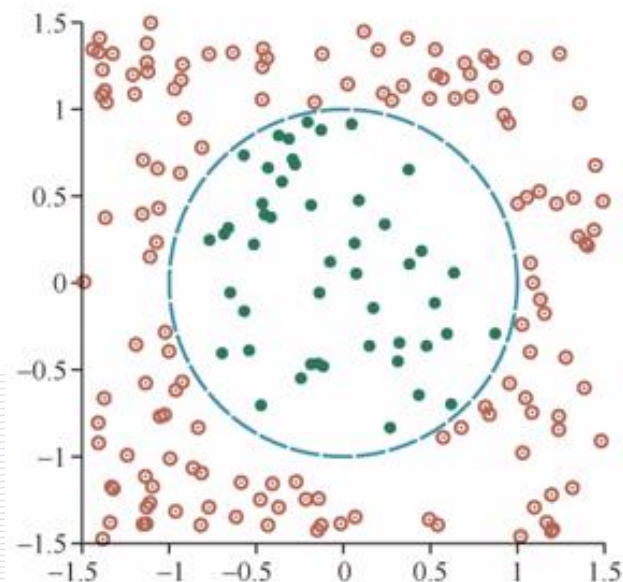
- 支持向量机 (support vector machine, SVM) 构造**极大边距分离器 (Margin)**，即到样例点距离最大的决策边界。有助于较好地泛化。



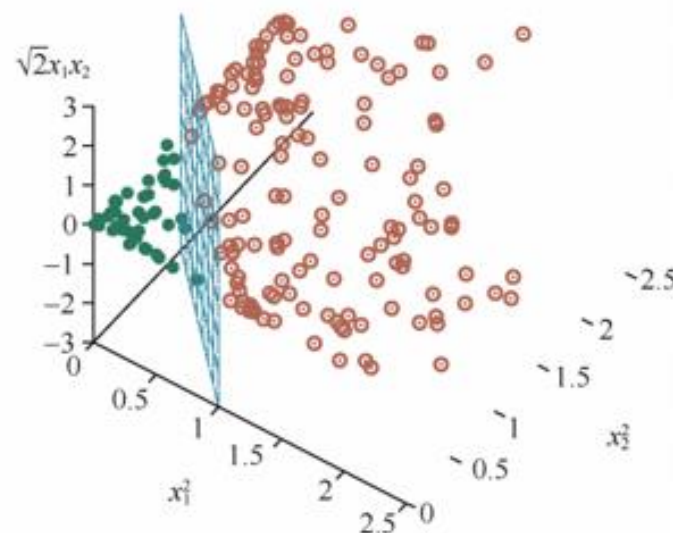
19.7 非参数化模型 - 支持向量机

■ 支持向量机——优势二

- 虽然SVM构造线性分离超平面，但能通过核技巧将数据嵌入更高维空间中。通常原始输入空间中不是线性可分的数据很容易在高维空间中被分离。



核技巧

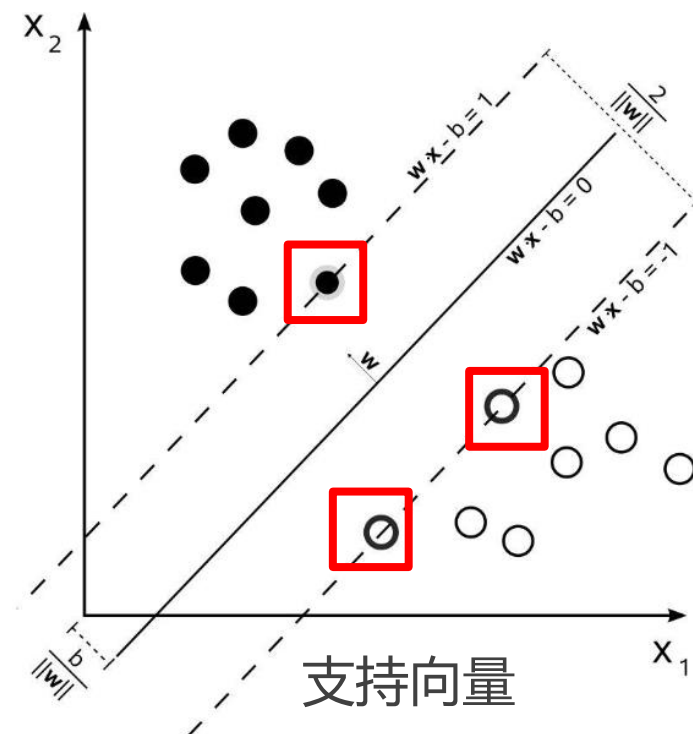


19.7 非参数化模型 - 支持向量机

■ 支持向量机——优势三

□ SVM的分离超平面是由一组样例点而不是参数值的集合定义的。

- ▶ 相比于最近邻等模型需要保留所有样例的信息，它仅需要保留最靠近分离平面的样例。
- ▶ 因此，SVM 结合了非参数模型和参数模型的优点：它们可以灵活地表示复杂的函数，同时可以防止过拟合。



- 学习形式
- 监督学习
- 决策树学习
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

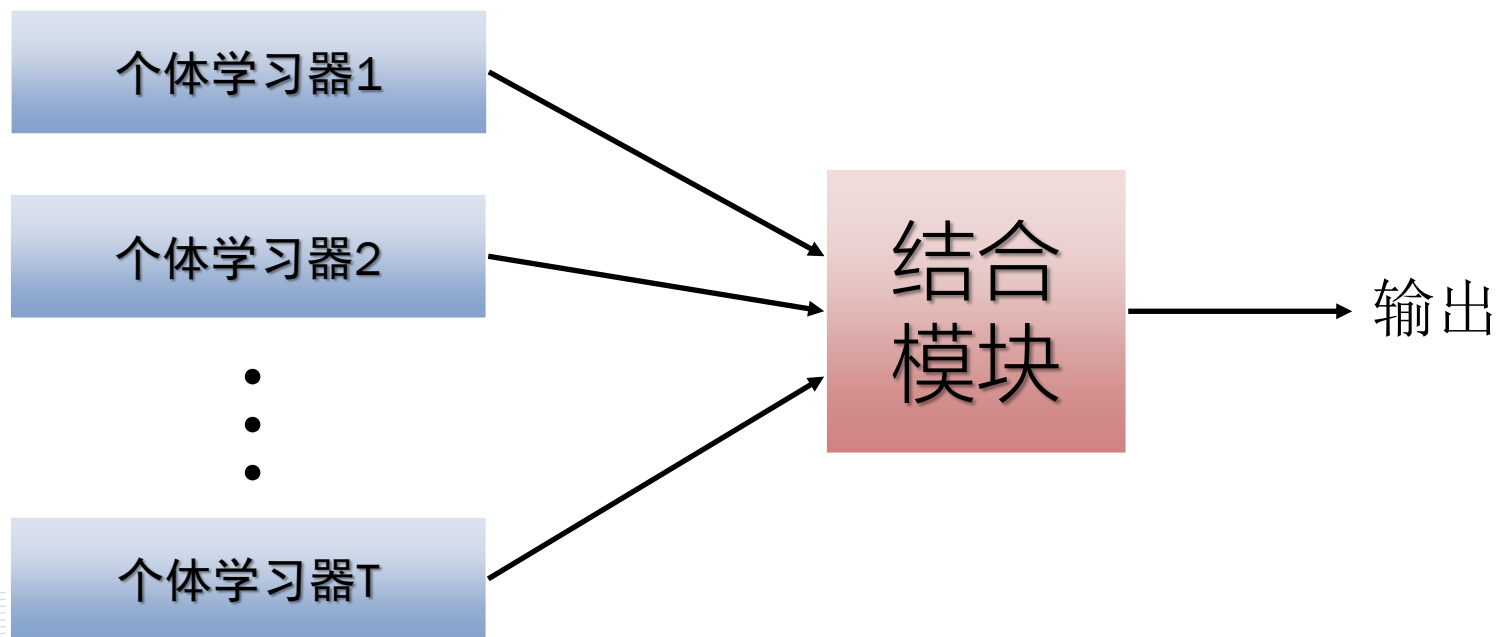
19.8 集成学习

■ 集成学习的概念

- **集成学习** (ensemble learning) : 选择一个由一系列假设 $h_1, h_2, \dots, h_n$ 构成的集合, 通过平均、投票或其他形式的机器学习方法将它们的预测进行组合。
- 单独的假设为**基模型** (base model)
- 其组合后的模型为**集成模型** (ensemble model) 。
- 集成模型可以帮助减少**偏差与方差**。
- 现在也称为模型融合。

19.8 集成学习

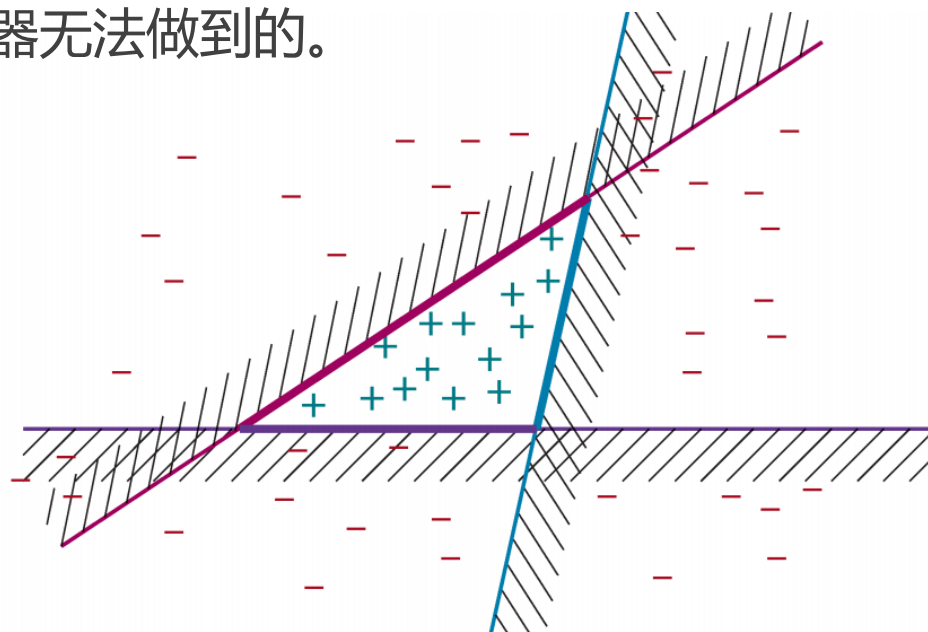
■ 集成学习的概念



19.8 集成学习

■ 集成学习的好处

- **减小偏差**（训练集误差）：基模型的假设空间可能过于局限，造成了很大的偏差。例如线性决策边界无法分离线性不可分的数据。而集成模型则具有更强的表达能力。
- 举例：三个线性分类器，构成了一个三角形的分类边界，这是单个线性分类器无法做到的。



19.8 集成学习

■ 集成学习的好处

- **减小方差**（测试集误差）：对于一个二分类问题，使用5个分类器进行集成，按照投票表决且少数服从多数的方式决定分类结果。那么，需要有至少3个分类器的分类结果错误，才会导致集成模型的结果错误。
- 最理想情况下，我们将5个分类器分别在训练集的5个不同子集上进行训练，从而得到5个独立的分类器，假设每个分类器的正确率为75%，那么5个模型集成的正确率将达到89%。（当然这些模型不可能独立，但总会有一些提升）

19.8 集成学习

■ 集成学习的好处

(1)

	测试例1	测试例2	测试例3
h_1	✓	✓	✗
h_2	✗	✓	✓
h_3	✓	✗	✓
集成	✓	✓	✓

集成提升性能

(2)

集成个体应“好而不同”
准确性 vs 多样性

(3)

	测试例1	测试例2	测试例3
h_1	✓	✗	✗
h_2	✗	✓	✗
h_3	✗	✗	✓
集成	✗	✗	✗

集成起负作用

19.8 集成学习 - 自助聚合法

■ 自助聚合法

- 从原始数据集中，采样 K 个子训练集，每个训练集中包含从原始数据集中采样的 N 个样例。
- 在这 K 个子训练集上分别运行机器学习算法，并获得 K 个假设。
- 预测时，汇总来自所有 K 个假设的预测
 - ▶ 对于分类问题，采用相对多数投票的方式决定最终的预测结果；
 - ▶ 对于回归问题，则使用平均值作为输出：

$$h(x) = \frac{1}{K} \sum_{i=1}^K h_i(x)$$

- 自助聚合法有助于减少方差，解决过拟合的情况。

19.8 集成学习 - 堆叠法

■ 堆叠法

- **堆叠泛化**方法则使用相同数据，训练多个模型类下的不同模型。
 - ▶ 使用同一个训练集，训练来自多个模型类的模型，如SVM，逻辑回归，决策树等。
 - ▶ 根据基模型的预测结果，训练结合模型，主要为各模型的权重。
 - ▶ 实际中，多个团队可以根据自己的喜好**独立训练模型**，最终将训练好的模型进行堆叠集成。

19.8 集成学习 - 自适应提升法

■ 自适应提升法 (boosting)

- boosting调整数据集中各样本的权重，使训练集变为**加权训练集**。
例如，一个样例的权重是3，相当于在训练集中有3个这个样例的副本。
- boosting的具体流程：
 - ▶ 首先将所有样例的权重初始化为 $w_j = 1$ ，并训练第一个假设 h_1
 - ▶ 增大 h_1 分类错误的样例的权重，并减小分类正确的样例的权重，重新训练得到假设 h_2 。（先前分错的样例更受重视）
 - ▶ 重复以上过程，最终得到 K 个假设，通过加权投票的方式来获得最后的预测结果：在各自的加权训练集上表现得更好的假设将获得更大的投票权重。（表现好的模型话语权大）

19.8 集成学习 - 自适应提升法

■ 自适应提升法变体：AdaBoost

function ADABOOST(*examples*, *L*, *K*) **returns** 一个假设

inputs: *examples*, 由 N 个带标签的样例 $(x_1, y_1), \dots, (x_N, y_N)$ 组成的集合

L, 学习算法

K, 集成中的假设个数

local variables: $\mathbf{w}$, 代表样例权重的 N 维向量, 初始为全 $1/N$

$\mathbf{h}$, 代表 K 个假设的向量

$\mathbf{z}$, 代表 K 个假设权重的向量

$\varepsilon \leftarrow$ 一个小的正数, 用于规避除以零的情况

for $k = 1$ **to** K **do**

$\mathbf{h}[k] \leftarrow L(\text{examples}, \mathbf{w})$

$\text{error} \leftarrow 0$

for $j = 1$ **to** N **do** // 计算 $\mathbf{h}[k]$ 的总错误率

if $\mathbf{h}[k](x_j) \neq y_j$ **then** $\text{error} \leftarrow \text{error} + \mathbf{w}[j]$

if $\text{error} > 1/2$ **then break** from loop // 当得到的假设比随机猜测性能差, 就停止生成假设

$\text{error} \leftarrow \min(\text{error}, 1 - \varepsilon)$

for $j = 1$ **to** N **do** // 赋予 $\mathbf{h}[k]$ 出错的样例更大的权重

if $\mathbf{h}[k](x_j) = y_j$ **then** $\mathbf{w}[j] \leftarrow \mathbf{w}[j] \cdot \text{error} / (1 - \text{error})$

$\mathbf{w} \leftarrow \text{NORMALIZE}(\mathbf{w})$

$\mathbf{z}[k] \leftarrow \frac{1}{2} \log((1 - \text{error}) / \text{error})$ // 赋予正确的 $\mathbf{h}[k]$ 更大的权重

return $\text{Function}(x): \sum \mathbf{z}_i \mathbf{h}_i(x)$

训练了 k 个模型

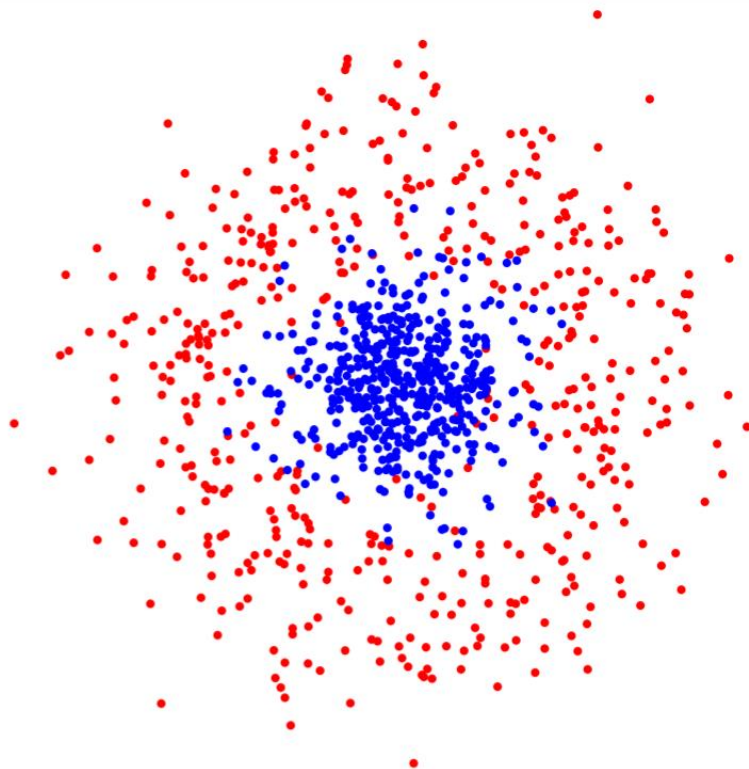
计算每个模型的错误率

正确样例更小的权重

总错误率更小的模型
更大的权重

19.8 集成学习

■ 示例:



二分类问题

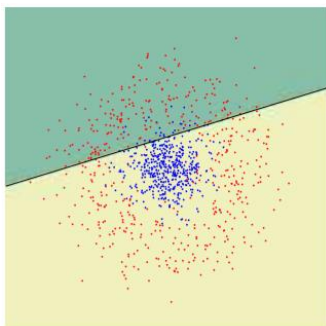
蓝点 $\sim N(0,1)$

红点 $\sim \frac{1}{r\sqrt{8\pi^3}} e^{-1/2(r-4)^2}$

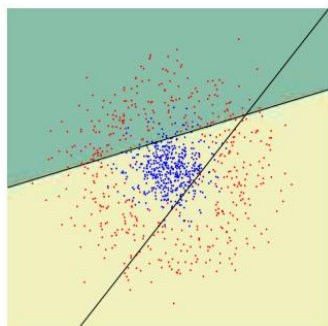
19.8 集成学习

■ 示例:

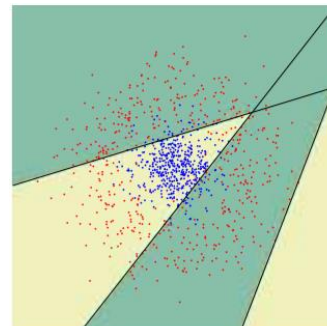
$m = 1$



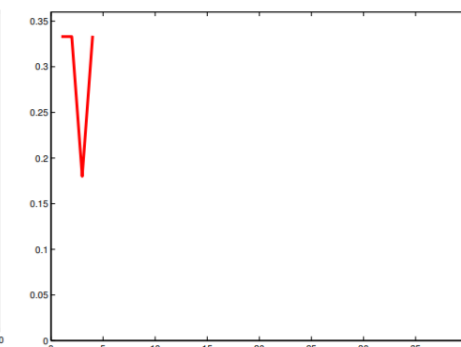
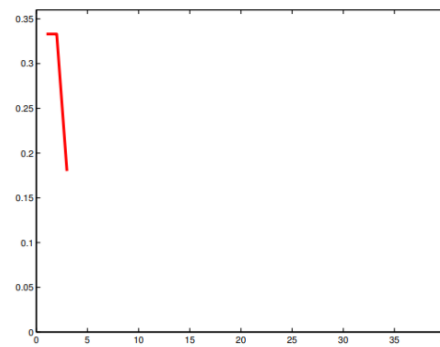
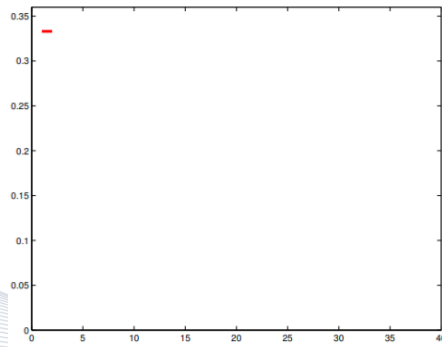
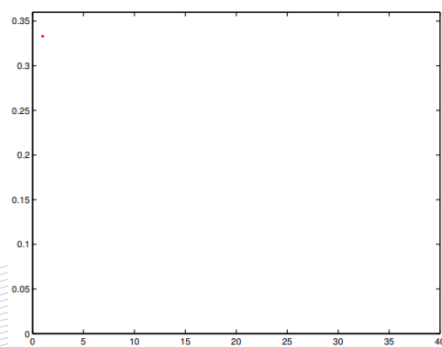
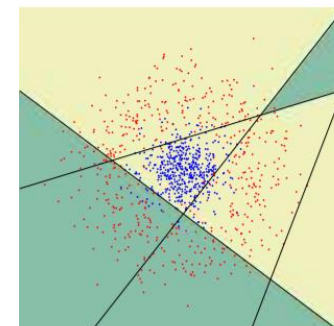
$m = 2$



$m = 3$



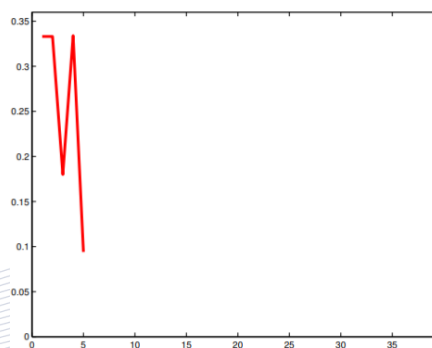
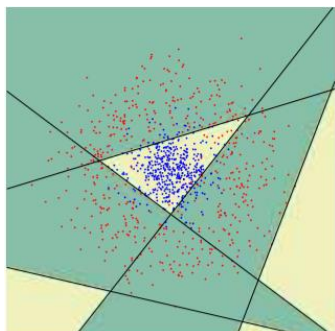
$m = 4$



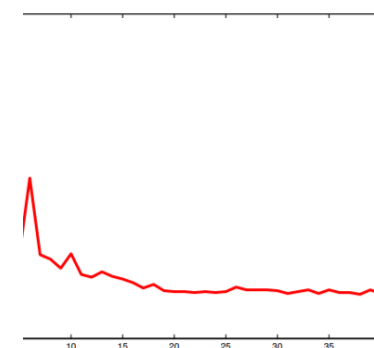
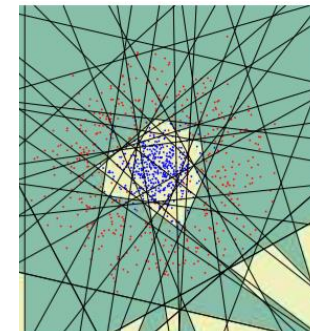
19.8 集成学习

■ 示例:

$m = 5$



$m = 40$



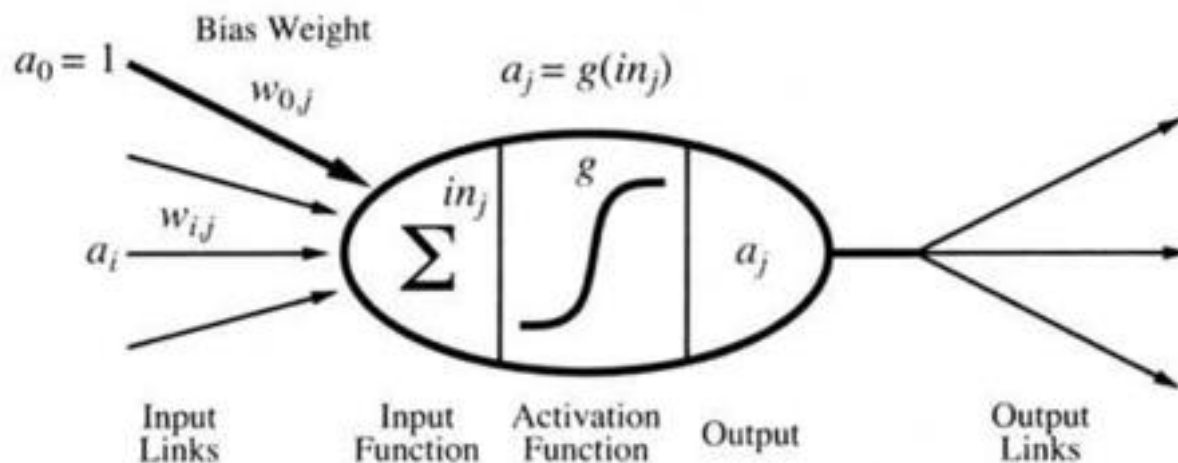
训练样本的最终
权重分布

- 学习形式
- 监督学习
- 决策树学习
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

人工神经网络

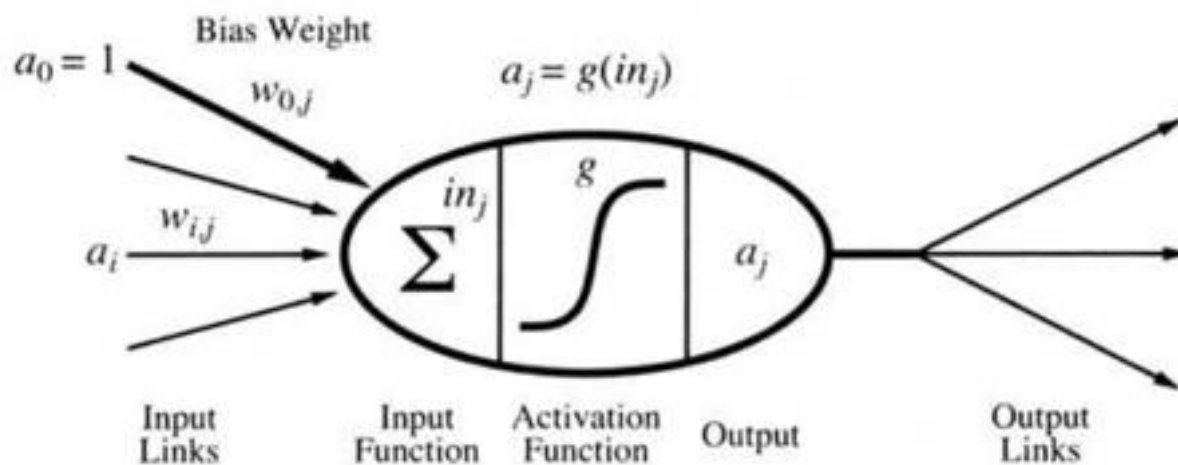
■ 基本组件：神经元

- 基于生物学中神经网络的基本原理，在理解和抽象了人脑结构和外界刺激响应机制后，以网络拓扑知识为理论基础，模拟人脑的神经系统对复杂信息的处理机制的一种数学模型。



人工神经网络

■ 基本组件：神经元



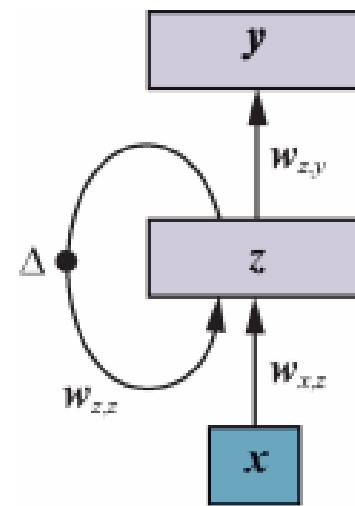
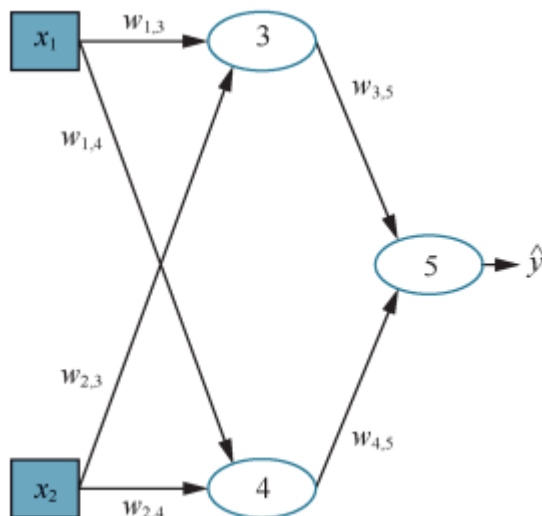
核心公式：
$$a_j = g\left(\sum_{i=0}^n w_{ij} a_i\right)$$

- 神经网络由结点或单元组成，单元通过有向链连接。
- **连接权重：** 每个链有一个数值权重 $w_{i,j}$ ，决定连接强度和符号。
- **激活函数：** logistic函数， Relu函数

人工神经网络

■ 基本架构

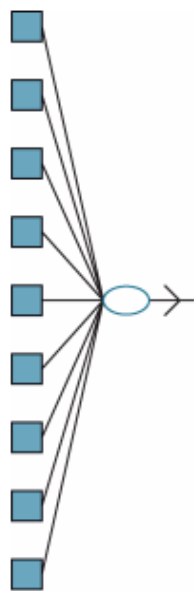
- 前馈网络：有向无环图且有指定的输入和输出节点。每个节点计算一个关于输入的函数，并将结果传递给网络中的后续节点。
- 循环网络：将其中间输出或最终输出反馈到自己的输入中。这意味着网络中的信号值将形成一个具有内部状态或记忆的动态系统。



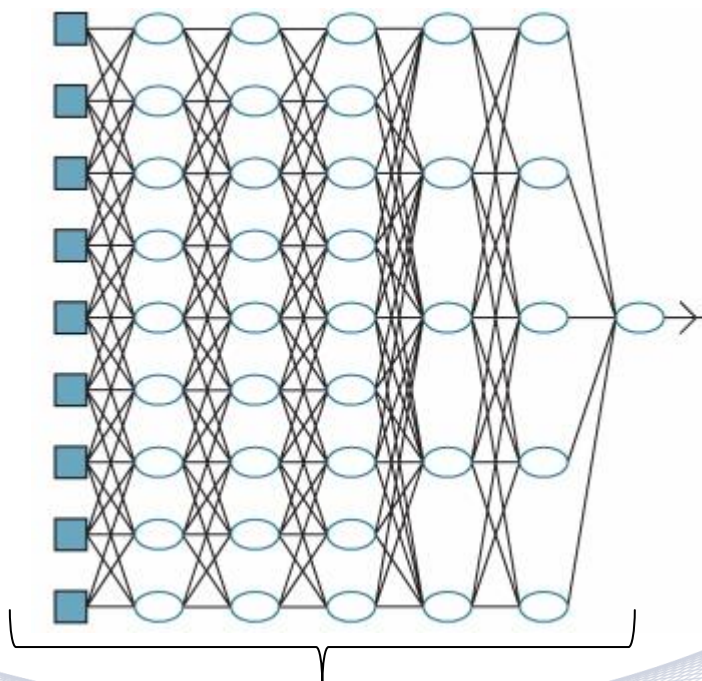
人工神经网络

■ 基本架构

- 感知机网络：所有输入直接连接到输出的网络称为单层神经网络。
- 深度学习网络：除输入层和输出层外，引入大量隐藏层。强调隐藏层的层数（深度），是深度学习网络的特色。



单层感知机

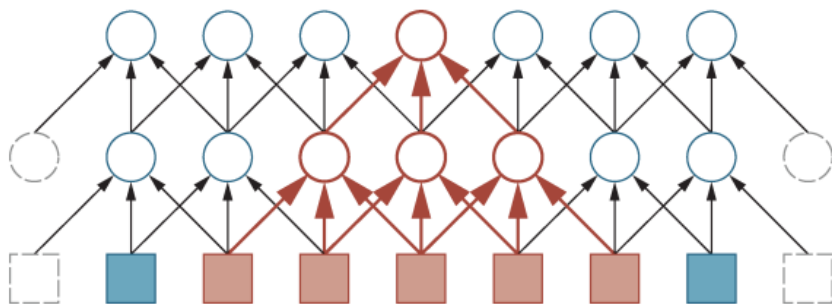


深度

人工神经网络

■ 基本架构

- 卷积神经网络：每个隐藏单元只接收来自输入的一个小局部区域的输入。
 - ▶ 在局部上注重了邻接关系（马尔可夫性），减少了权重的数量
- Transformer：对每一个输入结点添加位置编码，使用自注意力机制（Self-Attention）交换结点间的信息，生成下一层的结点。



卷积神经网络（CNN）

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Query

 $Q = W^Q X$

查询

Key

 $K = W^K X$

索引

Value

 $V = W^V X$

内容

自注意力机制

人工神经网络

■ 人工神经网络的训练

□ 随机梯度下降

- ▶ 每次随机选取一批数据(batch)，基于损失函数计算输出层结点的连接权重的梯度。

□ 反向传播算法

- ▶ 基于反向传播算法将输出层的梯度向下传播，获得隐藏层权重的梯度。
- ▶ 深度学习软件包通常会使用自动微分方法进行计算，获得任何数值程序的梯度（只要程序每个部件都可导）

□ 更新

- ▶ 对所有连接向负梯度方向进行更新 $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(w)$

人工神经网络

■ 人工神经网络的训练

```

function BACK-PROP-LEARNING(examples, network) returns a neural network
  inputs: examples, a set of examples, each with input vector  $x$  and output vector  $y$ 
           network, a multilayer network with  $L$  layers, weights  $w_{i,j}$ , activation function  $g$ 
  local variables:  $\Delta$ , a vector of errors, indexed by network node

  repeat
    for each weight  $w_{i,j}$  in network do
       $w_{i,j} \leftarrow$  a small random number
    for each example  $(x, y)$  in examples do
      /* Propagate the inputs forward to compute the outputs */
      for each node  $i$  in the input layer do
         $a_i \leftarrow x_i$ 
      for  $\ell = 2$  to  $L$  do
        for each node  $j$  in layer  $\ell$  do
           $in_j \leftarrow \sum_i w_{i,j} a_i$ 
           $a_j \leftarrow g(in_j)$ 
      /* Propagate deltas backward from output layer to input layer */
      for each node  $j$  in the output layer do
         $\Delta[j] \leftarrow g'(in_j) \times (y_j - a_j)$ 
      for  $\ell = L - 1$  to  $1$  do
        for each node  $i$  in layer  $\ell$  do
           $\Delta[i] \leftarrow g'(in_i) \sum_j w_{i,j} \Delta[j]$ 
      /* Update every weight in network using deltas */
      for each weight  $w_{i,j}$  in network do
         $w_{i,j} \leftarrow w_{i,j} + \alpha \times a_i \times \Delta[j]$ 
  until some stopping criterion is satisfied
  return network
  
```

图 18.24 多层网络中学习的反向传播算法

- 学习形式
- 监督学习
- 决策树学习
- 模型选择与模型优化
- 学习理论
- 线性回归和分类
- 非参数化模型
- 集成学习
- 人工神经网络
- 开发机器学习系统

19.9 开发机器学习系统

- 开发机器学习系统
 - 问题形式化
 - 数据收集、评估和管理
 - 数据预处理
 - 模型选择与训练
 - 信任

19.9 开发机器学习系统 - 问题形式化

■ 建模待解决的问题

首先需要弄清楚待解决的问题，并数学化，程序化：

- 第一部分是**我要为用户解决什么问题**：这个问题需要尽量精确，例如“帮助用户寻找与特定词条匹配的所有照片”
- 第二部分是**机器学习可以解决该问题的哪些部分**：例如，在解决上一部分提出的问题时，我们可以“学习将照片映射到一组标签的函数；在给定标签进行查询时，我们检索带有该标签的所有照片”

19.9 开发机器学习系统 - 问题形式化

■ 确定学习的类型

问题的形式化还需要确定学习的类型，即使用监督学习、无监督学习还是强化学习来解决问题。例如：

- **半监督学习**：我们拥有部分带有标注的数据，并利用这些标注从大量无标注的数据中挖掘更多的信息。
- **弱监督学习**：标签不一定精确的学习过程。例如，年龄估计中，人们在网上上传的照片与他们报告的年龄有时并不匹配。弱监督学习关注的问题就是当标签有噪声、不精确或被随意提供时的情形。

19.9 开发机器学习系统 - 数据收集、评估和管理

■ 数据的来源

每一个机器学习项目都需要数据

- 互联网是数据最大的来源地，优先考虑是否能从互联网收集数据。
 - ▶ 在图像识别项目中，我们有免费的图像数据集可以使用，如 **ImageNet**，它拥有 1400 多万张图像，其标签大约有 2 万种。
- 公开的学术数据库。
- 有时可能不得已需要自己制造数据，要考虑收集流程，成本和合法性。

19.9 开发机器学习系统 - 数据收集、评估和管理

■ 数据预处理

- **数据增强**：通过对每个图像进行旋转、平移、裁剪、缩放、更改亮度或颜色平衡及添加噪声来创建这个图像的多个版本，图像标签保持不变。在这种增强数据上训练得到的模型将更加有效。
- **长尾问题**：数据量足够时，也可能出现类别不平衡的情况。例如，一些类别样例很多，一些类别样例很少。这时，可以对多数类别进行**欠采样**，以及对少数类别进行**过采样**。
- **数据清洗**：**离群值**是应该仔细考虑的数据，如线性回归等方法很容易受到离群值影响，因为它无法将离群值与正常数据区别对待，使得单个的离群值可能对模型参数产生很大影响。

19.9 开发机器学习系统 - 数据收集、评估和管理

■ 特征工程

我们还需要对数据的特征进行预处理，使得它便于模型使用。

- ▣ 连续值输入可以通过区间分隔进行离散化，如餐厅等待问题中，等待时间分为0~10，10~30，30~60，>60分钟四类。分隔的阈值可以由领域知识确定，例如以18岁为阈值区分成年/未成年人群。
- ▣ 对数据进行归一化，使得标准差为1，会使模型更容易收敛。

19.9 开发机器学习系统 - 数据收集、评估和管理

■ 探索性数据分析与可视化

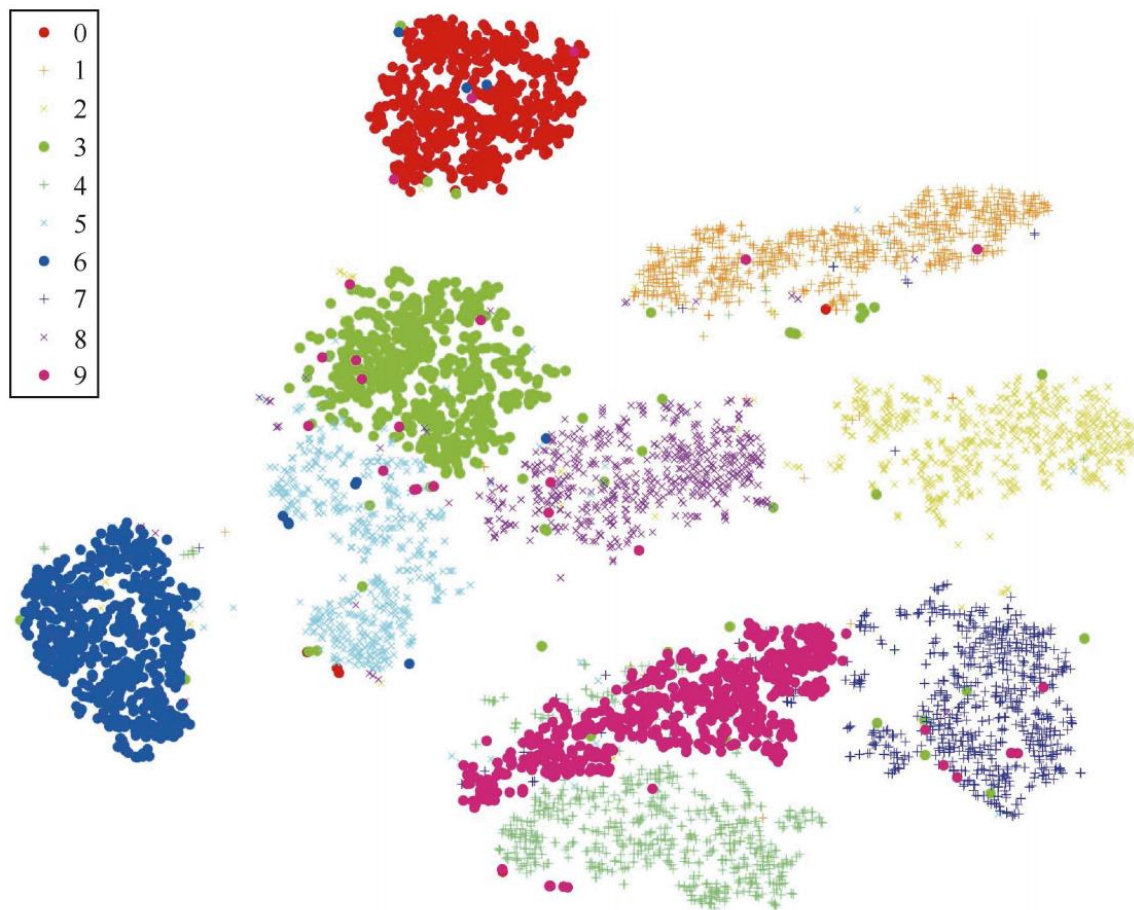
通过**可视化**或**简要的统计**来探索数据以获得对数据的**直观理解**，而不是直接进行预测或测试假设。也可以通过简要的统计完成。

- 可视化有助于确定数据是否有规律，观察数据是正态分布还是长尾分布，以及判断什么样的学习模型可能是合适的。
- 一个有效的方法是对数据进行聚类，然后在每个聚类簇的中心可视化一个原型数据点。例如，在图像的数据集中，我们可以识别出一簇猫脸，这簇数据的附近是一簇熟睡的猫，而其他聚类簇描绘了其他对象。

19.9 开发机器学习系统 - 数据收集、评估和管理

■ t 分布随机近邻嵌入 (t-SNE)

- 将高维数据降维到2维可视化，并尽量保持样例之间的关系



19.9 开发机器学习系统 - 模型选择与训练

■ 建立模型的步骤

有了干净的数据和对数据的直观感觉，可以建立模型了。包括以下步骤：

- 选择一个模型类
- 使用训练数据训练模型
- 利用验证数据来调优模型类的所有超参数
- 最后用测试数据对模型进行评估。

19.9 开发机器学习系统 - 模型选择与训练

■ 模型选择的经验

- 有很多特征，但特征之间不相关，随机森林是好用的；
- 当数据量很大，但没有足够先验知识时，可以选择非参数方法；
- 对线性可分数据，可以使用逻辑回归；
- 对少量的数据，可以使用SVM；
- 对于图像或语音处理问题，通常用深度神经网络。数据量非常大的问题也适合用神经网络。

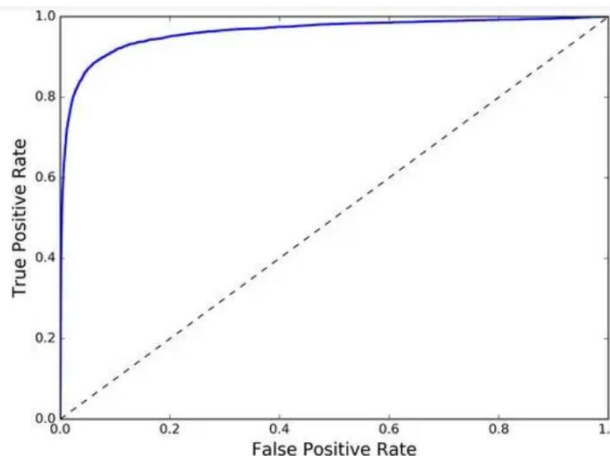
19.9 开发机器学习系统 - 模型选择与训练

■ 模型评估的可视化工具

- **接收者操作特征曲线 (ROC)**：X轴为假阳性率，Y轴为真阳性率的曲线；评估分类器性能随着阈值变化的过程。**ROC曲线下面积 (AUC)** 为 ROC 曲线提供了一个单数值摘要。

假阳性率：为False的样本，错分类为True的概率。

真阳性率：为True的样本，分类为True的概率。



X轴越小，评估越严格。
Y轴越大，性能越高。

- **混淆矩阵 (confusion matrix)**：二维计数表，它显示了每个类别被正确分类或误分类的频率。

19.9 开发机器学习系统 - 信任、可解释性、可说明性

■ 可解释性 (Interpretability): How

- 如果能观察**模型的实际行为**，即**理解为什么它会在得到给定输入后给出的特定输出**，以及当输入发生变化时输出将如何变化，我们就称此机器学习模型是**可解释的**。
- 例如，决策树被认为是可解释的。首先，我们人类有理解 if/then 规则的经验；其次，决策树在某种意义上是按照可解释性进行构造的，树的根节点就是具有最高信息增益的属性。
- 线性回归模型也被认为是可解释的：根据一个预测公寓租金的模型，每增加一间卧室，租金就会增加 500 美元。其中的想法“如果我改变 x ，输出会怎样变化？”是可解释性的核心。当然，相关性不是因果关系，所以可解释的模型所解释的是**会发生什么**，但不一定是**为什么**。

19.9 开发机器学习系统 - 信任、可解释性、可说明性

■ 可说明性 (Explainability): Why

- 一个可说明的模型可以帮助理解“为什么某个输入对应着某个输出”。模型本身可能是一个很难理解的黑匣子，对于一个将某图片分类为狗的神经网络，
- 坏的可说明性：“分类层中输出为狗的激活性高于任何其他类”（没有任何意义）；
- 好的可说明性：“它有 4 条腿、皮毛、尾巴、松软的耳朵和长长的鼻子；它比狼小，躺在狗床上，所以我认为它是狗”。
- **相比于说明，测试可能会给你更多的信心和信任。**例如两个机器的对比：一架从未飞行过但详细说明了为什么它是安全的实验飞机，或者一架安全地完成了 100 次飞行并经过精心维护但却没有任何关于安全保证的说明的飞机？

19.9 开发机器学习系统 - 信任、可解释性、可说明性

■ 信任——操作、监控和维护

把模型部署给用户，这时将面临更多的挑战。

- 首先要面临用户输入的**长尾** (long tail) 问题。如果系统很受欢迎，你很快就会看到之前从未测试过的输入。你需要知道你的模型是否适用于它们，这意味着需要监控模型在实时数据上的表现。
- 其次，你要面临**非平稳性** (nonstationary) 问题：外部世界会随时间变化。假设你的系统成功分类一批垃圾邮件，垃圾邮件发送者将看到你做了什么，并发送一种你之前没有见过的新类型的邮件；非垃圾邮件也在发展，因为用户改变了他们所使用的电子邮件与短信或桌面与移动服务的组合。

本章小结

- **学习形式**：监督，无监督，强化学习；分类，回归
- **决策树**：决策树学习，决策树剪枝
- **模型选择与优化**：模型选择，损失函数，正则化，超参数
- **计算学习理论**：PAC学习，决策列表
- **线性回归和分类**：线性回归，梯度下降，硬阈值线性分类器，逻辑回归线性分类器
- **非参数化模型**：最近邻模型（k-d树，局部敏感哈希），非参数回归，支持向量机
- **集成学习**：Boosting

历史与趣闻

- 大卫·休谟认识到从样本中泛化可以容许可能出现的错误，他认为没有方法可以给出对某个问题保证正确的解，但是他提出了自然齐一原则，也就是我们现在所称的平稳性。奥卡姆和休谟的观点是，当我们做归纳时，我们将从众多一致的模型中选择一个最可能的模型，因为它更简单且符合我们的期望。在现代，无免费午餐定理表明，如果某个学习算法在某一组问题上表现良好，那只是因为它将在某个不同的问题中表现较差。
- 机器学习是计算机科学诞生的关键思想之一。艾伦·图灵（Turing, 1947）预见到了这一点，他说：“设想一下，我们已经建立了一台带有某些初始指令表的机器，它能在有充足有利条件的情况下，自动地修改这个指令表。”亚瑟·塞缪尔（Samuel, 1959）将机器学习定义为“在不针对计算机编程的情况下，赋予计算机学习能力的一个领域”，并且创建了他的跳棋学习程序

历史与趣闻

- 交叉验证最早由拉森 (Larson, 1931) 提出, 其形式与我们所介绍的斯通 (Stone, 1974) 和戈卢布 (Golub et al., 1979) 提出的形式类似。正则化方法由吉洪诺夫 (Tikhonov, 1963) 提出。关于过拟合的问题, 戴森 (Dyson, 2004) 引用约翰·冯·诺依曼的话宣称, “用 4 个参数我可以拟合一头大象, 用 5 个参数我可以让它摆动躯干”, 意味着可以用高次多项式拟合几乎任何数据, 但代价是可能导致过拟合。迈耶 (Mayer et al., 2010) 通过演示 4 个参数的大象和 5 个参数的摇摆证明了戴森是对的。
- 张弛原等人 (Zhang et al., 2016) 分析了模型在什么条件下能够记住训练数据。他们使用随机数据进行实验, 显然, 在带有随机标签的训练集上获得零错误的算法必须完全记住数据集。然而, 他们认为该领域还没有发现一个精确的衡量标准, 来衡量奥卡姆剃刀意义下的模型“简单”意味着什么。

历史与趣闻

- 贝尔金 (Belkin et al., 2019) 讨论了机器学习中的偏差-方差权衡, 以及为什么一些模型类在达到插值点后仍能继续改进, 而其他模型类则呈现 U 形曲线。贝拉达 (Berrada et al., 2019) 开发了一种基于梯度下降的新学习算法, 该算法利用模型的记忆能力为学习率这一超参数设置良好的值。
- 带平方误差损失的线性回归的研究最早可以追溯到勒让德 (Legendre, 1805) 和高斯 (Gauss, 1809), 他们都致力于预测地球绕太阳运行的轨道。(高斯声称自己从 1795 年起就开始使用这项技术, 只是迟迟没有发表。) 毕晓普 (Bishop, 2007) 等教科书介绍了多变量回归在机器学习中的现代应用。吴恩达 (Ng, 2004) 以及穆尔和德内罗 (Moore and DeNero, 2011) 分析了 L1 和 L2 正则化之间的差异。

历史与趣闻

- 逻辑斯谛函数一词最早由统计学家皮埃尔·弗朗索瓦·韦吕勒（Pierre-françois Verhulst, 1804—1849）使用，他利用该曲线来为有限资源下的人口增长建模，该模型比托马斯·马尔萨斯（Thomas Malthus）所提出的无约束几何增长模型更贴近现实。鉴于该曲线与对数曲线之间的关系，韦吕勒称之为“courbe logistique”（英文为 logistic curve，即逻辑斯谛曲线）。
- 最近邻模型至少可以追溯到菲克斯和霍奇斯的技术报告（Fix and Hodges, 1951），并且从被提出以来它一直是统计和模式识别的标准手段。在人工智能领域，斯坦菲尔和华尔兹（Stanfill and Waltz, 1986）推广了这些方法，他们的研究使得这些方法可以适应数据间的距离。黑斯蒂和蒂施莱尼（Hastie and Tibshirani, 1996）提出了一种方法，根据空间中某个点周围的数据分布，来确定该点的距离度量。乔尼斯等人（Gionis et al., 1999）提出了局部敏感哈希（LSH），彻底改变了对高维空间中相似对象的检索。最近邻这一技术对基因组数据尤其有用，因为基因数据的每条记录都有数百万个属性。

历史与趣闻

- 核机器背后的思想来源于艾泽曼等人 (Aizerman et al., 1964) (核技巧由他提出), 但该理论得以全面发展要归功于瓦普尼克 (Vapnik) 及其同事伯泽尔 (Boser et al., 1992)。一篇获得2008年ACM理论与实践奖的论文 (Cortes and Vapnik, 1995) 对支持向量机引入了软边界分类器, 用于处理带噪声的数据, 普拉特 (Platt, 1999) 的论文还引入了序列最小优化 (SMO) 算法, 从而可以利用二次规划高效地求解支持向量机问题, 它们都使得支持向量机变得更加实用。文本分类 (Joachims, 2001)、计算基因组学 (Cristianini and Hahn, 2007) 以及手写数字识别 (DeCoste and Schölkopf, 2002) 等任务都表明支持向量机是非常有效的。

课后作业

- 假设我们从决策树生成了一个训练集，然后将决策树学习应用于该训练集。当训练集的大小趋于无穷时，学习算法将最终返回正确的决策树吗？为什么是或不是？

THANKS

Q & A

19.1 学习形式

■ 智能体的构成

想象一个智能体做出租车司机，它的功能部件即为：

- **决策：**根据车辆形式状况加速/刹车
- **识别：**识别道路上其他车辆、行人
- **预测：**在不同路况下预测刹车对车速的影响程度
- **效用：**开车过猛会导致乘客负面评价，因此尽量平稳行驶
- **目标：**最小化汽车和目的地的距离

19.3 决策树学习 - 泛化与过拟合

■ 属性相关性度量

- 通过下式，计算 实际的正负样例分布 与 期望的分布 之间的偏差：

$$\Delta = \sum_{k=1}^d \left[\frac{(p_k - \widehat{p}_k)^2}{\widehat{p}_k} + \frac{(n_k - \widehat{n}_k)^2}{\widehat{n}_k} \right] = \sum_{k=1}^d \left[\sqrt{\frac{\widehat{p}_k + \widehat{n}_k}{\widehat{p}_k \widehat{n}_k}} (p_k - \widehat{p}_k) \right]^2$$

- 零假设下，每个样例的分类结果服从一个伯努利分布（以 $\frac{p}{p+n}$ 的概率取值为真），在样本数量足够大的情况下，总正例数量均服从正态分布

$$p_k \sim N\left(\widehat{p}_k, \frac{pn(p_k + n_k)}{(p + n)^2}\right)$$

进而得到 $\sqrt{\frac{\widehat{p}_k + \widehat{n}_k}{\widehat{p}_k \widehat{n}_k}} (p_k - \widehat{p}_k)$ 服从标准正态分布 $N(0,1)$

- 因此 $\Delta = \sum_{k=1}^d \left[\sqrt{\frac{\widehat{p}_k + \widehat{n}_k}{\widehat{p}_k \widehat{n}_k}} (p_k - \widehat{p}_k) \right]^2$ 服从 χ^2 分布（卡方分布），自由度为 $d-1$ （因为有 $\sum_{k=1}^d p_k = p$ 限制，少一个自由度）

19.5 计算学习理论

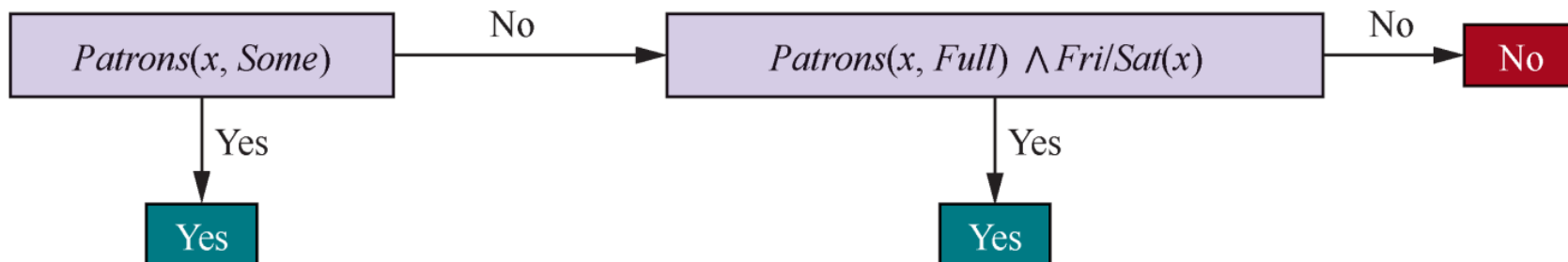
■ 限制假设空间的方法

- 我们需要限制假设空间 H ，同时避免排除掉真实函数，有三种方法可以帮助解决这一问题。
 - ▶ 利用先验知识限制假设空间。
 - ▶ 利用15.3节中介绍的正则化方法，利用正则化函数将训练返回的假设限制在比较简单的范围中，从而限制假设空间。
 - ▶ 第三种方法是找到一种受限的假设空间，该空间中包含与真实函数足够接近的假设 h 。受限的假设空间将更加易于泛化与搜索，接下来我们将研究一种这样的空间：**决策列表**。

19.5 计算学习理论 - 决策列表

■ 决策列表

- 决策列表顺序地对数据进行一系列的测试，每一步的测试都是一些文字的合取。如果数据在某一步中与测试相符，则这一步测试将给出返回的值；否则就前往列表中的下一个测试。
- 决策列表的结构与决策树类似，但永远只有一个分支，相应地，它的每一步测试更加复杂。下图描述了一个餐厅等待问题的决策列表。
- 在不限长度长度的情况下，决策列表将能表示任何布尔函数。通过限制每个测试中包含的文字数量，可以对假设空间进行限制，并从少量样例中成功泛化。



19.5 计算学习理论 - 决策列表

■ 决策列表的假设空间

- 用 k_{DL} 来表示“每个测试最多包含 k 个合取”的决策列表组成的假设空间。
- 首先证明 k_{DL} 是可学习的（即PAC学习 k_{DL} 所需的样例不过大）。
将使用 n 个属性，至多 k 个文字的合取组成的测试集合记为 $Conj(n, k)$ ，总共有 $c = |Conj(n, k)|$ 种测试，这些测试共有 $c!$ 种排列组合方式，每种测试有3种可能输出：yes/no/空（空代表该测试不在决策列表中），因此 k_{DL} 的空间大小上界为

$$|k_{DL}(n)| \leq 3^c c!$$

19.5 计算学习理论 - 决策列表

■ 决策列表的假设空间

- 又由于 $|Conj(n, k)| = \sum_{i=0}^k \binom{2n}{i} = O(n^k)$, 因此可以得到 k_{DL} 的量级为

$$|k_{DL}(n)| \leq 3^c c! = 2^{O(n^k \log_2 n^k)}$$

- 从而可以计算PAC学习 k_{DL} 所需的样本复杂度:

$$N \geq \frac{1}{\varepsilon} (\ln(\frac{1}{\delta}) + O(n^k \log_2 n^k))$$

当 k 较小时, 样本复杂度处于合理数量的范围内。

19.5 计算学习理论 - 决策列表

■ 决策列表学习

现在给出一个可以返回一致性决策列表的有效算法：**Decision-List-Learning** 贪心算法。该算法反复地从样本集中取出部分标签相同的样例子集，并查找一个测试 t 使得这部分样例能通过该测试进行分类。之后，将测试 t 加入决策列表中，并从样本集中删去这部分样例。重复这一过程，直到没有剩余样例为止。

function DECISION-LIST-LEARNING($examples$) **returns** 一个决策列表或 *failure*

if $examples$ 为空 **then return** 平凡的决策列表 No

$t \leftarrow$ 与一个 $examples$ 的非空子集 $examples_i$ 对应的测试，
使得 $examples_i$ 的元素均为正样例或者负样例

if 不存在这样的 t **then return** *failure*

if 集合 $examples_i$ 中的样例均为正 **then** $o \leftarrow$ Yes **else** $o \leftarrow$ No

return 一个初始测试为 t ，输出为 o ，

且由 DECISION-LIST-LEARNING($examples - examples_i$) 给出剩余测试的决策列表

19.8 集成学习 - 随机森林法

■ 随机森林法

- 将自助聚合法应用于决策树时，经常会导致输出多个相关性很高的树：如果存在一个信息增益很高的属性，那么这个属性可能成为大多数树的根。
- 随机森林法：随机化属性选择。
 - ▶ 方案一：在每次分割点选择属性时，我们首先从所有属性中进行抽样，再从抽样出的属性中选择信息增益最高的属性作为测试属性。总共有 n 个属性时，一般随机抽样 $\sqrt{n}$ （分类）个或 $n/3$ （回归）个属性。
 - ▶ 方案二：在选择分割点数值时引入随机性，在选定属性后，从属性的值域中采样几个候选值，并选取信息增益最高的一个。这种方式构造的树称为**极端随机树**。

19.8 集成学习 - 随机森林法

■ 随机森林的性质

- **构建效率高**：虽然需要创建 K 颗树，但引入的随机性会加快分割点处的属性选择，且可以不考虑剪枝，因为随机森林已经具备了剪枝防止过拟合的能力。
- **避免过拟合**：随机森林是复杂的模型，但它可以避免过拟合。布赖曼（Breiman, 2001）给出了一个数学证明，他证明了（几乎在所有情况下）随着向森林中添加更多树，验证集上的错误率趋于收敛，而且它不会增长。

19.8 集成学习 - 在线学习

■ 在线学习

- 在此前的问题中，我们认为所有样例都服从平稳性假设，即它们都是独立同分布(i.i.d)的。
- 本节中，我们将研究当数据不是i.i.d时的情况，我们将采用被称为**在线学习**的概念：模型首先获得输入 x_j 并预测输出 y_j ，随后将正确答案告诉模型，之后模型再对输入 x_{j+1} 做出预测，以此类推。
- 例如，我们使用模型对股市进行预测，将K个专家对股市的预测作为输入，模型汇总这些预测并输出自己的预测。

19.8 集成学习 - 在线学习

■ 随机加权多数算法

研究每个专家过去的表现，并选择与他们过去的表现成比例的方式给予他们相应的信任。

初始化一个权重集合 $\{w_1, \dots, w_K\}$ 全为 1

for each 需要解决的问题 **do**

- (1) 从专家处收到预测 $\{\hat{y}_1, \dots, \hat{y}_K\}$
- (2) 根据权重比例随机选择一个专家 k^* : $P(k) = w_k$
- (3) **yield** $\hat{y}_{k^*}$ 作为专家 k^* 对这个问题的答案
- (4) 收到正确的答案 y
- (5) 对每一个 $\hat{y}_k \neq y$ 的专家 k , 更新 $w_k \leftarrow \beta w_k$
- (6) 归一化权重使得 $\sum_k w_k = 1$

其中 β 是一个常数, $0 < \beta < 1$, 该常数描述的是我们给每一个判断错误的专家添加惩罚的程度

19.8 集成学习 - 在线学习

■ 在线学习的评估

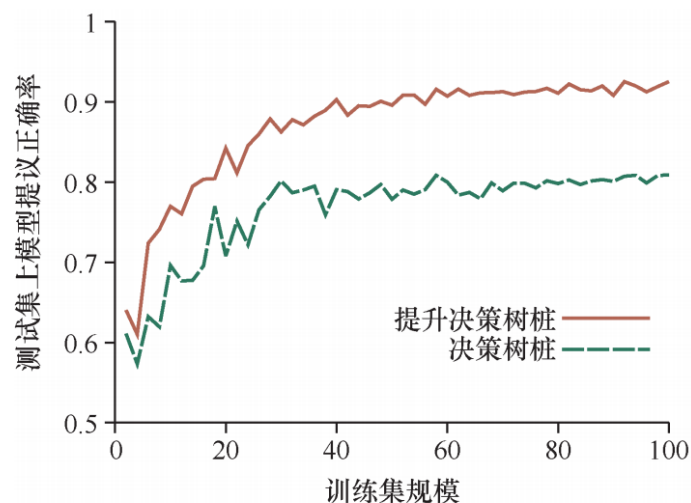
- 如果 β 接近于 1，那么从长远来看，我们将会对数据的变化做出响应；如果预测最好的专家发生了改变，我们不久就会掌握这一变化。但是，我们也会在较长时间中收到坏专家的建议。
- 如果 β 接近于 0，则上述情况恰好相反。
- 用**遗憾**来衡量该算法是否成功。遗憾是指与我们事后知道的拥有最佳预测记录的专家相比，算法所犯的额外错误数量。假设最好的专家预测错误数量为 M^* ，那么随机加权多数算法产生的错误数 M 的上界为

$$M < \frac{M^* \ln(1/\beta) + \ln K}{1 - \beta}$$

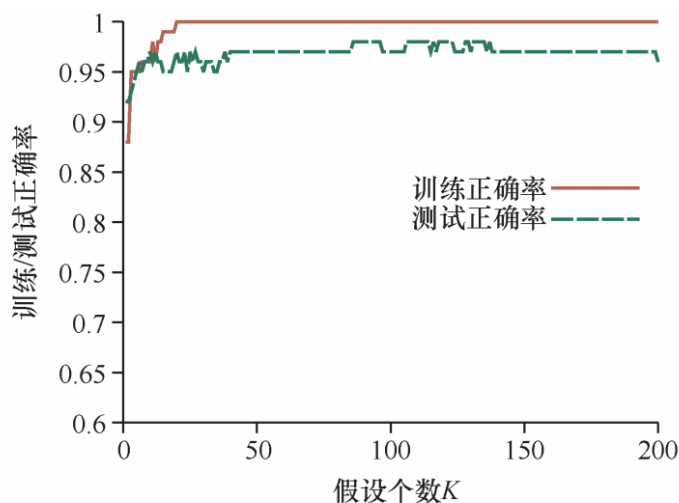
19.8 集成学习 - 自适应提升法

■ 自适应提升法的效果

- **决策树桩：** 仅在根节点处进行一次测试的决策树
- 使用自适应提升法对餐厅等待问题进行训练，左图中，未使用自适应提升法时，决策树桩使用100个样例仅能达到81%准确率，而使用自适应提升法集成 $K = 5$ 个假设后，则将性能提升到了93%。



(a)

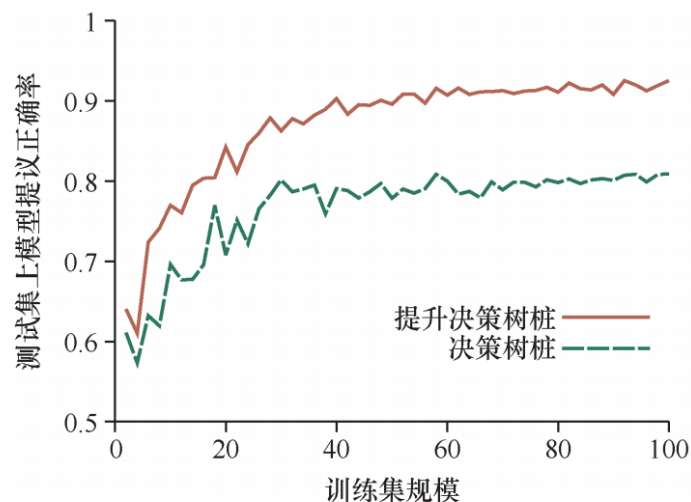


(b)

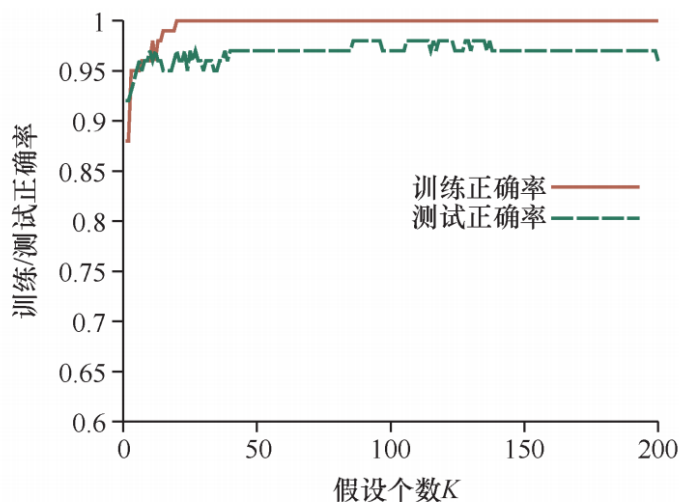
19.8 集成学习 - 自适应提升法

■ 自适应提升法的效果

- 右图中，随着集成规模增加，当达到 $K = 20$ 时，训练误差达到0。但我们发现，继续增大 K ，测试正确率将在一定范围内继续提升，这与奥卡姆剃刀原理恰好相反：预测性能随着整体假设更加复杂而改善。这可能是因为自适应提升法类似于**贝叶斯学习**，也可能是因为新的假设加入使得分类边界的置信度提高。



(a)



(b)

19.8 集成学习 - 梯度提升法

■ 梯度提升法

- 梯度提升法是一种使用了梯度下降的自适应提升法。在梯度提升法中，我们并不关注特定的样例，而注重正确答案与先前假设所给出的答案之间的梯度。
- 在15.6节中，我们使用梯度下降法训练假设，让参数朝着使损失函数下降最快的方向更新。使用梯度提升法时，我们不会更新现有模型的参数，我们更新的是下一个决策树的参数，但是必须通过沿着梯度的方向移动来减小损失。
- 通过学习率 α 来对模型进行正则化，学习率越小意味着我们在集成时需要的决策树越多。